

Technische Universität Berlin

Intelligent Multi-Robot Coordination Lab

Institute of Computer Engineering and Microelectronics



Master's Thesis

**Learning Residual Physics Distributions
of Multirotors with Cable-Suspended Payloads**

Christoph Scherer

Matriculation Number: 474279
27.09.2024

Examiners

Prof. Wolfgang Hönig
Prof. Dr.-Ing. Jörg Krüger

Supervisor

Khaled Wahba

Second supervisor

Kevin Haninger

Eigenständigkeitserklärung

Hiermit versichere ich, dass ich die vorliegende Arbeit eigenständig ohne Hilfe Dritter und ausschließlich unter Verwendung der aufgeführten Quellen und Hilfsmittel angefertigt habe. Alle Stellen die den benutzten Quellen und Hilfsmitteln unverändert oder sinngemäß entnommen sind, habe ich als solche kenntlich gemacht.

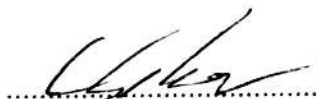
Sofern generative KI-Tools verwendet wurden, habe ich Produktnamen, Hersteller, die jeweils verwendete Softwareversion und die jeweiligen Einsatzzwecke (z.B. sprachliche Überprüfung und Verbesserung der Texte, systematische Recherche) benannt. Ich verantworte die Auswahl, die Übernahme und sämtliche Ergebnisse des von mir verwendeten KI-generierten Outputs vollumfänglich selbst.

Die Satzung zur Sicherung guter wissenschaftlicher Praxis an der TU Berlin vom 15. Februar 2023.

https://www.static.tu.berlin/fileadmin/www/10002457/K3-AMB1/Amtsblatt_2023/Amtliches_Mitteilungsblatt_Nr._16_vom_30.05.2023.pdf
habe ich zur Kenntnis genommen.

Ich erkläre weiterhin, dass ich die Arbeit in gleicher oder ähnlicher Form noch keiner anderen Prüfungsbehörde vorgelegt habe.

Berlin, den 27.9.2024

 Unterschrift

Abstract

Unmanned aerial vehicles (UAVs) such as quadrotors are widely used for tasks that require autonomous operation in confined and dangerous environments, such as surveillance and monitoring tasks, transport of payloads on construction sites or in search and rescue scenarios. However, current state-of-the-art systems for collaborative payload transport with UAVs that employ first-principles models and control algorithms do not take into account higher-order dynamics, such as cable vibrations. As a result, they do not provide the stability and robustness necessary to operate in such environments. Improving the models to facilitate this is still an open research topic. Augmenting the first-principles models used in flight controllers in a hybrid manner with data-driven methods to account for unmodeled higher-order dynamics has been shown to improve flight stability. Learning residual dynamics through residual forces is particularly promising. As most data-driven methods suffer from potentially poor generalization in unknown conditions, it is crucial to learn the distributions of trained data for safe operation in a multi-UAV payload system (MPS). Generative models have been shown to be particularly effective tools for learning such distributions. This thesis tries to explore the applicability of variational autoencoders (VAEs) and diffusion models as particularly promising instances of generative models to learn the distributions of residual forces, to gain additional information about the underlying distributions in the state of a given system. Using real-world flight data provided for this thesis, a nearest-neighbor search is used to find residual distributions conditioned on information such as the state of the system using a flexible metric that accommodates varying amounts of conditioning information. The models were compared on several metrics including their ability to capture residual distribution and computational efficiency for real-time applications. The results show that diffusion models outperform VAEs capturing residual distributions offering the higher prediction accuracy and the ability to estimate the prediction uncertainty based on the sampling variance. However, for real-time applications, the supported sampling rate may be insufficient. Additional experiments analyzed the effects of different conditioning features, data smoothing, and generalization for teams of two and three UAVs. The tests revealed that the system state derivatives significantly affect the prediction quality, suggesting that consecutive states are best modeled by a Markov chain with memory. This thesis demonstrates that generative models, particularly diffusion models, hold promise in improving the MPS in terms of robustness and stability, provided that the sampling rate is sufficient for the respective application.

Zusammenfassung

Unbemannte Luftfahrzeuge wie Quadroptoren werden häufig für Aufgaben eingesetzt, die Autonomie in engen und gefährlichen Umgebungen erfordern, wie Überwachungs- und Monitoringaufgaben, den Transport von Lasten auf Baustellen oder in Such- und Rettungseinsätzen. Allerdings berücksichtigen aktuelle, hochmoderne Systeme für den kollaborativen Lastentransport mit Quadroptoren, die auf physikalischen Modellen und Steuerungsalgorithmen basieren, keine Dynamiken höherer Ordnungen, wie z.B. Kabelschwingungen. Da sie dadurch nicht die für diese Zwecke nötige Stabilität und Robustheit bieten, ist deren Verbesserung ein offenes Forschungsthema. Das Verbessern der in Flugsteuerungen verwendeten physikalischen Modelle mit datengetriebenen Methoden in hybriden Modellen, um nicht modellierte Dynamikeffekte zu berücksichtigen, hat sich als vielversprechend zur Verbesserung der Flugstabilität erwiesen. Insbesondere das Erlernen von Residualdynamiken durch Residualkräfte ist verheißungsvoll. Da die meisten datengetriebenen Methoden in unbekannten Bedingungen eine potenziell schlechte Generalisierung aufweisen, ist es wichtig, die Verteilungen der Trainingsdaten für einen sicheren Betrieb in einem kollaborativen Quadroptor Transportsystem (MPS) zu erlernen. Generative Modelle haben sich als besonders effektive Werkzeuge für das Erlernen solcher Verteilungen erwiesen. Diese Arbeit untersucht die Anwendbarkeit von *Variational Autoencodern* (VAE) und *Diffusionsmodellen* als besonders vielversprechende Beispiele für generative Modelle, um die Verteilungen von Residualkräften zu erlernen und zusätzliche Informationen über deren Verteilungen im aktuellen Zustand eines Systems zu gewinnen. Anhand von realen Flugdaten, die für diese Arbeit bereitgestellt wurden, wird eine *Nearest-Neighbor-Suche* verwendet, um Residualkraftverteilungen basierend auf Informationen wie dem Zustand des Systems zu finden. Dafür wird eine flexible Metrik, die eine unterschiedliche Anzahl an Informationen des Systemzustands berücksichtigt. Die Modelle wurden anhand mehrerer Metriken verglichen, darunter ihre Fähigkeit, Residualkraftverteilungen zu erfassen, sowie ihre Recheneffizienz für Echtzeitanwendungen. Die Ergebnisse zeigen, dass Diffusionsmodelle VAE in ihrer Erfassung und Vorhersagegenauigkeit von Residualkraftverteilungen übertreffen, und über die Fähigkeit verfügen die Vorhersageunsicherheit basierend auf der Varianz der generierten Residualkräfte abzuschätzen. Die geringere unterstützte Abtastrate der Diffusionsmodelle könnte jedoch für Echtzeitanwendungen unzureichend sein. Zusätzliche Experimente analysierten die Auswirkungen verschiedener Bedingungsmerkmale, Glättung der gesammelten Flugdaten und der Generalisierung für Teams aus zwei und drei Quadroptoren. Die Tests zeigten, dass die Ableitungen des Systemzustands die Vorhersagequalität erheblich beeinflussen, was darauf hindeutet, dass aufeinanderfolgende Zustände am besten durch eine Markovkette mit Gedächtnis modelliert werden können. Diese Arbeit zeigt, dass generative Modelle, insbesondere Diffusionsmodelle, vielversprechend sind, um das MPS in Bezug auf Robustheit und Stabilität zu verbessern, vorausgesetzt, dass die Abtastrate ausreicht.

Supplementary material

All code developed for the purpose of this thesis is available under
https://github.com/c-scherer/ms_thesis_scherer

Acknowledgments

I would like to express my gratitude to everyone who supported me during the development of this thesis.

I also acknowledge the use of the following tools for the purpose of this thesis:

- **Python**: For data analysis and machine learning experiments.
- **PyTorch**: For implementing of the generative models developed.
- **SciKit-learn**: For data preprocessing and unsupervised learning algorithms.
- **SciPy**: For data preprocessing.
- **ChatGPT (GPT-4o) by OpenAI**: For help with phrasing and translation.
- **Writefull for Overleaf (version 2024.111.0) by Writefull**: For improving spelling and grammar.

Contents

1	Introduction	1
1.1	Motivation	1
1.2	Goals	2
2	Related work	5
3	Background	7
3.1	First principle models	7
3.1.1	Multirotor dynamics and Residual dynamics	7
3.1.2	Dynamics of the multi-UAV payload system	8
3.2	Metrics	9
3.2.1	State space metric	10
3.2.2	Minkowski distances	10
3.2.3	Standardized euclidean distance	11
3.2.4	Maximum mean discrepancy	11
3.3	Machine learning	12
3.3.1	Nearest-neighbor search	12
3.3.2	Principal component analysis	13
3.3.3	Radial-basis function	13
3.3.4	Generative models	14
3.4	Markov processes	14
4	Residual dynamics for the multi-UAV payload System	17
4.1	Residual force calculations	17
4.2	Markov assumption for the multi-UAV payload model	19
4.2.1	Modeling the dynamics as a first-order system	19
4.2.2	Residual forces as random variables	20
4.3	Effect of the state space conditioning on the stochastic process	20
5	Datasets	23
5.1	Test flight data	23
5.2	Pre-processing	24
5.2.1	Interpolation using curve-fitting splines	25
5.2.2	Synchronization	29
5.2.3	Residual calculation and dataset extraction	32
5.3	Selection of conditioning features	32

6	Generative models for predicting residual forces	37
6.1	General architecture	37
6.2	Training process	38
6.3	Autoencoders	38
6.3.1	Variational autoencoders	39
6.3.2	Implementation	40
6.3.3	Variational diffusion models	42
6.3.4	Implementation	43
7	Validation metrics and visualization tools	45
7.1	Mean squared error	45
7.2	Calculating maximum mean discrepancy loss on residual distributions	46
7.3	Calculating the average inference time	47
7.4	Qualitative analysis and plots	47
8	Ablation studies	49
8.1	Choice of final hyperparameters	50
8.2	Results	50
8.2.1	Variational autoencoder	50
8.2.2	Diffusion model	52
9	Evaluation	55
9.1	General comparison of the generative models	55
9.1.1	Impact on the general performance	55
9.1.2	Residual distributions	57
9.2	Comparison of the effects of team sizes	59
9.3	Effect of smoothing	61
9.3.1	Impacts of smoothing on overall performance	61
9.3.2	Effect on residual distributions	66
9.4	Selection of conditioning features	67
9.4.1	Analysis using non-smoothed data	67
9.4.2	Effect on residual distributions	71
9.4.3	Analysis using smoothed data	73
9.5	Comparison of different residual force components	78
9.6	Discussion	78
10	Conclusion	83
10.1	Summary	83
10.2	Key findings	84
10.3	Outlook	85
	Bibliography	87

Appendix	93
1 Ablation studies tables	93
2 Crazyflie and payload parameters	101

List of Abbreviations

CFG	classifier-free guidance
CG	classifier guidance
CVAE	conditional variational autoencoder
DNN	deep neural network
ELBO	evidence lower bound
FFT	fast Fourier transform
k-NN	k-Nearest Neighbors
kernel PCA	kernel principal component analysis
KL divergence	Kullback-Leibler divergence
LeakyReLU	leaky rectified linear unit
LLM	large language models
MMD	maximum mean discrepancy
MPS	multi-UAV payload system
MSE	mean squared error
PCA	principal component analysis
RBF	radial basis function
RKHS	reproducing kernel Hilbert space
SED	scaled Euclidean distance
STD	standard deviation
UAV	unmanned aerial vehicle
VAE	variational autoencoder

1 Introduction

1.1 Motivation

Unmanned aerial vehicles (UAVs) provide maneuverability and versatility, which makes them, combined with their low cost, ideal for autonomous system platforms in many aspects of robotics research. They are also well suited for tasks that require autonomous operation in hard-to-reach or hazardous locations like construction sites or for carrying payloads. Multi-robot systems such as multiple UAVs collaboratively transporting a payload allow for even more flexibility, making them perfect candidates in search and rescue and other scenarios in confined spaces. As these applications require precise motion planning and control capabilities of such a multi-robot system, it is crucial to have an accurate dynamical model. Current state-of-the-art approaches for cable-driven payload with multiple UAVs system (which will be called MPS for brevity) rely solely on traditional first-order principle models. These models, while successful under ideal conditions, fail to account for higher-order dynamics such as interacting cable vibrations or aerodynamic effects such as drag and interaction forces between multiple UAVs. However, for use in real, uncertain environments, it is crucial to improve the robustness by developing more accurate models.

In recent years, different approaches have emerged to increase the accuracy of dynamic systems by incorporating machine learning models. Approaches relying on the estimation of model parameters are well-studied and widely used in control theory. However, the approach is limited in capturing complex and unmodeled effects of the selected model. Purely data-driven methods are suitable for learning complex dynamics. However, they impose strong requirements for large amounts of high-quality training data, which is difficult to obtain from real-world experiments. Although simulations can be used for data collection, transferring learned models on simulation data to real-world applications imposes challenges known as Sim-to-Real transfer, as they also fail to capture the full complexity of real environments [1, 2].

As a result, hybrid approaches to combine nominal first-principle models and data-driven models were developed, in an attempt to mitigate the weaknesses of single-paradigm approaches [3, 4, 5]. A particularly promising approach is the learning of residual forces, as the collected forces of all unmodeled and higher-order dynamics. Models enhanced by the learned residual forces have been shown to capture complex interaction forces between multiple multirotor UAVs and improve flight stability [3, 4]. In contrast to purely data-driven approaches, analyzing these residual forces also allows to gain insight about the complex effects, which could be helpful in better understanding the limits of a given system. In the context of the MPS, research has shown that

1 Introduction

effects such as cable-induced forces are difficult to model using first principles alone but strongly affect the stability of the system [6, 7]. Other effects like aerodynamic interactions of UAVs during uncertain or aggressive maneuvers that are currently not taken into account limit the model’s capabilities further. Therefore, an accurate model for predicting residual forces promises substantial improvements in this regard.

A problem associated with data-driven methods is the limited understanding of the inner workings of the models. Purely regressive models for the prediction of residual forces do not provide information on the uncertainty of the prediction and other characteristics of the distribution of residual forces. As a consequence, compensating for residual forces of the high prediction uncertainty could endanger the system. Consequently, knowing the associated residual distributions for a particular state can offer significant insights for various scenarios where model uncertainty data is beneficial. This knowledge can be applied to improve flight controllers, assist in decision making or motion planning, and improve the safety and stability of a given system.

The central problem therefore is how to model the complex dynamics of the MPS including the unpredictable residual dynamics that are not fully captured by existing methods. Understanding the probabilistic distribution of these residual dynamics is essential for improving the system’s robustness, accuracy, and safety.

1.2 Goals

Generative models have gained significant popularity with recent advances in deep learning due to their ability to learn complex underlying distributions of data and generate new samples from those distributions. Especially VAEs and diffusion models have proven their ability and associated strengths and weaknesses. This thesis explores the general applicability of these two generative models for learning probability distributions of residual forces to be used to enhance current state-of-the-art models and to gain insight into the higher-order dynamics that limit the capability of current systems. This involves deriving functions that capture the residual dynamics. This thesis proposes methods for deriving residual forces for the payload and for each UAV in the MPS. Residual torques for each UAV are also derived for completeness, but their analysis is beyond the scope of this thesis.

The core of this work involves benchmarking diffusion models and VAEs regarding their ability to replicate distributions of residual forces and a comparison regarding their feasibility for real-world applications. Further evaluations will be performed to analyze the effects of using different conditioning features, as well as using smoothed data in contrast to non-smoothed data. To identify the optimal hyperparameters for the generative models ensuring a fair comparison, ablation studies are performed. For the benchmarks, it is essential to define suitable metrics as quantitative measures of the model’s performance that allow for a broader comparison highlighting the strengths and weaknesses of each model. An additional state space metric is defined to quantify the similarity between states of the MPS that allows to approximate residual distributions for a given state using nearest-neighbor search. As the current understanding of resid-

ual dynamics is limited, exploring the underlying residual distributions of the training dataset, suitable visualization tools help to gain insight into the higher-order dynamics of the MPS and other influences on the residual distribution. Because of this, the development of such tools to visualize and analyze these distributions effectively is part of this thesis.

2 Related work

In recent years, several advances in **multirotor** control have led to the proposition and research of increasingly complex systems involving both load transport and multi-robot systems [6, 7, 8, 9].

Approaches for single **UAV** transporting a payload using models derived from Lagrangian mechanics have been demonstrated and validated, however, given the limited actuation of **UAVs** they impose challenges in control like the swinging of the payload and disturbances, not allowing for agile maneuvers [7]. The added degrees of freedom imposed without direct actuation capability of the payload are essentially a pendulum stabilization problem that needs to be addressed in addition to the already challenging control of the **UAV** itself [6].

One solution to the problem of swinging payloads is carrying the load by a team of multiple **UAVs**, allowing for more controlled manipulation of the payload and increasing the payload capacity [6]. Early research on such multi-robot systems have shown, that the physical connection via cables introduce non-trivial interactions between **UAV**, that relied on simplification of the complicated dynamical models, to the point of the neglect of coupling effects [10]. Current control and planning systems incorporating the coupled dynamics have been shown to be stable ([9]) or were tested in real experiments [8, 11].

However, even though one advantage of using the **MPSS** over single **UAV** systems carrying payloads is their theoretical capability of performing more aggressive maneuvers, such experiments have not been published yet, possibly due to the inaccuracies in the complex dynamical systems of **MPSS**, such as not accounting for hard to accurately predict aerodynamic effects.

Multirotor swarms without coupling as in a payload transport scenario have already been shown to experience strong aerodynamic interaction forces such as downwash or ground effect [4]. In efforts to account for these effects in planning and control, learning the residual dynamics of dynamic models and incorporating them in these systems have been shown to significantly improve performance for single **UAV** scenarios ([5, 12]) as well as for swarms of **UAVs** [4]. However, learning of residual dynamics or other data-driven approaches to improve the performance of **cable-driven payload with multiple UAVs systems** has not yet been carried out.

3 Background

3.1 First principle models

Physics-based dynamic models of robot systems can be derived using Newton's equations of motion or Lagrangian mechanics, yielding systems of typically nonlinear second-order differential equations accounting for the different degrees of freedom of the robot system [13, p.59-63]. Such models are important for several applications in robotics, as they allow for the design of control systems and numerical simulation of the system considered [13, p.62].

These systems of equations can be written in the generic form of

$$\mathbf{f}(\mathbf{q}, \dot{\mathbf{q}}, \ddot{\mathbf{q}}, \boldsymbol{\eta}) = \mathbf{0} \quad (3.1)$$

where $\mathbf{q}, \dot{\mathbf{q}}, \ddot{\mathbf{q}}$ are vectors of generalized variables for the system's degrees of freedom and their respective derivatives, and $\boldsymbol{\eta}$ is the vector of force and torque inputs called *wrench* [13, p.62].

3.1.1 Multirotor dynamics and Residual dynamics

The dynamics of a swarm of n multirotors without any attached payload can be modeled as floating rigid bodies under the influence of the gravitational force with 6 degrees of freedom each, as developed by Shi et al. [3] and will be referenced in this section. Each UAV $i \in \{1, \dots, n\}$ is parameterized by its associated mass m_i and moment of inertia $\mathbf{J}_i$. The manifold of the configuration space is defined by $\mathcal{C}_{mps} = (\mathbb{R}^3 \times SO(3))^n$. At any given time the state of the multirotor is given by $\mathbf{x}_i = (\mathbf{p}_i, \dot{\mathbf{p}}_i, \mathbf{R}_i, \boldsymbol{\Omega}_i)^\top$. Here $\mathbf{p}_i \in \mathbb{R}^3$ is the position and $\dot{\mathbf{p}}_i \in \mathbb{R}^3$ the velocity of the UAV in the world frame. The rotation of the UAV can be described by its rotation matrix $\mathbf{R}_i \in SO(3)$ with its associated body angular velocity $\boldsymbol{\Omega}_i \in \mathbb{R}^3$. The operator $(\cdot)$ denotes the skew-symmetric mapping. Furthermore, g is the gravitational constant and $\mathbf{e}_3 = (0, 0, 1)^\top \in \mathbb{R}^3$. The collective thrust and torques $\mathbf{M}_i = (\tau_{xi}, \tau_{yi}, \tau_{zi})^\top$ of the UAV is given by the wrench $\boldsymbol{\eta}_i = (f_{ci}, \mathbf{M}_i)^\top$. The wrench vector can then be obtained from the motor force control inputs $\mathbf{u}_i = (f_{1i}, f_{2i}, f_{3i}, f_{4i})^\top$ and the actuation matrix $\mathbf{B}_0$ by linear mapping $\boldsymbol{\eta}_i = \mathbf{B}_0 \mathbf{u}_i$.

Defining the collected residual forces $\mathbf{f}_{ai}$ and residual torques $\boldsymbol{\tau}_{ai}$ account for unmodeled effects such as disturbance forces and torques and other model mismatches.

The dynamics can then be described by the system of equations [3].

3 Background

$$m_i(\ddot{\mathbf{p}}_i + g\mathbf{e}_3) = f_{ci}\mathbf{R}_i\mathbf{e}_3 + \mathbf{f}_{ai} \quad (3.2a)$$

$$\dot{\mathbf{R}}_i = \mathbf{R}_i\dot{\hat{\boldsymbol{\Omega}}}_i \quad (3.2b)$$

$$\mathbf{J}_i\dot{\boldsymbol{\Omega}}_i = \mathbf{J}_i\boldsymbol{\Omega}_i \times \boldsymbol{\Omega}_i + \mathbf{M}_i + \boldsymbol{\tau}_{ai} \quad (3.2c)$$

Introducing the collected residual forces and residual torques to the equations of equilibrium of forces in Equation 3.2a and equilibrium of torques in Equation 3.2c, a full dynamic model for a UAV can be derived that can be used to calculate residual forces and residual torques from test flight data.

Given complete information on the state $\mathbf{x}$, its derivative $\dot{\mathbf{x}}$ and wrench $\boldsymbol{\eta}$, residual forces $\mathbf{f}_{ai}$ and residual torques $\boldsymbol{\tau}_{ai}$ can be calculated by the following:

$$\mathbf{f}_{ai} = f_{ci}\mathbf{R}_i\mathbf{e}_3 - m_i(\ddot{\mathbf{p}}_i - g\mathbf{e}_3) \quad (3.3a)$$

$$\boldsymbol{\tau}_{ai} = \mathbf{J}_i\boldsymbol{\Omega}_i \times \boldsymbol{\Omega}_i + \mathbf{M}_i - \mathbf{J}_i\dot{\boldsymbol{\Omega}}_i \quad (3.3b)$$

Using the derivative of the state $\dot{\mathbf{x}}_i = (\dot{\mathbf{p}}_i, \ddot{\mathbf{p}}_i, \dot{\mathbf{R}}_i, \dot{\boldsymbol{\Omega}}_i)^\top$ allows the formulation of the Equations 3.3 as functions of the state:

$$\mathbf{f}_{ai} = g_{a,\text{scs}}(\mathbf{x}_i, \dot{\mathbf{x}}_i, \boldsymbol{\eta}_i) \quad (3.4a)$$

$$\boldsymbol{\tau}_{ai} = g_{\tau,\text{scs}}(\mathbf{x}_i, \dot{\mathbf{x}}_i, \boldsymbol{\eta}_i) \quad (3.4b)$$

3.1.2 Dynamics of the multi-UAV payload system

As for any model, the dynamic model of the MPS is derived under assumptions that try to find a balance between accuracy and model complexity. This thesis adopts the dynamic model and notation developed by Wahba et al. [14], which this section references. Here the payload is modeled as a point mass, connected by cables to the center of gravity of each UAV. The cables are modeled as rigid rods under the assumption, that the cables never experience slack. The position of the payload is given by $\mathbf{p}_0$ and its velocity by $\dot{\mathbf{p}}_0$. The unit vectors of the cables and their angular velocity are denoted by $\mathbf{q}_i \in \mathbb{S}^2$ with $\{\mathbf{q}_i \in \mathbb{R}^3 \mid \|\mathbf{q}_i\| = 1\}$ and $\boldsymbol{\omega}_i \in \mathbb{R}^3$, respectively, and point from the UAV to the payload. The assumption of cables acting as rigid rods allows for the computation of the positions of UAVs using

$$\mathbf{p}_i = \mathbf{p}_0 - l_i\mathbf{q}_i \quad (3.5)$$

The configuration space of the system can therefore be described by the manifold $\mathcal{C}_{\text{mps}} = \mathbb{R}^3 \times (\mathbb{S}^2 \times SO(3))^n$. The complete dynamics and kinematics can then be described by the following system of equations [14].

$$\dot{\mathbf{q}}_i = \boldsymbol{\omega}_i \times \mathbf{q}_i \quad (3.6a)$$

$$\mathbf{M}_t(\ddot{\mathbf{p}}_0 + g\mathbf{e}_3) = \sum_{i=1}^n (f_{ci}\mathbf{R}_i\mathbf{e}_3 - m_i l_i \|\boldsymbol{\omega}_i\|^2 \mathbf{q}_i) \quad (3.6b)$$

$$m_i l_i \dot{\boldsymbol{\omega}}_i = m_i \hat{\mathbf{q}}_i(\ddot{\mathbf{p}}_0 + g\mathbf{e}_3) - f_{ci} \hat{\mathbf{q}}_i \mathbf{R}_i \mathbf{e}_3 \quad (3.6c)$$

$$\dot{\mathbf{R}}_i = \mathbf{R}_i \hat{\boldsymbol{\Omega}}_i \quad (3.6d)$$

$$\mathbf{J}_i \dot{\boldsymbol{\Omega}}_i = \mathbf{J}_i \boldsymbol{\Omega}_i \times \boldsymbol{\Omega}_i + \mathbf{M}_i \quad (3.6e)$$

The equilibrium of moments applied at the center of gravity of each UAV is unaffected by the added payload because of our assumption of cables attached exactly to the center of gravity of the payload and UAV. Therefore, there are no additional moments induced via cable forces, yielding the same equilibrium of moments as in Equation 3.2c.

The combined state of the MPS is then composed of the states of each UAV $\mathbf{x}_i$ with its associated cable vector, their wrench $\boldsymbol{\eta}_i$ and the position and velocities of the payload by

$$\mathbf{x}_i = (\mathbf{p}_i, \dot{\mathbf{p}}_i, \mathbf{q}_i, \mathbf{R}_i, \boldsymbol{\omega}_i, \boldsymbol{\Omega}_i)^\top \quad (3.7a)$$

$$\mathbf{x} = (\mathbf{p}_0, \dot{\mathbf{p}}_0, \mathbf{x}_1, \dots, \mathbf{x}_n)^\top \quad (3.7b)$$

$$\boldsymbol{\eta} = (\boldsymbol{\eta}_1, \dots, \boldsymbol{\eta}_n)^\top \quad (3.7c)$$

Similarly, the derivative of the state is given by

$$\dot{\mathbf{x}}_i = (\dot{\mathbf{p}}_i, \ddot{\mathbf{p}}_i, \dot{\mathbf{q}}_i, \dot{\mathbf{R}}_i, \dot{\boldsymbol{\omega}}_i, \dot{\boldsymbol{\Omega}}_i)^\top \quad (3.8a)$$

$$\dot{\mathbf{x}} = (\dot{\mathbf{p}}_0, \ddot{\mathbf{p}}_0, \dot{\mathbf{x}}_1, \dots, \dot{\mathbf{x}}_n)^\top \quad (3.8b)$$

3.2 Metrics

To quantify similarities and differences between points of a set, distance functions or metrics are tools often used as they provide a measure of distance between any given points that satisfy certain conditions. Especially metrics have important properties, making them important tools for numerous applications in mathematics. In mathematical literature ([15]) the term distance is defined by a function $d : X \times X \rightarrow \mathbb{R}$ on a set X if d holds for all $x, y \in X$:

3 Background

$$d(x, y) \geq 0 \quad \text{non-negativity} \quad (3.9a)$$

$$d(x, y) = d(y, x) \quad \text{symmetry} \quad (3.9b)$$

$$d(x, x) = 0 \quad \text{reflexivity} \quad (3.9c)$$

The combination (X, d) of the set X and distance d is also called Metric space [15]. Furthermore, if the following properties hold for all $x, y, z \in X$:

$$d(x, y) \geq 0 \quad \text{non-negativity} \quad (3.10a)$$

$$d(x, y) = d(y, x) \quad \text{symmetry} \quad (3.10b)$$

$$d(x, y) = 0 \iff x = y \quad \text{identity of indiscernibles} \quad (3.10c)$$

$$d(x, y) \leq d(x, z) + d(z, y) \quad \text{triangle inequality} \quad (3.10d)$$

the function d is also called a metric or distance metric, and the space (X, d) is called Metric space [15].

3.2.1 State space metric

Clustering techniques such as nearest neighbor search rely on a distance metric that is used to quantify the distance between points. Thus, to group states within the cable-driven payload with multiple UAVs system, a state space metric must be established to measure the distance between states accurately. Consequently, the selection of this metric has a significant influence on clustering algorithm outcomes, which is especially challenging in high dimensional contexts [16]. Common choices for configuration metric in robotics for tasks such as motion planning involve the use of the Euclidean distance by considering the configuration space as a Cartesian space and normalization of some configuration space features [17, pp.313-314].

3.2.2 Minkowski distances

One of the most popular family of distances on $\mathbb{R}^n$ in machine learning and other applications dealing with high-dimensional data are L_k distances [16]. For values $k \in \mathbb{Z} \geq 1$ they satisfy the metric axioms of Equation 3.10 and the distance between $\mathbf{x}, \mathbf{y} \in \mathbb{R}^n$ can be calculated as:

$$d_{lk}(\mathbf{x}, \mathbf{y}) = \left(\sum_{i=1}^n |x_i - y_i|^k \right)^{\frac{1}{k}} \quad (3.11)$$

[17, p.301]. These distances are sometimes also called *Minkowski distances* and are equivalent to the L_p norm $\|\mathbf{x} - \mathbf{y}\|_p$ for $p = k$ [18]. Of special importance are the instances L_1 , also called *Manhattan metric*, the *Euclidean distance* L_2 and the *Infinity distance* or *Chebyshev distance* L_∞ [17, p.301].

3.2.3 Standardized euclidean distance

A relative of the *Euclidean distance* is the **scaled Euclidean distance (SED)**, which is used for *heterogeneous data sets* with individual variances σ_i^2 for each dimension $i \in \{1, \dots, n\}$ of points $\mathbf{x}, \mathbf{y} \in \mathbb{R}^n$

$$d_{\text{SED}}(\mathbf{x}, \mathbf{y}) = \sqrt{\sum_{i=1}^n \frac{(x_i - y_i)^2}{\sigma_i^2}} \quad (3.12)$$

and is a special case of the Mahalanobis distance

$$d_A(\mathbf{x}, \mathbf{y}) = \sqrt{(\mathbf{x} - \mathbf{y})^T \mathbf{A}^{-1} (\mathbf{x} - \mathbf{y})} \quad (3.13)$$

if the covariance matrix $\mathbf{A}$ is a diagonal matrix [17, p.303].

The **SED** is also equivalent to the *Euclidean distance* used with data sets $\mathcal{D}$ after a performed standardization

$$\tilde{x}_i = \frac{x_i - \mu_i}{\sigma_i} \quad (3.14)$$

with the mean $\mu_i = \frac{1}{n} \sum_{j=1}^n x_i^{(j)}$ for $\mathbf{x}^{(j)} \in \mathcal{D}$. Many applications in machine learning require standardization of data sets and therefore implicitly use the **SED** [19].

3.2.4 Maximum mean discrepancy

Maximum mean discrepancies (MMDs) is a metric that is often used in statistics and machine learning for the evaluation of generative model. It can serve as a distance measurement between samples from probability distributions and can therefore be used as a loss function for training of machine learning models [20]. It is also possible to derive a two-sample test to test the null hypothesis that $p = q$ given observations of two distributions $X = \{x_1, \dots, x_m\}$ of $p \in \mathcal{X}$ and $Y = \{y_1, \dots, y_n\}$ of $q \in \mathcal{X}$ against the alternative hypothesis of $p \neq q$ [21].

The basic steps for calculating the **maximum mean discrepancy (MMD)** involve the following: Both distributions are mapped in a universal **reproducing kernel Hilbert space (RKHS)** $\mathcal{H}$ using a feature map $\phi : \mathcal{X} \rightarrow \mathcal{H}$. Their mean embedding $\mu_p \in \mathcal{H}$ and $\mu_q \in \mathcal{H}$ can then be calculated using a reproducing kernel function $k(x, y) = \langle \phi(x), \phi(y) \rangle_{\mathcal{H}}$ [22]. Finally, we can yield the **MMD** as a distance of the mean embeddings.

This is equivalent to the formal definition of **MMD** as a measure of disparity between p and q as an integral probability metric [23]

$$\text{MMD}[\mathcal{F}, p, q] = \sup_{f \in \mathcal{F}} (\mathbb{E}_{x \sim p}[f(x)] - \mathbb{E}_{y \sim q}[f(y)]) \quad (3.15)$$

where $\mathcal{F}$ is a class of functions $f : \mathcal{X} \rightarrow \mathbb{R}$ [21]. It can be shown that **MMD** is a metric when $\mathcal{H}$ is a universal **RKHS** on a compact metric space $\mathcal{X}$, which requires continuous $k(\cdot, \cdot)$ and $\mathcal{H}$ being dense in $C(\mathcal{X})$ with respect to the L_∞ norm [21].

3 Background

Given $\mathcal{H}$ being a universal RKHS it is possible to derive an empirical estimate of the MMD given by Equation 3.15 with

$$\text{MMD}_{[\mathcal{F}, X, Y]} = \left[\sum_{i=1}^m \sum_{j \neq i}^m \frac{k(x_i, x_j)}{m(m-1)} - 2 \sum_{i=1}^m \sum_{j=1}^n \frac{k(x_i, y_j)}{mn} + \sum_{i=1}^n \sum_{j \neq i}^n \frac{k(y_i, y_j)}{n(n-1)} \right]^{\frac{1}{2}} \quad (3.16)$$

using the reproducing kernel function $k(x, y) = \langle \phi(x), \phi(y) \rangle_{\mathcal{H}}$ [22].

A major advantage of this nonparametric approach is supposed to be a good choice for high-dimensional settings by circumventing the explicit calculation of density estimations, which is difficult in high-dimensional settings. However, MMD and other kernel-based hypothesis tests also suffer from the curse of dimensionality and the corresponding decreasing power of expressiveness in high dimensions [24].

3.3 Machine learning

Machine learning is a field of artificial intelligence that includes a collection of algorithms using training data for solving complex problems like regression, classification, pattern recognition or the generation of new data with generative modeling [25, p.3-4]. Commonly, machine learning algorithms are classified using either supervised learning, unsupervised learning or reinforcement learning, however, recent advances in generative modeling use self-supervised learning [25, p.3-5]. Supervised learning uses data with predetermined labels where each point of input data has a corresponding output label. Algorithms that rely on supervised learning are often used for tasks like regression or classification [26, p.4]. Unsupervised learning deals with unlabeled data uses pattern recognition to solve different problems [26, p.4]. Typical applications include clustering, principal component analysis (PCA) or anomaly detection [27].

3.3.1 Nearest-neighbor search

Often when dealing with high-dimensional data an important problem to solve is the so-called *closest-point problem* or *post-office problem*: Find the most similar point $p \in \mathbb{R}^d$ for a given query $q \in \mathbb{R}^d$ and set $S \in \mathbb{R}^d$ of n points. [28]. The similarity between points is typically measured using a *Minkowski* metric. A popular and efficient implementation involves the construction of KD-Trees.

KD-Tree

According to Jeffers, Reinders, and Sodani, KD-Trees are data structures that arrange points of a given set in a k -dimensional space by their distance with respect to a specific metric, that can be explained in detail by as follows: The data gets partitioned at each node in a way such that close points belong to the same partition. Each node can therefore be seen as a hyperplane that splits the space with a roughly equal number

of points in each partition. The data is split along a single dimension for each node, cycling through all k dimensions of the given space. To ensure that each node splits the data roughly in half, the split gets performed at the approximated median point regarding the dimension of the split. KD-Trees are an efficient data structure for finding k -Nearest Neighborss (k -NNs) given a query point and a search radius. The key idea of it is being able to search more efficiently is to avoid evaluating the metric between points. We achieve this by keeping track of an upper bound of the distance of the k -NN and comparing it with the bounding box of each node. This allows subtree pruning if all possible points in a given bounding box would be farther than the upper bound, effectively reducing the complexity of distance calculations from linear to logarithmic. Although KD-Tree can be computationally expensive to construct, querying points is relatively fast. Once built, it is possible to efficiently query many points for different search radii or k -NN [29, pp.539-540].

3.3.2 Principal component analysis

One of the most important unsupervised learning tools used in machine learning and other high-dimensional scenarios is PCA [30, p.5445]. Two important applications include dimensionality reduction and visualization, for which points $p \in \mathbb{R}^d$ are projected into a lower-dimensional space $\mathbb{R}^m$, with $m < d$ by maximizing the variance of the projected data, or equivalently, by minimizing the average projection cost between the original data and the projected data [25, p.497]. This involves the decomposition of the covariance matrix $\mathbf{C} = \frac{1}{n} \sum_t \mathbf{x}_t \mathbf{x}_t^\top$ in its ordered eigenvalues $\Lambda_1 \geq \dots \geq \Lambda_d$ and orthogonal eigenvectors $\mathbf{v}_1 \geq \dots \geq \mathbf{v}_d$ [30, p.5450]. The data can then be projected into the lower-dimensional space by

$$\hat{\mathbf{p}} = \sum_{i=1}^m (\mathbf{p}^\top \mathbf{v}_i) \mathbf{v}_i, \quad (3.17)$$

retaining as much variance in the data as possible [30, p.5451].

A special kind of PCA involves using kernel methods similar to MMD. While standard PCA is a linear method, projecting the data into a typically higher-dimensional kernel space extends PCA into a non-linear method [30, p.5464]. This allow to capture non-linear correlations within the data, whereas standard PCA is only suitable for linear subspaces [31]. The main idea behind kernel principal component analysis (kernel PCA) is instead of computing the eigen-decomposition of the covariance matrix, a so-called kernel matrix is constructed on which the eigen-decomposition is performed [30, p.5464].

3.3.3 Radial-basis function

Both kernel PCA and MMD rely on kernel functions, that are used for the transformation into kernel spaces. A popular choice for such kernel functions is the radial basis function (RBF) or Gaussian Kernel [31, 32, 33]. It is defined as

3 Background

$$k(\mathbf{x}, \mathbf{y}) = e^{-\frac{\|\mathbf{x}-\mathbf{y}\|^2}{2\sigma^2}} \quad (3.18a)$$

$$(3.18b)$$

where σ controls the width of the kernel [32, 33]. RBF kernels are widely used when there is no additional information about the data for tasks like clustering, classification and regression [33].

3.3.4 Generative models

Generative models can be summarized by machine learning models that can be used to generate completely new, synthetic data [25, p.4]. In contrast to discriminate models used for predicting a conditional probability $p(y|\mathbf{x})$, generative models also learn the underlying probability distribution $p(\mathbf{x})$, and by extension the joint distribution $p(\mathbf{x}, y) = p(y|\mathbf{x})p(\mathbf{x})$ [34, p.1]. This information about the general structure of training data allows the generative model to generate new data similar to the training data [34, p.4]. Most prominent applications of generative AI include the generation of photo realistic images, speech in both text and audio or even videos [25, p.4]. The large amount of available text from various sources like the internet make it possible to train a large language models (LLM) to generate language on a human level using self-supervised learning, a hybrid approach to the other two main training paradigms in machine learning that are supervised learning and unsupervised learning [25, p.5].

3.4 Markov processes

In the context of modeling dynamical systems with stochastic behavior on a discrete timescale, the sequence of states, that the considered system follows can be described by a stochastic process.

In cases, where the probability of the next state $\mathbf{x}^{(t+1)}$ only depends on the current state $\mathbf{x}^{(t)}$ of the system, the system adheres to the markov property and is called markovian [35]. The probability of a given next state can then be described by

$$P(\mathbf{x}^{(t+1)}|\mathbf{x}^{(t)}). \quad (3.19)$$

A sequence of such random variables $\{\mathbf{x}^{(t)}\}_{t=0}^{\infty}$ is then called Markov chain.

In dynamical systems that include a control input η , this can be extended to

$$P(\mathbf{x}^{(t+1)}|\mathbf{x}^{(t)}, \boldsymbol{\eta}^{(t)}). \quad (3.20)$$

If the next probability of the next state depends not only on the current state of the system, but also on a fixed number of m prior states can be defined by

$$P(\mathbf{x}^{(t+1)}|\mathbf{x}^{(t)}, \dots, \mathbf{x}^{(t-m)}, \boldsymbol{\eta}^{(t)}), \quad (3.21)$$

and the process is called Markov chain with memory [35].

In real systems, approximations are often necessary. If the information of prior states is negligible, systems are often considered to be markovian. However, if prior states affect the probabilities of the next state significantly, modeling the systems using a Markov chain with memory may become necessary, depending on the accuracy required.

4 Residual dynamics for the multi-UAV payload System

Before the final datasets for the training of the generative models can be generated, it is necessary to derive functions for the computation of residual forces from the provided test flight data. Although the general approach of defining residual force for the MPS follows the [3.3], computing the residual forces in the case of the MPS requires extra steps to extend the idea of residual dynamics for coupled multibody systems.

4.1 Residual force calculations

As demonstrated in Equations [3.6], coupling the individual UAVs by attaching them to a common payload modifies the equilibrium of forces for each UAV. Introducing more than one residual force to the system leads to interdependencies between these residual forces, making it impossible to define multiple residual forces $\mathbf{f}_{ai}$ for individual UAVs at once, using a single equilibrium of force. As the number of unknown variables increases, additional constraints would be necessary to solve for each residual force. To better explain this issue, the free-body principle can be used to explicitly show the cable forces T_i , as Figure [4.1] demonstrates. However, it is possible to use the same approach defining residual forces for the complete system. In the literature, the system dynamics of the MPS is described by applying the equilibrium of forces at the payload instead of the UAV [14]. The introduced residual force $\mathbf{f}_{a0}$ for this equilibrium of forces can therefore be attributed to the payload.

The free body principle can be applied at the payload, demonstrating that the interaction forces between UAV and payload act as cable forces T_i along the direction of the cable vector $\mathbf{q}_i$. The equilibrium of forces can then be formulated as follows:

$$m_0(\ddot{\mathbf{p}}_0 + g\mathbf{e}_3) = \sum_{i=1}^n -T_i\mathbf{q}_i + \tilde{\mathbf{f}}_{a0} \quad (4.1)$$

The same method can be applied for each UAV, showing the

$$m_i(\ddot{\mathbf{p}}_i + g\mathbf{e}_3) = f_{ci}\mathbf{R}_i\mathbf{e}_3 + T_i\mathbf{q}_i + \tilde{\mathbf{f}}_{ai} \quad (4.2)$$

with corresponding residual forces $\tilde{\mathbf{f}}_{ak} \mid k = 0, 1, \dots, n$ for payload residual force and UAV residual force. However, since the provided flight data do not include measurements of the cable forces, the system of $n + 1$ equations now contains n unknown variables for cable forces T_i in addition to the $n + 1$ unknown variables for the residual forces $\tilde{\mathbf{f}}_{ak}$. This

4 Residual dynamics for the multi-UAV payload System

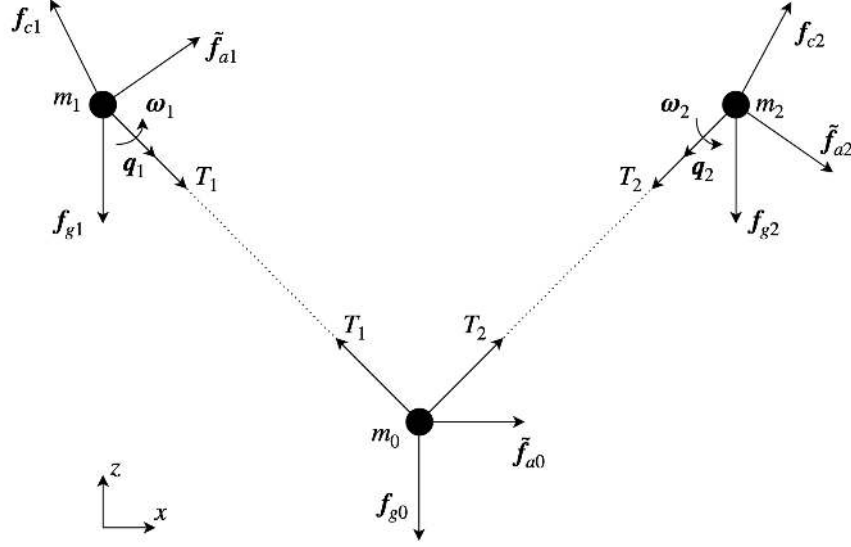


Figure 4.1: Free body diagram of a team of two UAVs carrying a shared payload. The force of gravity on each UAV is represented as $\mathbf{f}_{g_i} = -m_i g \mathbf{e}_3$ for better visualization, while the thrust vector is shown as $\mathbf{f}_{c_i} = f_{c_i} \mathbf{R}_i \mathbf{e}_3$. For better visualization the diagram shows a two-dimensional representation of the otherwise three-dimensional case.

system of $n + 1$ equations and $2n + 1$ unknown variables is therefore underdetermined, making it impossible to compute payload residual force and UAV residual force without interdependencies.

To still be able to define individual yet interdependent residual forces for each UAVs the cable forces can be incorporated in the UAV residual force as these combined forces $\mathbf{f}_{ai} = \tilde{\mathbf{f}}_{ai} + T_i \mathbf{q}_i$. This again yields an equation equivalent to Equation 3.2a that allows to compute residual force using only data of the considered UAV:

$$m_i(\ddot{\mathbf{p}}_i + g\mathbf{e}_3) = f_{c_i} \mathbf{R}_i \mathbf{e}_3 + \mathbf{f}_{ai} \quad (4.3)$$

As this definition requires $\ddot{\mathbf{p}}_i$ that was not tracked during the test flights of the provided datasets, it is necessary to compute it using the included data. Using the assumption of no slack in the cables, it can be calculated using

$$\ddot{\mathbf{p}}_i = \ddot{\mathbf{p}}_0 - l_i \ddot{\mathbf{q}}_i \quad (4.4)$$

To avoid the need to use finite differences twice, the first derivative $\dot{\mathbf{q}}_i$ can also be computed from other data:

$$\dot{\mathbf{q}}_i = \boldsymbol{\omega}_i \times \mathbf{q}_i \quad (4.5)$$

Differentiating once more yields

4.2 Markov assumption for the multi-UAV payload model

$$\ddot{\mathbf{q}}_i = \dot{\boldsymbol{\omega}}_i \times \mathbf{q}_i + \boldsymbol{\omega}_i \times \dot{\mathbf{q}}_i \quad (4.6)$$

Therefore, finite differences are only needed to compute residual forces and residual torques in the case of $\ddot{\mathbf{p}}_0, \dot{\mathbf{q}}_i, \dot{\boldsymbol{\omega}}_i$ and $\dot{\boldsymbol{\Omega}}_i$. This is done using a symmetric stencil for $\mathbf{y}_i \in [\dot{\mathbf{p}}_0, \mathbf{q}_i, \boldsymbol{\omega}_i, \boldsymbol{\Omega}_i]$ by

$$\dot{\mathbf{y}}_i^{(j)} \approx \frac{\mathbf{y}_i^{(j+1)} - \mathbf{y}_i^{(j-1)}}{t^{(j+1)} - t^{(j-1)}} \quad (4.7)$$

to achieve second-order accuracy, where j denotes the index of one row of test flight data and t denotes the respective timestamp.

Finally, we can use Equations 3.6 and 4.3 to compute the payload residual force $\mathbf{f}_{a0}$, UAV residual forces $\mathbf{f}_{ai}$ and residual torques $\boldsymbol{\tau}_{ai}$ with $i = 1, \dots, n$ for a MPS of n UAVs from the test flight data using

$$\mathbf{f}_{a0} = M_t(\ddot{\mathbf{p}}_0 + g\mathbf{e}_3) - \sum_{i=1}^n (f_{ci}\mathbf{R}_i\mathbf{e}_3 - m_i l_i \|\boldsymbol{\omega}_i\|^2 \mathbf{q}_i) \quad (4.8a)$$

$$\mathbf{f}_{ai} = m_i(\ddot{\mathbf{p}}_0 - l_i(\dot{\boldsymbol{\omega}}_i \times \mathbf{q}_i + \boldsymbol{\omega}_i \times \dot{\mathbf{q}}_i) + g\mathbf{e}_3) - f_{ci}\mathbf{R}_i\mathbf{e}_3 \quad (4.8b)$$

$$\boldsymbol{\tau}_{ai} = \mathbf{J}_i\dot{\boldsymbol{\Omega}}_i - \mathbf{J}_i\boldsymbol{\Omega}_i \times \boldsymbol{\Omega}_i - \mathbf{M}_i \quad (4.8c)$$

and similar to Equations 3.3 use shorthand notation to illustrate the dependencies in terms of the states $\mathbf{x}$ and $\mathbf{x}_i$, its derivatives $\dot{\mathbf{x}}$ and $\dot{\mathbf{x}}_i$, and wrench $\boldsymbol{\eta}$ and $\boldsymbol{\eta}_i$:

$$\mathbf{f}_{a0} = f_{a0,\text{mps}}(\mathbf{x}, \dot{\mathbf{x}}, \boldsymbol{\eta}) \quad (4.9a)$$

$$\mathbf{f}_{ai} = f_{ai,\text{mps}}(\mathbf{x}_i, \dot{\mathbf{x}}_i, \boldsymbol{\eta}_i) \quad (4.9b)$$

$$\boldsymbol{\tau}_{ai} = f_{\tau,\text{mps}}(\mathbf{x}_i, \dot{\mathbf{x}}_i, \boldsymbol{\eta}_i) \quad (4.9c)$$

4.2 Markov assumption for the multi-UAV payload model

4.2.1 Modeling the dynamics as a first-order system

To evaluate whether MPS can be assumed to be markovian, the sequences of states of the MPS dynamics can be modeled as a first-order system. Similarly to defining Equations 4.9 for the individual residual forces, using the complete dynamics of the MPS defined by Equations 3.6 and Equations 4.8 it is possible to derive a step function f_{step} to calculate the derivative of the state $\dot{\mathbf{x}}$, modeling the MPS as a first order system that can more generally be denoted by

$$\dot{\mathbf{x}} = f_{\text{step}}(\mathbf{x}, \boldsymbol{\eta}) + \mathbf{f}_{\text{res}} \quad (4.10)$$

Here, $\mathbf{f}_{\text{res}}$ is a modified residual force vector, that can be computed using a function of the state, its derivative and the wrench $\mathbf{f}_{\text{res}} = g_{\text{res}}(\mathbf{x}, \dot{\mathbf{x}}, \boldsymbol{\eta})$, so that Equation 4.10 holds. Without covering the specifics of the structure of the functions f_{step} and g_{res} , this provides a manageable structure for the system.

Provided that all necessary information about the state of the system $\mathbf{x}^{(t)}$ is given at any given time step t and the residuals $\mathbf{f}_{\text{res}}$ are given, the next state $\mathbf{x}^{(t+1)}$ of the system can be evaluated by integration of Equation 4.10, or approximated using an *explicit Euler method* and a corresponding step size Δt by

$$\mathbf{x}^{(t+1)} = \mathbf{x}^{(t)} + \Delta t \left(f_{\text{step}}(\mathbf{x}^{(t)}, \boldsymbol{\eta}^{(t)}) + \mathbf{f}_{\text{res}}^{(t)} \right) \quad (4.11)$$

This process $\{\mathbf{x}^{(t)}\}_{t=0}^{\infty}$ is deterministic and therefore a special case of a Markov chain under the assumption that the residual force are deterministic and the error of using numerical integration with a fixed step size Δt is negligible. Although it is possible to compute the deterministic values for the residual force vector $\mathbf{f}_{\text{res}}$ using Equation 4.9, this requires information about the state derivative $\dot{\mathbf{x}}_i$. However, the information of the system derivative is only available after measuring the actual accelerations of the system in test flights. Therefore, it is particularly interesting if the residual forces can be predicted using only the available information of the state of the system.

4.2.2 Residual forces as random variables

Because of this, it makes sense to treat the residual forces as random variables $\mathbf{f}_{\text{res}} \sim P(\mathbf{f}_{\text{res}}|\mathbf{x}, \dot{\mathbf{x}}, \boldsymbol{\eta})$ conditioned by the state $\mathbf{x}$, its derivative $\dot{\mathbf{x}}$ and wrench $\boldsymbol{\eta}$. By this, the process denoted by Equation 4.11 becomes a *stochastic process* described using

$$P(\mathbf{x}^{(t+1)}|\mathbf{x}^{(t)}, \boldsymbol{\eta}^{(t)}, \mathbf{f}_{\text{res}}^{(t)}) \quad (4.12a)$$

$$= P(\mathbf{x}^{(t+1)}|\mathbf{x}^{(t)}, \dot{\mathbf{x}}^{(t)}, \boldsymbol{\eta}^{(t)}) \quad (4.12b)$$

As here, the next state not only depends on the current state and input but also on the derivative of the state, the system does not necessarily satisfy the markov property anymore.

4.3 Effect of the state space conditioning on the stochastic process

The influence of the information provided by the state derivative can be evaluated further by approximating the state vector derivative using finite differences, $\dot{\mathbf{x}}^{(t)} \approx$

4.3 Effect of the state space conditioning on the stochastic process

$f(\mathbf{x}^{(t)}, \dots, \mathbf{x}^{(t-m)})$ the stochastic process can be described by a Markov chain with memory

$$P(\mathbf{x}^{(t+1)} | \mathbf{x}^{(t)}, \dots, \mathbf{x}^{(t-m)}, \boldsymbol{\eta}^{(t)}). \quad (4.13)$$

In the case of $P(\mathbf{f}_{\text{res}} | \mathbf{x}, \dot{\mathbf{x}}, \boldsymbol{\eta}) \approx P(\mathbf{f}_{\text{res}} | \mathbf{x}, \boldsymbol{\eta})$, the model can be assumed to be markovian by

$$P(\mathbf{x}^{(t+1)} | \mathbf{x}^{(t)}, \boldsymbol{\eta}^{(t)}) \quad (4.14)$$

This exemplifies the effect of approximating $P(\mathbf{f}_{\text{res}} | \mathbf{x}, \dot{\mathbf{x}}, \boldsymbol{\eta})$, the choice of conditioning $\mathbf{c} \subseteq (\mathbf{x}, \dot{\mathbf{x}}, \boldsymbol{\eta})$ features on the prediction of the residual vector using $P(\mathbf{f}_{\text{res}} | \mathbf{c}) \approx P(\mathbf{f}_{\text{res}} | \mathbf{x}, \dot{\mathbf{x}}, \boldsymbol{\eta})$, and the implications for the markov property of the MPS.

5 Datasets

The performance of the generative models is highly dependent on the quality of the datasets used for training and validation. The training data used are derived from actual test flight data that was provided for this thesis. However, generating usable training and validation datasets from the raw test flight data involves several steps of augmentation that can be summarized by the following.

1. Extraction of flight data
2. Pre-processing
3. Residual calculation
4. Selection of conditioning features

The first three steps of this pipeline were then used to generate and save data structures containing all necessary information for each of in sum 20 different flight experiments. This allows for an efficient and deterministic process of final training and validation datasets generation for arbitrary choices of conditioning features without intensive pre-processing and residual calculations.

5.1 Test flight data

The flight data from the test flights, which was kindly provided for the purpose of this thesis, was originally collected by Wahba et al. [14] during experiments to test a *kinodynamic motion planner* for the MPS [14]. The flights were conducted indoors using multiple Crazyflies UAVs, which are an open platform of small and lightweight quadrotors that are popular for use in research due to their versatility [36].

As the generative models are trained and evaluated using different experiments in the provided data, the experiments are explained below.

The general strategy behind the test flights was to perform repeated flight experiments testing different configurations of the MPS and components. All test flights for MPS were conducted with two or three UAVs carrying the payload. For each team size, the flight planners had to solve one of two scenarios, which involved repositioning UAVs relative to the payload during the flights. The goal was to traverse a narrow passage similar to a flight through a window as shown in Figure 5.1a, or a forest-like environment in *forest* as demonstrated by Figure 5.1b. For each case there were two reference trajectories precomputed, the first by a *geometric* planner with the second, *optimized* one being computed by the kinodynamic motion planner with the computed solutions of

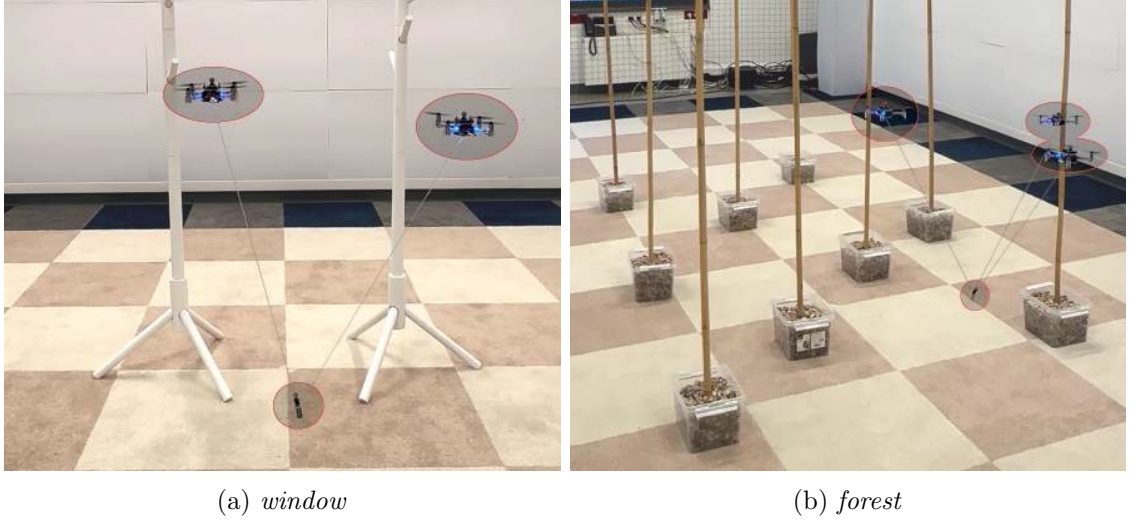


Figure 5.1: Test flights of the MPS for the flight scenario *window* and *forest*. The columns represent the vertical boundaries of a window or trees respectively, that the MPS must navigate around and avoid.

the *geometric* planner as initial guesses. This allowed for repeated tests using the same reference trajectory. More detailed information on the experimental setup and the data collection process can be found in the original source [14].

In summary, a single test flight out of a series of n repeated flights can be described by a set of the following properties:

- Number of UAVs in the system (2 or 3)
- Flight scenario (*forest* or *window*)
- Motion planning method (*geometric* or *optimized*)
- Iteration ($i \in 1, \dots, n$ for values $\{n \in \mathbb{Z} | n \in [6, 10]\}$)

5.2 Pre-processing

To generate datasets that are ready for training of generative models, several pre-processing steps are necessary. This is due to the data structure of the flight data provided and the findings discussed in detail later that smoothing of the flight data might be necessary to generate meaningful training datasets. The basic process can be summarized as follows.

In the data provided, each UAV collected flight data including its state, the position and velocities of the payload, the reference position of the payload that the MPS is trying to track, and the corresponding timestamp in a fixed interval. The internal clocks of all UAV are synchronized for a given flight experiment, and all UAVs operate with

the same sample frequency of 100 Hz. However, although the sampling intervals are the same, the UAVs do not save individual samples at the same time, but rather shifted by an offset. Because of this, complete state information for the entire **MPS** is not available at any time. However, since this is required to be able to compute residual forces for the payload using the respective function of Equations **4.9**, interpolation of the flight data on a shared timescale is required.

Similarly, since the clocks are not synchronized between repeated test flights, an optional second step of adjusting the timestamp values for a given flight experiment of repeated test flights allows for better comparison between each flight.

After this, the interpolated, synchronized, and possibly smoothed flight data can be used to compute the residual forces, yielding tuples of state space information and the corresponding residual force that are suitable for training.

5.2.1 Interpolation using curve-fitting splines

The interpolation of state space information is performed by fitting spline functions for each tracked variable of the system, which allows evaluation on a shared timescale for all UAVs. Although higher sampling frequencies are possible for fitted spline functions, for this thesis the original sampling frequency of 100 Hz was used. Additional advantages of using curve fitting splines are the ability of smoothing the given data as well as the ability of analytic differentiation, in contrast to using numerical differentiation which is prone to introduce noise. A spline is defined as a piecewise function using a polynomial for each piece. Depending on the degree of polynomial, it is possible to define constraints in such a way that the function values and derivatives of adjacent pieces are continuous **37**.

BSpline

A common choice for spline functions is the use of *BSpline*, where the smoothness of the curve and its shape can be controlled by the degree of the polynomial and the number of knots and control points **37**. Although smoothing of the data is possible with BSplines, here BSplines were only used for interpolation without smoothing using the function *scipy.interpolate*. *BSpline* from the *Python* module *SciPy* **38**.

Least-square fit smoothing

An intuitive, albeit computationally expensive, approach to fitting splines to a function with discrete observations is to partition the domain in N segments, and define polynomials of degree D for each segment. The optimization problem can then be defined to find the optimal coefficients for the polynomials, minimizing the **mean squared error (MSE)** of the observations and the function values of the polynomials. Additional constraints can be added to ensure the continuity of values and derivatives across adjacent segments or the regularization of coefficients.

For this, weights are defined to balance the fit of the polynomials to the data with the smoothness and regularity of the polynomials, where α is the weight for the least squares error term and λ is the regularization weight preventing the coefficients being too large. The coefficients of the monomials $d = 0, \dots, D$ of the polynomial segment i are defined by the coefficient vector $\mathbf{c}_{i,d}$. For a given segment i , t_j is the timestamp value and f_j is the corresponding output value at the index $j = 1, \dots, M_i$ where M_i is the length of the segment i , to account for segments of unequal length. This is necessary because division of large intervals into segments of fixed integer length would lead to possibly large remainders. This corresponds to data that cannot be accounted for in the segments of the fixed integer length and could therefore not be used for fitting.

The optimization problem can be defined by:

$$\min_{\mathbf{c}_1, \mathbf{c}_2, \dots, \mathbf{c}_N} \alpha \sum_{i=1}^N \sum_{j=1}^{M_i} \left(\sum_{d=0}^D \mathbf{c}_{i,d} \cdot t_j^d - f_j \right)^2 + \lambda \sum_{i=1}^N \sum_{d=0}^D \mathbf{c}_{i,d}^2 \quad (5.1)$$

Continuity constraints at segment boundaries

The constraints for each boundary between segment $i = 1, \dots, N-1$ and $i+1$ with associated timestamps at the boundaries t_{M_i} are then

$$\sum_{d=0}^D \mathbf{c}_{i,d} \cdot t_{M,i}^d = \sum_{d=0}^D \mathbf{c}_{i+1,d} \cdot t_{M,i}^d \quad (5.2a)$$

$$\sum_{d=1}^D d \cdot \mathbf{c}_{i,d} \cdot t_{M,i}^{d-1} = \sum_{d=1}^D d \cdot \mathbf{c}_{i+1,d} \cdot t_{M,i}^{d-1} \quad (5.2b)$$

$$\sum_{d=2}^D d(d-1) \cdot \mathbf{c}_{i,d} \cdot t_{M,i}^{d-2} = \sum_{d=2}^D d(d-1) \cdot \mathbf{c}_{i+1,d} \cdot t_{M,i}^{d-2} \quad (5.2c)$$

Additional constraints for the continuity of higher derivatives can be added by trivial extension but were not used for this thesis.

The weights α and λ for their corresponding loss terms affect the solution as they influence their respective error terms. However, the choice of the degree of the polynomial D and the number of segments N have a significantly stronger impact on the found solution, as the length of a segment and the degree of the polynomial pose an approximate upper bound for the supported maximum frequency of the fitted signals, as will be demonstrated in [5.2.1](#).

Smoothing

Plotting values of state space features over the flight time of a single test flight reveals how the signals of certain state space features are dominated by high frequency parts, as Figures for [5.2](#) and Figure [5.5](#) demonstrate. An analysis using [fast Fourier transform](#)

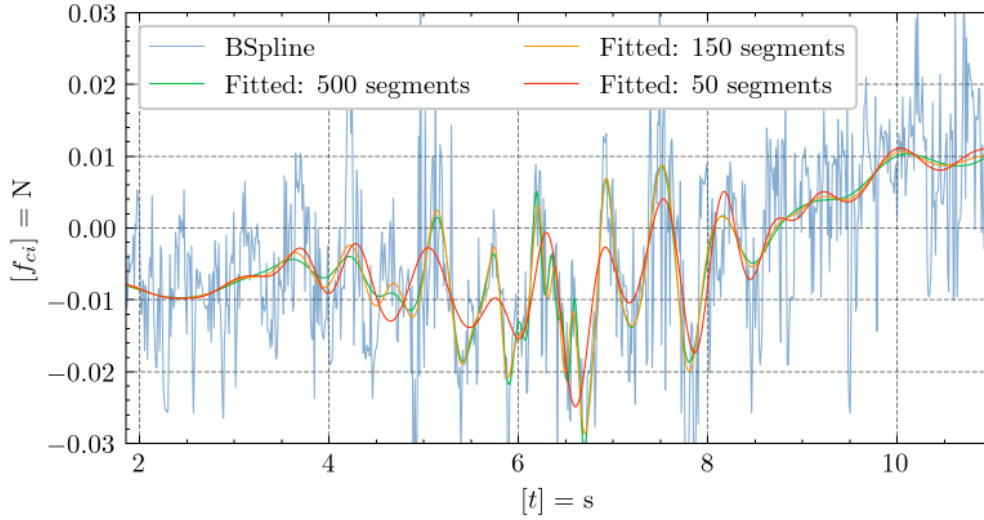


Figure 5.2: Collective thrust f_{ci} of one UAV over the course of one test flight, interpolated using BSplines and a least-square fit for different number of segments.

(FFT) in 5.3 further exemplify this, by showing how the parts of lower frequency are captured well, while the frequencies higher than 30 Hz are not well represented, even by increasing the number of segments per flight from 150 to 500. However, the FFT also shows how the fitted BSpline functions without smoothing capture the higher frequency parts much better. Figure 5.2 also demonstrates the effect of smoothing of high-frequency signals with respect to the absolute signal values. Since high frequency parts dominate lower frequency parts in this example, the command values of the collective thrust f_{ci} have a strong bias towards smaller absolute values.

Although other state space features such as those shown in Figure 5.4 also contain parts of high frequencies, their corresponding power is significantly lower, resulting in much better smoothing results, compared to high frequency signals in Figure 5.5. However, high-frequency parts in important state features for the computation of residual forces result in similarly noisy values of residual forces when plotted over time for single test flights, as demonstrated in Figure 5.6. Smoothing of state-space features before the computation of residual forces mitigates this problem, however, since the high-frequency parts dominate the lower-frequency parts of the signal of residual forces over time, they do not represent the original values well.

It is crucial to note that smoothing can only be performed on input features such as state space features, as this preserves the computation of correct values of residual forces for a smoothed state. The values of residual forces are calculated using the Equations 4.9 and no additional smoothing of the output values is performed, therefore, the output values are not altered. Since all functions in Equations 4.9 are smooth, the noise in residual force values as a function of time is indeed introduced by the raw test flight data provided. Therefore, using smoothing this way impacts them residual forces most,

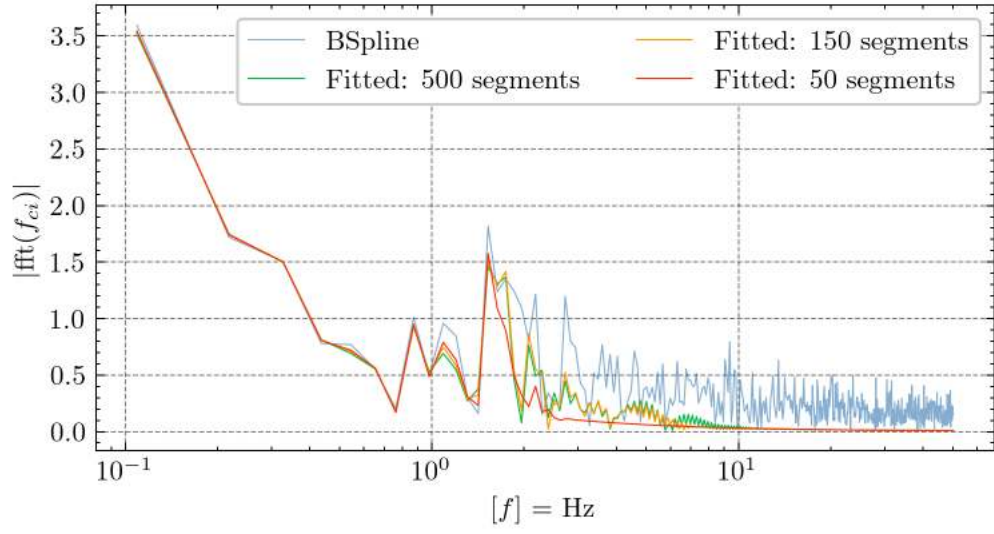


Figure 5.3: **FFT** of the collective thrust f_{ci} of one UAV during a test flight as demonstrated in **5.2**

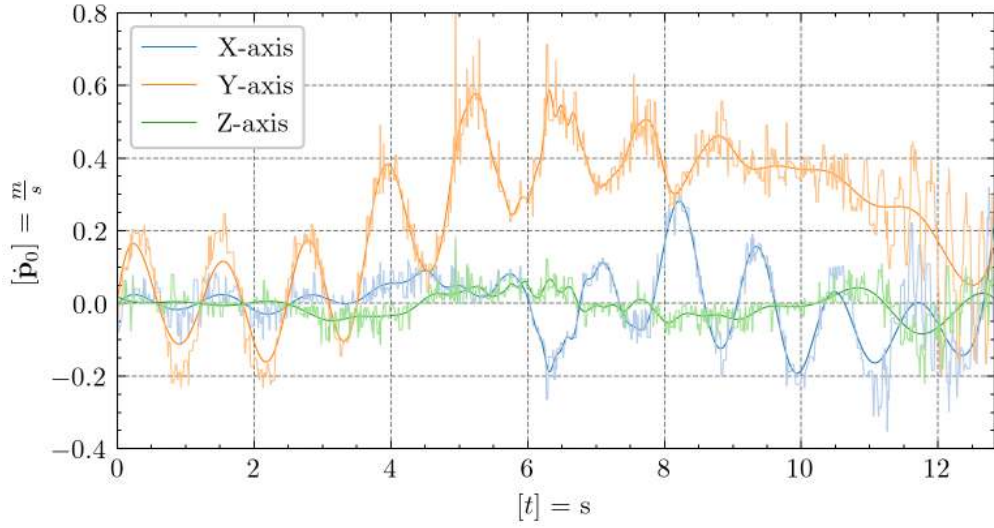


Figure 5.4: Effect of smoothing for payload velocities $\dot{\mathbf{p}}_0$ during one test flight. Non-smoothed data is shown in pastel colors, while solid colors show the data after smoothing.

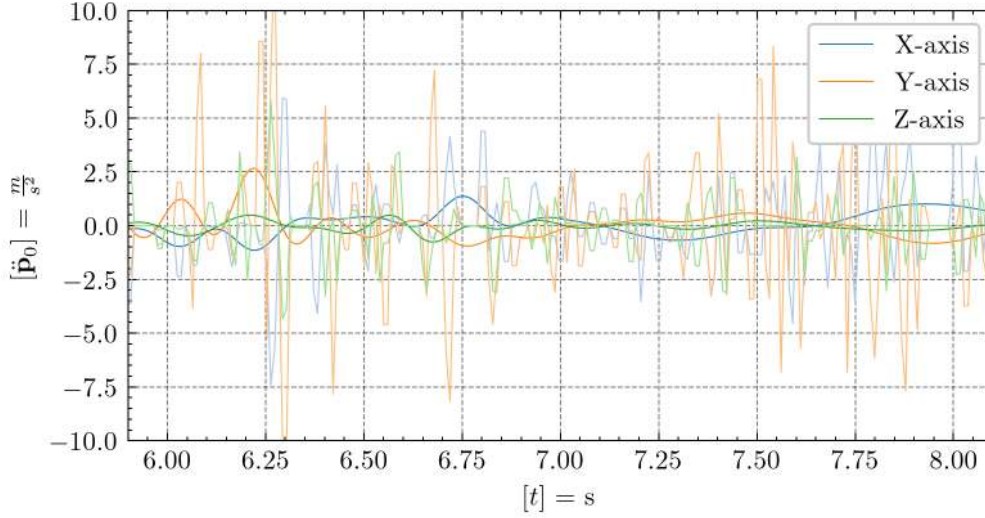


Figure 5.5: Effect of smoothing for payload accelerations $\ddot{\mathbf{p}}_0$ for a 2s window of one test flight. Non-smoothed data is shown using light colors, while solid colors show the data after smoothing.

when they are evaluated as a function over time for single test flights.

Figure 5.4 demonstrates how this smoothing method allows for a better evaluation of the general trends for residual forces in certain parts of a test flight. However, as the smoothing changes the distribution of state-space features, the distribution of residual forces is changed in a similar way. Because of this, datasets with or without smoothing could be used for training and evaluation of the same generative models if used for pure regression of the functions in Equations 4.9. Essentially, this method of smoothing creates artificial training data of the residual functions covered in Equations 4.9 that could be encountered in low noise scenarios, for example, by forcing smooth motor inputs for the UAVs.

5.2.2 Synchronization

The datasets $\mathbb{D}^{(n)} = \{\mathbf{x}_k, \dot{\mathbf{x}}_k, \boldsymbol{\eta}_k, \mathbf{t}_k, \mathbf{p}_{\text{ref},k}\}_{k=1}^{K^{(n)}}$ for $n = 1, \dots, N$ repeated test flights with $K^{(n)}$ logged samples per dataset share similar data, including the logged information of the state using the shorthand notation $\mathbf{x}$, its derivative $\dot{\mathbf{x}}$, the combined wrench $\boldsymbol{\eta}_k$, logged timestamps $\mathbf{t}$ and positions for the payload in the form of a precomputed reference trajectory $\mathbf{p}_{\text{ref}}$ for the payload, which the MPS tracks during the flight and is shared across all iterations of the given experiment. Under the assumption that the states of the MPS for a given position on the reference trajectory are similar, a first approximation of distributions of residual forces can be computed.

Even more intuitively, it is also possible to obtain these distributions for any given timestamp by $\mathbf{f}_{a0}^{(n)}(\tau)$ and $\mathbf{f}_{ai}^{(n)}(\tau)$ for any given timestamp τ across repeated test flights. For this, it is mandatory to get a unique mapping of timestamps corresponding to their

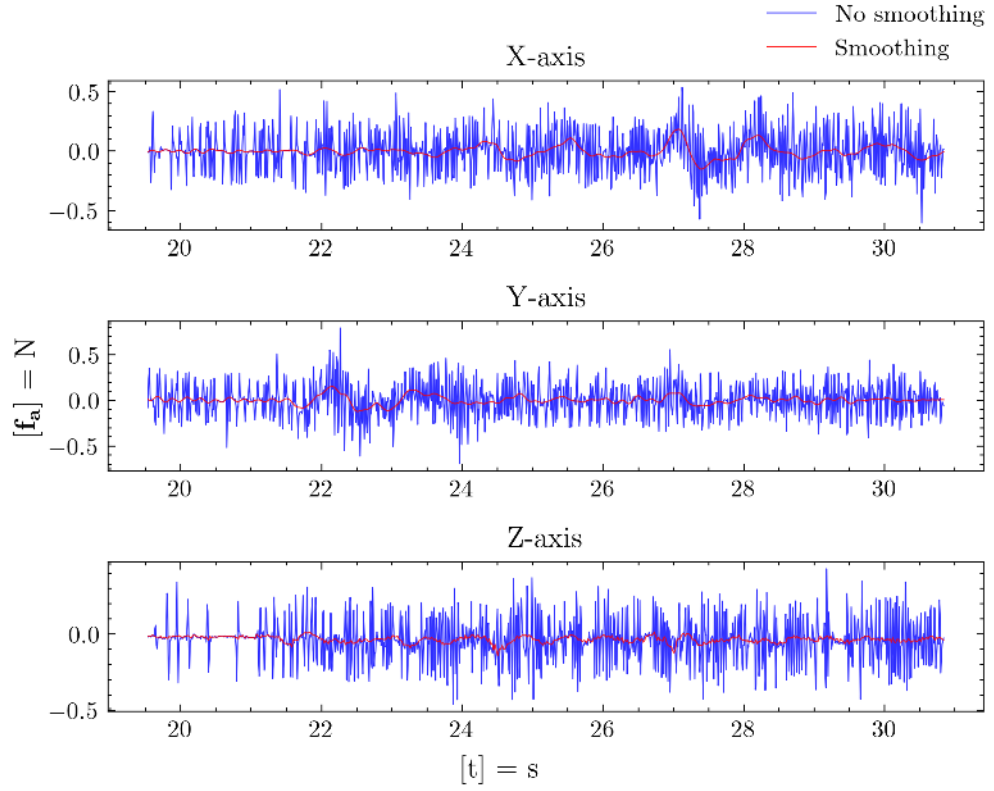


Figure 5.6: Effect of smoothing of input state features on the residual forces f_{a0} . Note, that smoothing was only performed on the input features before computation of the residual forces.

payload reference trajectory position. As a result, for any given timestamp, the payload reference trajectory positions match up across all repeated test flights.

Since absolute timestamp values are arbitrary, it is possible to align the timestamps and reference trajectory positions using a simple offset for each repeated flight to match against a chosen baseline. For this, the baseline dataset is denoted by $\mathbb{D}^{(0)}$. Here, the first of each repeated test flight is used as a baseline with $\mathbb{D}^{(0)} = \mathbb{D}^{(1)}$, the process therefore has to be performed for the remaining datasets $\mathbb{D}^{(n)}$ with $n = 2, \dots, N$.

Finding the correct offset involves two steps that can be repeated for each repeated flight, resulting in all timestamp values matching the same reference trajectory position. The first steps involve an approximate normalization of timestamp values. This helps for the solving of a subsequent optimization problem that tries to find an optimal offset by evaluating fitted Spline functions for the reference trajectory against a common baseline, to match both reference trajectories as functions of time.

The approximate normalization of all timestamps using a scalar offset $t_{\text{norm}}^{(n)} \in \mathcal{R}$ with $\hat{\mathbf{t}}^{(n)} = \mathbf{t}^{(n)} + t_{\text{norm}}^{(n)}$ for a given flight removes large offsets in the timestamp values in a given dataset $\mathbb{D}^{(n)}$. This is done by choosing the offset $t_{\text{norm}}^{(n)} = -\mathbf{t}_0^{(n)}$ where $\mathbf{t}_0^{(n)}$ is the first entry in a given dataset n with the result that all timestamp values are nonnegative and, on average, as small as possible.

The fitting of BSplines on two payload reference trajectory positions $\tilde{\mathbf{p}}_{\text{ref}}^{(n)}$ and baseline $\tilde{\mathbf{p}}_{\text{ref}}^{(0)}$ as functions of $\hat{\mathbf{t}}^{(n)}$ and $\hat{\mathbf{t}}^{(0)}$, respectively, allows for finding a second offset t_{off} , so that $\tilde{\mathbf{p}}_{\text{ref}}^{(n)}(\hat{\mathbf{t}}^{(0)} - t_{\text{off}}) \approx \tilde{\mathbf{p}}_{\text{ref}}^{(0)}(\hat{\mathbf{t}}^{(0)})$. This is done by solving a simple optimization problem that minimizes the MSE of the Euclidean norm of the differences in the positions of the reference trajectory when evaluated along a series of timestamps $\mathbf{t}_j$ with $j = (1, \dots, J)$ with respect to the timestamp offset $t_{\text{off}}^{(n)}$ by

$$\frac{1}{J} \sum_{j=1}^J \|\tilde{\mathbf{p}}_{\text{ref}}^{(n)}(\mathbf{t}_j - t_{\text{off}}) - \tilde{\mathbf{p}}_{\text{reference}}^{(0)}(\mathbf{t}_j)\|_2^2 \quad (5.3)$$

The approximate normalization of the timestamp values prior is necessary, as the timestamps used to evaluate the fitted functions must be in their interval of timestamps $[\min(\hat{\mathbf{t}}^{(n)}), \max(\hat{\mathbf{t}}^{(n)})]$ for each dataset $\mathbb{D}^{(n)}$ as well as the timestamps $[\min(\hat{\mathbf{t}}^{(0)}), \max(\hat{\mathbf{t}}^{(0)})]$ of the baseline $\mathbb{D}^{(0)}$. Ensuring that this interval is roughly the same across all datasets $\mathbb{D}^{(n)}$ allows choosing an interval of shared timestamps by through the intersection of $t_{\text{off}} \in [\min(\hat{\mathbf{t}}^{(n)}), \max(\hat{\mathbf{t}}^{(n)})]$ and $[\min(\hat{\mathbf{t}}^{(0)}), \max(\hat{\mathbf{t}}^{(0)})]$ and ensures that initial guesses for the correct offset $t_{\text{off, initial}} = 0$ and subsequent queries are valid. The optimization itself is performed using the function `scipy.optimize.minimize` from the *SciPy* library [38].

Finally, the calculated offsets are applied for each dataset, thus synchronizing the datasets and allowing for the plot of the subsequently computed residual forces as a function of time.

5.2.3 Residual calculation and dataset extraction

After the pre-processing of the test flight data, calculating the residual force is straightforward. For each set of state space features, the functions in Equations 4.9 can be used to calculate the corresponding residuals. This produces the joint datasets $\mathbb{D}_{\text{joint}}^{(n)} = \{\mathbf{f}_{a0,k}, \mathbf{f}_{ai,k}, \mathbf{x}_k, \dot{\mathbf{x}}_k, \boldsymbol{\eta}_k, \mathbf{t}_k, \mathbf{p}_{\text{ref},k}\}_{k=1}^{K^{(n)}}$ with $i = 1, \dots, M$ for the team sizes $M \in \{2, 3\}$, which represent the joint distributions of residual forces and the flight data logged and processed. These can then be used to extract datasets of conditional distributions of residual forces and the corresponding conditioning information, or for visualization as demonstrated in Figures 5.7 and 5.8.

The actual process of extracting the datasets for training is straightforward. After selecting a subset of conditioning features, the selected features of a given sample are concatenated in a single vector $\mathbf{c}_k^{(n)}$. Similarly, the residual force vectors $\mathbf{f}_{a0,k}$ and $\mathbf{f}_{ai,k}$ are concatenated into a combined residual vector $\hat{\mathbf{f}}_{a,k}^{(n)} = (\mathbf{f}_{a0,k}^{(n)}, \mathbf{f}_{ai,k}^{(n)})^\top$ with $i = 1, \dots, M$.

Finally, datasets for conditional distributions of residual forces can be constructed by $\mathbb{D}_{\text{selected}}^{(n)} = \{\hat{\mathbf{f}}_{a,k}^{(n)}, \mathbf{c}_k^{(n)}\}_{k=1}^{K^{(n)}}$, that can be used for training of generative models.

5.3 Selection of conditioning features

To explore the effect of using a subset of available information on the distribution of residual force, three different sets of conditioning features are selected and used for the evaluation, as demonstrated in Table 5.1. By testing different sets and comparing the performance of the generative models, it is possible to assess how the models transition from a model that relies on regression by learning the functions used to calculate residual forces to a model that learns the underlying distributions. Furthermore, comparing residual distributions for different sets of conditioning information allows for reasoning with regard to markov property as explained in Section 4.2.2. The different sets of conditioning information therefore do not only assess the models performance, but also allow exploration of the residual distributions independently of the generative models. This is also a deciding factor for the clustering algorithm introduced in Chapter 7. This relies on the same conditioning features of the selected in the datasets and, therefore, possibly lead to different clustering results.

To simplify the representation of the orientation of UAVs, the rotation matrices are converted into Euler angles $\mathbf{r}_i = f_{\text{matrix_to_euler}}(\mathbf{R}_i)$ using the *rowan* package, by converting the rotation matrices first to quaternions and then back to euler angles using the functions *rowan.from_matrix* and *rowan.to_euler* [39].

The first set *full regression conditioning* contains all state space features necessary to compute both residual force as well as residual torque using the functions in Equations 4.9. Since the thesis only evaluates the residual distributions for residual forces, this set of conditioning features provides additional information that is not needed for evaluating the functions and computing the residual forces alone. The purpose of this set is to evaluate whether possibly redundant information has a positive or negative impact on

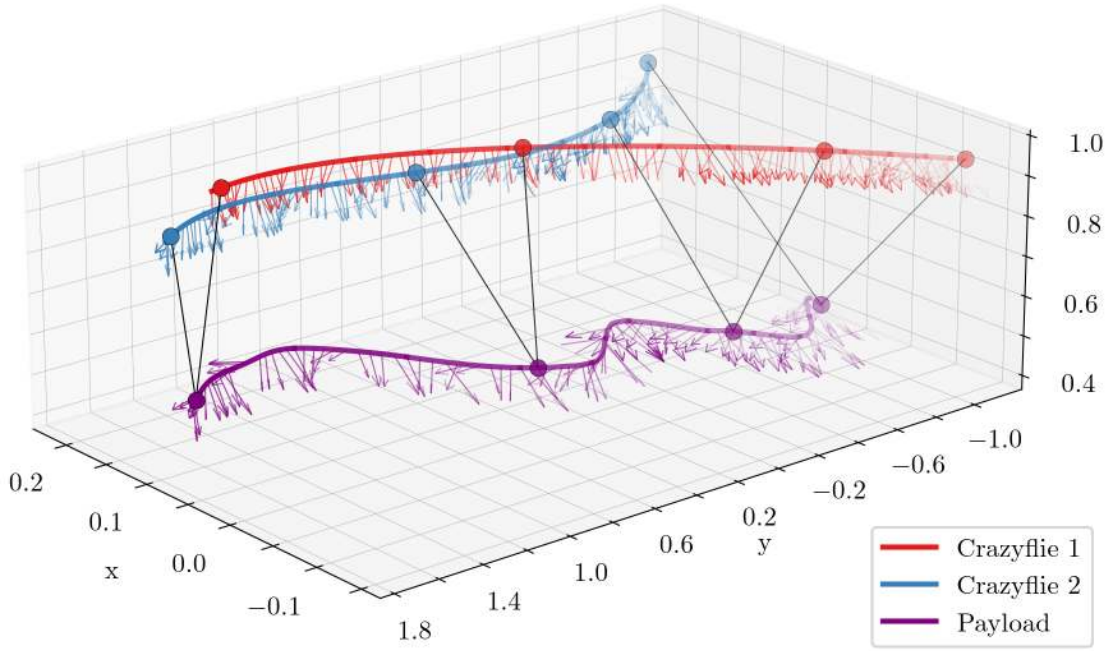


Figure 5.7: Subsequent configurations of the MPS over the course of one test flight of the corresponding flight scenario *forest* with 2 UAVs. Transparency indicates configurations further back in time. The arrows indicate the directions of residual forces $\frac{\mathbf{f}_{a0}}{|\mathbf{f}_{a0}|}$ for the payload and $\frac{\mathbf{f}_{ai}}{|\mathbf{f}_{ai}|}$ for the UAVs acting at the respective trajectories of the body positions $\mathbf{p}_0$ and $\mathbf{p}_i$. The residual forces were computed on smoothed state features and were normalized for better visualization.

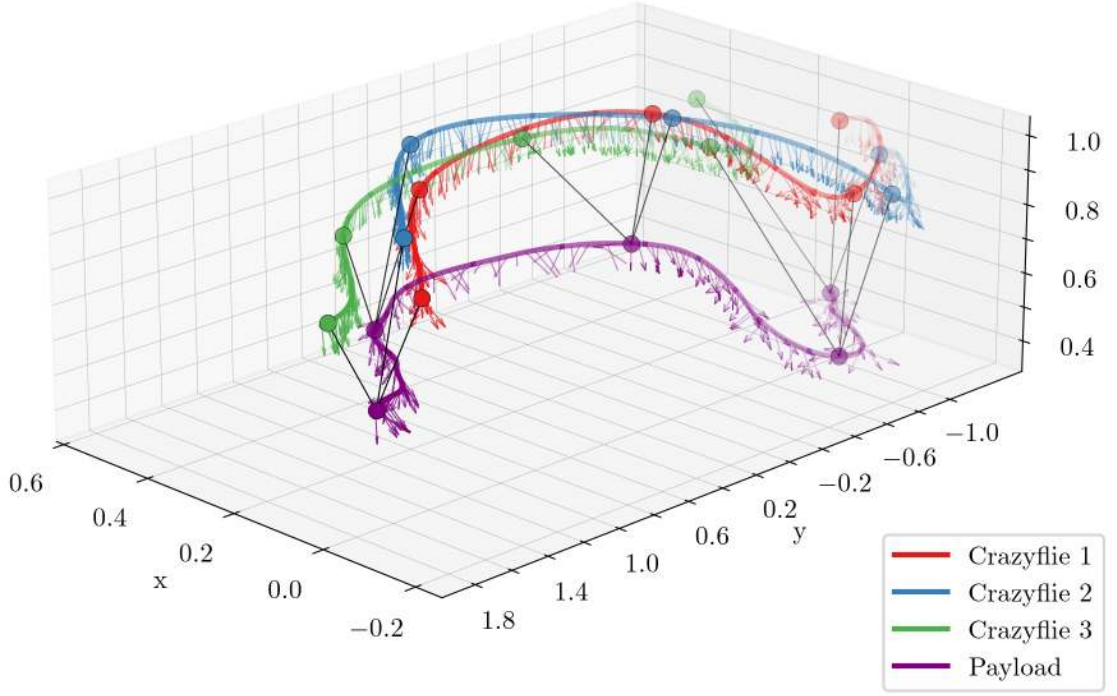


Figure 5.8: Subsequent configurations of the MPS over the course of one test flight of the corresponding flight scenario *forest* with 3 UAVs. Transparency indicates configurations further back in time. The arrows indicate the directions of residual forces $\frac{\mathbf{f}_{a0}}{|\mathbf{f}_{a0}|}$ for the payload and $\frac{\mathbf{f}_{ai}}{|\mathbf{f}_{ai}|}$ for the UAVs acting at the respective trajectories of the body positions $\mathbf{p}_0$ and $\mathbf{p}_i$. The residual forces were computed on smoothed state features and were normalized for better visualization.

Conditioning Set	Conditioning Vector $\mathbf{c}^{(n)}$
<i>full regression conditioning</i>	$(\mathbf{p}_0, \dot{\mathbf{p}}_0, \ddot{\mathbf{p}}_0, \mathbf{p}_i, \dot{\mathbf{p}}_i, \mathbf{q}_i, \dot{\mathbf{q}}_i, \boldsymbol{\omega}_i, \dot{\boldsymbol{\omega}}_i, \mathbf{r}_i, \boldsymbol{\Omega}_i, \dot{\boldsymbol{\Omega}}_i, f_{ci}, \mathbf{M}_i)^\top$
<i>reduced regression conditioning</i>	$(\ddot{\mathbf{p}}_0, \mathbf{q}_i, \dot{\mathbf{q}}_i, \boldsymbol{\omega}_i, \dot{\boldsymbol{\omega}}_i, \mathbf{r}_i, f_{ci})^\top$
<i>minimal regression conditioning</i>	$(\mathbf{q}_i, \boldsymbol{\omega}_i, \mathbf{r}_i, \boldsymbol{\Omega}_i, f_{ci})^\top$

Table 5.1: Sets of conditioning features for residual force (indices for time steps omitted for brevity)

the performance of the models, which could in theory operate as full regression models, and the underlying residual distributions.

The second set *reduced regression conditioning* uses the exact features that are necessary for a regression model of the functions for computing residual forces, which is necessary for a comparison with the complete conditioning information *full regression conditioning*.

The third set *minimal regression conditioning* covers the information on the state of the system $\mathbf{x}_{\text{mps}}$ and the collective thrusts f_{ci} without the absolute positions of the payload $\mathbf{p}_0$.

Although the relative positions of UAVs and the payload possibly can affect the residual forces because of their mutual interactions, including the absolute position of the payload in the conditioning vector could lead to overfitting. Aerodynamic interactions with the walls or the floor such as *ground effect* are known problems and have been addressed, for example by learning ground effect interactions using the approach of residual forces for single UAV in [40]. This indicates that the distance to the ground or walls should be included in the conditioning vector if obstructions like this are present. However, for the complete duration of the logged flight data, the UAVs of the MPS were in safe heights without walls close by, which means that there ground effect or similar aerodynamic should not have affected the flights. Because of this, information of the absolute positions of the MPS in the world frame can be safely omitted for the data used in this thesis, while the relative positions of the UAVs and payload are encoded in the cable vectors $\mathbf{q}_i$.

The conditioning information set *minimal regression conditioning* therefore consists of a modified system state and thrust inputs that could describe the dynamics of a MPS under the assumption that the system satisfies the markov property. Thus, a generative model that accurately reproduces the corresponding ground truth residual distributions can be an indicator of this assumption, as explained in Section 4.2.

6 Generative models for predicting residual forces

This chapter integrates both the background theory and implementation details for the [VAE](#) and diffusion model, for sake of better understanding of the implementation details.

6.1 General architecture

To train models that generalize well across the entire problem it is important to find suitable hyperparameters so that the trained models give good approximations for all inputs of the training dataset as well as the validation set or for entirely different test settings. The type of hyperparameters can be set apart by more general hyperparameters that apply to both generative models and generative model type-specific hyperparameters.

The set of hyperparameters relevant for both generative models is summarized as follows:

1. Learning rate
2. Batch size
3. Choice of optimizer
4. Number of training epochs

The learning rate generally determines how quickly the model learns. However, if set too high, it can cause the training to become unstable and hinder convergence. A learning rate too low can possibly hinder the escape of a local minimum, preventing a better solution. The learning rate should be tuned in a way that balances a suitable convergence rate with stability [\[25\]](#), pp.222-223].

According to Bishop and Bishop [\[25\]](#), the neural network training is often performed using *mini-batches*, which consist of a subset of entries in the training dataset, the size of which is often called *batch size* and affects the training as follows: The choice of batch size directly affects how much memory is required during training, so that the available memory for training introduces an upper bound. A greater batch size also results in faster training since there are fewer batches to iterate for each training epoch. In contrast, smaller batch sizes introduce more volatility for each training step with the opportunity of escaping local minima similar to a higher learning rate. In the same way, small batch sizes can lead to instabilities during training and hindering the convergence of the models.

The optimizer affects the training stability, the convergence speed, and the sensitivity to changes in other hyperparameters. For this thesis, all models are trained using the Adam optimizer [25, pp.223-224].

The number of training epochs determines how well the training dataset is learned and is directly influenced by the choice of the learning rate and optimizer. The number of training epochs has to balance the model underfitting and overfitting as already discussed. Although a fixed number of training epochs is possible, methods like early stopping, which is an alternative to regularization, can mitigate this problem by finding the optimal choice of this hyperparameter during training instead of having to define it before the start of training [25, p.266].

6.2 Training process

Each generative model is trained using the same training process. After loading the training and validation datasets, the generative models is instantiated with a set of parameters to adjust for the changing shape of the datasets, as well as the selected hyperparameters that define the architecture of a given generative model. The actual training is done in mini-batches.

Each time after the end of 20 epochs, a validation step is executed that calculates both the training loss and the validation loss to track the training process. The validation loss uses the exact same loss function that is used for training using batches from the validation dataset instead of the training dataset.

To save computation resources, the training is stopped early if the validation loss does not improve for ten consecutive validations of the model. Additionally, a model checkpoint is saved if the validation loss is lower than the validation loss of the previous checkpoint, overwriting it so that only the best-performing model is saved. While other metrics, such as the average MMD, could also be incorporated into the validation process, they require additional steps like clustering. This would significantly slow down training. As a result, validation loss is used as a quicker, approximate measure of performance during training, making tasks like hyperparameter tuning and ablation studies feasible. If the training is not stopped early, it will be stopped after a specified maximum number of epochs. The checkpoint saved for each training process is therefore guaranteed to have the lowest validation loss.

6.3 Autoencoders

Autoencoders serve the function of finding latent space representations for distributions of input data, by predicting its own input, and preserving as much information as possible prioritizing the most important information finding its best possible representation [41, 34].

According to Bishop and Bishop [25] an autoencoder is typically implemented using a single neural network that can be viewed of two separate neural networks called encoder and decoder: The encoder maps input data (e.g. images) from data space to a latent

representation on a typically lower-dimensional latent space. This latent space representation then serves as input for the decoder which maps it back to data space. The encoder and the decoder are then trained together by minimizing the loss of reconstruction between the input data and the reconstruction from its latent space representation. A special case of autoencoders that only use linear network layers with a lower dimensional latent space representation than the original data mirrors the behavior of principal component analysis [25, pp.563-564].

6.3.1 Variational autoencoders

A **VAE** is an autoencoder variant in which the latent space representation is characterized by probability distributions, enabling the generation of new samples from these distributions, which allows them to classify as a generative model [42, pp.1-5].

In contrast to regular autoencoders, a **VAE** is implemented using separate encoder and decoder networks. The encoder $q_\phi(\mathbf{z}|\mathbf{x})$ with weights ϕ maps the input data $\mathbf{x}$ to vectors of mean $\boldsymbol{\mu}_j(\mathbf{x})$ and variance $\boldsymbol{\sigma}_j(\mathbf{x})$ where $j = 1, \dots, M$ parameters, that define a Gaussian probability distribution with a diagonal covariance matrix in the latent space [25, p.572]. The decoder $p_\theta(\mathbf{x}|\mathbf{z})$ with model weights θ then allows drawing samples from the latent space distribution $p(\mathbf{z})$, which is assumed to be a standard normal distribution [42].

Defining the encoder, decoder and latent space this way is beneficial, as it allows drawing samples from the distribution $q_\phi(\mathbf{z}|\mathbf{x})$ via the reparameterization trick using

$$\mathbf{z} = \sigma\epsilon + \boldsymbol{\mu} \quad (6.1)$$

with $\epsilon \sim \mathcal{N}(0, 1)$, where ϵ is a sample of a standard normal distribution with a mean of zero and unit variance and allows the gradients to propagate through the stochastic layer, using standard backpropagation techniques [25, p.576].

The samples are then propagated through the decoder network defined by $p_\theta(\mathbf{x}|\mathbf{z})$, which allows for the evaluation of the **evidence lower bound (ELBO)** [25, p.576].

Using the reparameterization trick with Gaussian distributions allows for an analytical evaluation of the *Kullback-Leibler divergence* by

$$\text{KL}(q_\phi(\mathbf{z}|\mathbf{x})||p(\mathbf{z})) = \frac{1}{2} \sum_{j=1}^M (1 + \ln \sigma_j^2(\mathbf{x}) - \boldsymbol{\mu}_j^2(\mathbf{x}) - \sigma_j^2(\mathbf{x})) \quad (6.2)$$

between the two Gaussian distributions $q(\mathbf{z}|\mathbf{x}, \phi)$ and $p(\mathbf{z})$ [25, p.574].

The **ELBO** can then be maximized by minimizing the loss function for a given dataset $\mathbb{D} = \{\mathbf{x}_1, \dots, \mathbf{x}_N\}$ by

$$\mathcal{L} = \sum_{n=1}^N \left[\frac{1}{2} \sum_{j=1}^M (1 + \ln \sigma_{nj}^2(\mathbf{x}_n) - \boldsymbol{\mu}_{nj}^2(\mathbf{x}_n) - \sigma_{nj}^2(\mathbf{x}_n)) + \frac{1}{L} \sum_{l=1}^L \ln p(\mathbf{x}_n|\mathbf{z}_n^{(l)}, \boldsymbol{\theta}) \right] \quad (6.3)$$

,

where $\frac{1}{L} \sum_{l=1}^L \ln p(\mathbf{x}_n | \mathbf{z}_n, \theta)$ is the reconstruction loss calculated by sampling from $\mathbf{z}_n^{(l)} \sim q_\phi(\mathbf{z}_n^{(l)} | \mathbf{x}_n)$ multiple times with $l = 1, \dots, L$ using Equation 6.1 [25, p.575].

A trained VAE can therefore be used to generate new samples by sampling from the prior distribution of the latent space, which is assumed as a standard normal distribution $\mathbf{z} \sim \mathcal{N}(0, \mathbf{I})$ where $\mathbf{I}$ denotes the identity matrix and feeding them into the decoder to obtain samples in the data space [25, p.576].

Conditional variational autoencoders

Conditional generation of new samples can be employed, by instead of sampling from the unconditional latent space distribution $p(\mathbf{z}) = \mathcal{N}(0, \mathbf{I})$, sampling from a conditional distribution $p(\mathbf{z} | \mathbf{c})$ [25, p.576].

To obtain this, the encoder can be augmented by using the concatenated vector $\hat{\mathbf{x}} = (\mathbf{x}, \mathbf{c})^\top$ as input for the encoder. By this, the encoder learns the conditional distribution $q_\phi(\mathbf{z} | \mathbf{x}, \mathbf{c})$, and training a separate encoder $q_\psi(\tilde{\mathbf{z}} | \mathbf{c})$, which maps the conditioning vector to the latent space [43]. As a sum of two Gaussian distributions is also a Gaussian distribution, the latent space vectors of both the usual encoder and conditioning encoder can be added $\hat{\mathbf{z}} = \mathbf{z} + \tilde{\mathbf{z}}$ to obtain a latent space vector $\hat{\mathbf{z}}$ conditioned by $\mathbf{c}$. Therefore, both encoders can be trained jointly using the otherwise unchanged loss function defined by Equation 6.3 [44].

Similarly, the conditional variational autoencoder (CVAE) can be sampled from, by sampling from $\mathbf{z} \sim \mathcal{N}(0, \mathbf{I})$ and propagating $\hat{\mathbf{z}} = \mathbf{z} + \tilde{\mathbf{z}}$ for a given conditioning vector $\mathbf{c}$ using $q_\psi(\tilde{\mathbf{z}} | \mathbf{c})$ [44].

As this thesis exclusively employs conditional sampling, the CVAE will be referred to simply as VAE for the sake of brevity and simplicity.

6.3.2 Implementation

The general architecture of the single modules of a CVAE follows the explanation above and is composed of one neural network each for the encoder, decoder, and the encoder for conditioning information, which maps the conditioning information in the latent space for conditional sampling from the latent space.

The encoder network consists of a series of linear layers with different numbers of input and output nodes as well as non-linear leaky rectified linear unit (LeakyReLU) layers with a negative slope of 0.2 between hidden linear layers. The input layer of the encoder maps the input data to the specified size of the hidden layers. After the data passes through the hidden layers, the data is mapped to the latent space. Since the latent space in a VAE is constructed to be parameterized by Gaussian distributions, each dimension of the latent space needs to be parameterized by the two different parameters being the mean and typically their standard deviation. Because of this reason there are two different output layers which map the output of the hidden layers into the latent space for the specified amount of latent space dimensions. Both share the same parameters regarding the number of input nodes being the number of hidden nodes as well as output nodes specified by the number of latent space dimensions. Note that for improved numerical

stability the second output layer of the encoder outputs the log-variance of the Gaussian distributions instead of the standard deviation. While this is fundamentally equivalent, the loss function needs to account for this difference.

The conditioning encoder neural network follows the same general architecture of the encoder, however, it needs only one linear output layer instead of two directly mapping the last hidden layer to the latent space.

The decoder consists of a linear input layer from latent space to the first hidden layer, multiple pairs of linear hidden layers, and a linear output layer from the last hidden layer back to data space. The nonlinear layers are the same as the hidden layers of the encoder and conditioning encoder with the difference of being able to define a different number of hidden nodes per hidden layer as well as a different number of hidden layers. However, for this thesis, the encoder, decoder and conditioning encoder use the same number of hidden layers and hidden nodes to limit the amount of hyperparameters to tune. All linear layers in the encoder, decoder, and conditioning encoder are fully connected.

The final loss function for the training of the VAE can then be defined by using Equation 6.3 and the MSE for the reconstruction term by

$$\mathcal{L} = \sum_{n=1}^N \left[\beta \frac{1}{2} \sum_{j=1}^M (1 + \ln \sigma_{nj}^2(\mathbf{x}_n) - \mu_{nj}^2(\mathbf{x}_n) - \sigma_{nj}^2(\mathbf{x}_n)) + \alpha \frac{1}{L} \sum_{l=1}^L \|\mathbf{x}_n^{(l)} - \bar{\mathbf{x}}_n^{(l)}\|^2 \right] \quad (6.4)$$

Here $\bar{\mathbf{x}} \sim p(\bar{\mathbf{x}}|\mathbf{z}, \boldsymbol{\theta})$ is the reconstructed input data, obtained after sampling from $\mathbf{z} \sim q(\mathbf{z}|\mathbf{x}, \boldsymbol{\phi})$ using the reparameterization trick in Equation 6.1. Another set of hyperparameters introduced here are weights of the loss function α, β . Introducing weights is often necessary, as the Kullback-Leibler divergence (KL divergence) loss can lead to poor performance of the VAE when not weighted accordingly. This was observable during the implementation of the VAE for this thesis, and is also discussed in the literature [45, 46].

Similarly to the learning rate or batch size, this parameter was tuned manually on the problem. A generally good working estimate was found to be $\alpha = 1.0$ for the reconstruction loss and $\beta = 0.5$ for the KL divergence loss.

The hyperparameters of the VAE that are left to be tuned for the ablation studies are therefore:

1. Number of latent space dimensions
2. Number of hidden layers
3. Number of nodes per hidden layer

The VAE (or more specifically CVAE) developed for the purpose of this thesis and all its modules were implemented using the *Python* framework *PyTorch* [47].

6.3.3 Variational diffusion models

Diffusion models gained popularity for being able to generate high-quality and photorealistic images from noise, especially with *variational diffusion models* [48, 49]. The core idea is training a **deep neural network (DNN)** that is able to reverse the process of adding noise to the data called diffusion [25, pp.581-582]. Noisy data is repeatedly processed through the **DNN**, with each iteration incrementally reducing some of the noise from the data. Ultimately, this allows to generate data from pure noise [48].

During training, one sample of original training data $\mathbf{x}_0$ is incrementally corrupted with the noise generated according to a variance schedule $\beta_1, \dots, \beta_T$ resulting in intermediate latent space variables $\mathbf{x}_1, \dots, \mathbf{x}_{T-1}$ and the final latent variable $\mathbf{x}_T \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ [49, p.10]. The so-called *forward process* or *diffusion process* governs the incremental updates of added noise ϵ between variables using

$$q(\mathbf{x}_t | \mathbf{x}_{t-1}) = \mathcal{N}(\mathbf{x}_t | \sqrt{1 - \beta_t} \mathbf{x}_{t-1}, \beta_t \mathbf{I}) \quad (6.5)$$

or in closed form from $\mathbf{x}_0$ by

$$q(\mathbf{x}_t | \mathbf{x}_{t-1}) = \mathcal{N}(\mathbf{x}_t | \sqrt{\bar{\alpha}_t} \mathbf{x}_0, (1 - \bar{\alpha}_t) \mathbf{I}) \quad (6.6)$$

where $\alpha_t = 1 - \beta_t$ and $\bar{\alpha}_t = \prod_{s=1}^t \alpha_s$, which makes it possible to complete the diffusion process for any given timestep t in one step [48].

The training can then be performed as demonstrated in Algorithm 1 by minimizing the **MSE** of actual added noise $\epsilon \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ for a given sample from the dataset $\mathbf{x}_0$ and the timestep of the diffusion process t , and the noise predicted by the **DNN** $\epsilon_\theta(\sqrt{\bar{\alpha}_t} \mathbf{x}_0 + \sqrt{1 - \bar{\alpha}_t} \epsilon, t)$, which is also called *denoising score matching* [48].

Algorithm 1 Training of a diffusion model according to Algorithm 1 in [48].

- 1: **repeat**
- 2: $\mathbf{x}_0 \sim q(\mathbf{x}_0)$
- 3: $t \sim \text{Uniform}(\{1, \dots, T\})$
- 4: $\epsilon \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$
- 5: Take gradient descent step on

$$\nabla_\theta \|\epsilon - \epsilon_\theta(\sqrt{\bar{\alpha}_t} \mathbf{x}_0 + \sqrt{1 - \bar{\alpha}_t} \epsilon, t)\|^2$$

- 6: **until** converged
-

The *reverse process* can then be used to predict the variable $\mathbf{x}_{t-1}$ before a given diffusion step t for the reverse transition distribution approximated by the **DNN** $p_\theta(\mathbf{x}_{t-1} | \mathbf{x}_t)$ [48]. This can be computed by $\mathbf{x}_{t-1} = \frac{1}{\sqrt{\alpha_t}} \left(\mathbf{x}_t - \frac{1 - \alpha_t}{\sqrt{1 - \bar{\alpha}_t}} \epsilon_\theta(\mathbf{x}_t, t) \right) + \sigma_t \mathbf{z}$ as demonstrated in Algorithm 2 for a given timestep t , where $\mathbf{z} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ and $\sigma_t = \sqrt{\beta_t}$ [48].

Using Algorithm 2, a latent variable $\mathbf{x}_T \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ can be sampled, and then iteratively denoised for each timestep of the diffusion process $t = T, \dots, 0$ until the complete diffusion process is reversed.

Algorithm 2 Training of a diffusion model according to Algorithm 2 in [48].

```

1:  $\mathbf{x}_T \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ 
2: for  $t = T, \dots, 1$  do
3:   if  $t > 1$  then
4:      $\mathbf{z} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ 
5:   else
6:      $\mathbf{z} = \mathbf{0}$ 
7:   end if
8:

```

$$\mathbf{x}_{t-1} = \frac{1}{\sqrt{\alpha_t}} \left(\mathbf{x}_t - \frac{1 - \alpha_t}{\sqrt{1 - \bar{\alpha}_t}} \epsilon_{\theta}(\mathbf{x}_t, t) \right) + \sigma_t \mathbf{z}$$

```

9: end for
10: return  $\mathbf{x}_0$ 

```

Classifier free guidance

To add conditioning information to the diffusion model, a popular method is to employ classifier-free guidance (CFG). In contrast to classifier guidance (CG), CFG does not use a separate classifier that needs to be trained, but only uses a single DNN for conditional and unconditional sampling, learning the conditional distribution in a similar way, as discussed for the CVAE with $\hat{\mathbf{x}} = (\mathbf{x}, \mathbf{c})^\top$ [50]. This is advantageous, as the continuous conditioning features in the datasets do not allow for classification.

6.3.4 Implementation

The diffusion model architecture is much simpler compared to CVAE as it consists of only one neural network. Instead of separate neural networks for the residuals and the conditioning, the input and output of the neural network consists of the concatenated residuals, conditioning, and the denoising step. A linear input layer maps these vectors to the first linear hidden layer. The hidden layers themselves follow the same architecture as the hidden layers of the VAE. After the data passes through the hidden layers, a single linear output layer maps the data back to the data space. Additional diffusion-specific hyperparameters are the number of denoising steps as well as the CFG interpolation factor. Similar to the weights for the loss function of the VAE this value was tuned manually as there was no interdependency on the choice of the other hyperparameters found. The final value used for the ablation studies is 0.5 essentially equally weighting both conditional and unconditional sampling. Similar to the VAE, the diffusion model was implemented using the *PyTorch Python* framework [47].

The tunable hyperparameters of the diffusion model can therefore be enumerated by:

1. Number of denoising steps
2. Number of hidden layers

6 *Generative models for predicting residual forces*

3. Number of nodes per hidden layer

7 Validation metrics and visualization tools

To be able to compare and evaluate the ability of generative models to capture the distribution of the residual dynamics, it is necessary to find validation metrics that give quantitative and qualitative insights into the performance of the models. For this, the thesis employs metrics using the **MSE**, which is widely used as a loss function in supervised learning, and metrics using **MMD** in conjunction with nearest-neighbor search to find clusters of similar conditioning vectors in the validation dataset and their corresponding residual distributions.

7.1 Mean squared error

A straight-forward way to measure the performance of any model in supervised learning is to test the model on data for which the ground truth solution is known, for example, by using a separate validation dataset. Since both diffusion model and VAE use specific loss functions for training, it is necessary to find a suitable loss function for the validation of both models to achieve comparability. For regression tasks, **MSE** is an often used metric. Therefore, it was chosen to measure the performance in predicting the correct residual force vector containing the concatenated residual force of the payload and UAVs for a given conditioning vector.

Given the stochastic sampling process of generative models and that the generative models would sample each conditioning vector several times to get an estimate about the residual distribution, it makes sense not only to sample one time for each conditioning vector to calculate the validation loss, but also to compare the means of these residual distributions for each state. Assuming that residual dynamics follow a simple distribution, such as a Gaussian, and the generative models can accurately capture this distribution, averaging multiple samples of the residual forces for a given state may lead to more accurate predictions for the corresponding conditioning vector by reducing the impact of outliers and noise in individual samples.

For this reason, the validation **MSE** loss will be computed in two ways: First, based on a single sample per state. Then, after sampling and averaging the results for each conditioning vector for $n = 5$ times, with a corresponding value **MSE_n**. For this, all the conditioning vectors in the validation dataset are used. Since the **MSE** is sensitive to the scale of the values, it is calculated on the standardized and dimensionless residual vectors. As a consequence, the computed **MSE** values are also dimensionless.

7.2 Calculating maximum mean discrepancy loss on residual distributions

As already discussed, it is possible to obtain residual distributions from generative models for the same conditioning by sampling multiple times. However, it is not trivial to obtain ground truth residual distributions from the datasets. For this, the nearest-neighbor search and radius search are implemented using a KD-Tree, to be able to approximate residual distributions by finding conditioning vectors in the datasets that are similar under the proposed metric. For example, if the nearest-neighbor search is performed on states of a system, a corresponding state space metric can be used. For the purpose of this thesis, the standardized Euclidean distance as defined in Section 3.2.3 is used, as the evaluation is performed with a varying number of features in the conditioning vectors. Because of this, the term state space metric is not suitable here. The standardized Euclidean distance can be applied for arbitrary dimensional vectors, making it a good choice for the varying dimensionality of the conditioning vectors used.

Under the assumption that these found conditioning vectors are indeed very similar to each other, their corresponding residual forces can approximate the residual distribution around the selected conditioning vector being the center of the search. To do this efficiently, a KD-Tree is trained on the entire set of conditioning vectors of the given validation dataset. The trained KD-Tree is then used to query a given conditioning vector with a fixed radius. Then a group of conditioning vectors found is used to sample residual forces from the generative model. The similarity between the sampled distribution of residual force and the corresponding ground truth residual force from the validation dataset is then calculated using MMD.

The KD-Tree used in this thesis is built using the function `sklearn.neighbors.KDTree` from the *Python* module *Scikit-learn* [51].

For the MMD metric, the RBF kernel is chosen due to its general-purpose nature, and flexibility capturing both linear and non-linear relationships [32, 33].

The implementation of MMD used in this work was sourced from [52] without additional modifications. In this, the chosen RBF kernel is used with different bandwidth values of $\sigma^2 = \{10, 15, 20, 50\}$. The kernel results are summed to capture the differences in the distributions across smaller and larger scales.

Since this approach heavily depends on the similarity of the conditioning vectors, finding a suitable query radius is crucial for this approach. Although small query radii produce clusters with highly similar conditioning vectors, which are likely to provide more accurate approximations of the residual dynamics at the conditioning vector queried, there is a higher risk of finding only a few similar states within the given radius, which is a problem, as meaningful distributions rely on a sufficiently high number of samples. Therefore, it is necessary to balance the small search radius, so that all conditioning vectors in a cluster are as similar as possible versus having sufficient vectors per cluster, so that the corresponding residual forces can be used to accurately approximate residual distributions.

Since the search radius is crucial for this, two different search radii are used for the

validation of each model: The general idea is to enforce clusters of certain sizes for a given upper bound of a search radius. This is done by first searching for the nearest-neighbor search for all states in the validation dataset for a given number of nearest neighbors k . This allows to calculate the mean distance of the neighbors for each cluster, and subsequently the mean of all of those clusters, giving an estimate for an optimal query radius that maximizes the number of clusters of size k . By querying again all states in the validation set, this time with the fixed mean distance calculated, many clusters of the target size can be obtained from the dataset. To further constrain the size of the clusters, the clusters are filtered by their number of points. For this thesis, the minimum amount of points per cluster is set to 10 and the maximum amount of points per cluster is set to 30 to ensure that there are enough clusters of the specified size in each validation dataset.

The largest search radius is calculated using the mean distance between conditioning vectors in a cluster after performing the nearest-neighbor search for a fixed number of neighbors with $k = 20$, and averaging again across all clusters. For the smaller search radius, the process is repeated for a second time, this time querying for $k = 2$ neighbors. By keeping the clusters sized between 10 and 30, only densely populated regions of conditioning vectors in the validation dataset are used, even if their number of vectors per cluster is not representative for the given search radius. This gives a good estimate of residual distributions due to the small search radius, while the selected cluster sizes ensure good evidence for the distributions on the cost of having much fewer clusters to use for comparison against the generative models.

The obtained clusters of conditioning vectors are then used for sampling residual forces from the generative model. The resulting distributions are then compared with the ground truth residual forces using the MMD, with their respective values MMD for the larger search radius and $\text{MMD}_{\text{small}}$ for the smaller search radius. Similarly to the MSE loss, the MMD is calculated on the standardized and dimensionless residual vectors.

7.3 Calculating the average inference time

The real world applications like compensation of residual force or real-time decision making, can have different requirements for the sampling frequency the models must support. For the generative models, this is largely dependent on the time it takes to sample one batch of data known as the inference time. This greatly depends on computational resources and implementation details, which is why the corresponding values allow only for a relative comparison. Here, the mean inference time for a single batch of data is calculated by performing conditional sampling $n = 100$ times.

7.4 Qualitative analysis and plots

In addition to quantitative measures, visualizations such as graphs can provide more detailed information on the behavior of the models. To gain insight into temporal patterns

or variations in residual dynamics that are not captured by the nearest-neighbor search, the generative models can be used to sample the residual force using the conditioning vectors in the correct order from a single test flight in the validation dataset. This allows for the visualization of the residual forces by plotting them as functions over time. In the same way as discussed for the validation **MSE** loss, each point of conditioning can be used to sample multiple times to obtain an estimate of the distribution of the generated samples, allowing an estimate for the uncertainty of the models predictions. It is important to note that a prediction with high accuracy can have multiple reasons. A high-variance sampled distribution might accurately reflect a highly multimodal ground truth distribution, but it could be inaccurate when predicting a residual forces if the underlying ground truth distribution is low-variance and unimodal. In other words, the variance of the sampled residual distributions alone are no measure for the performance of a generative model, without the additional knowledge over the structure of the ground truth residual distributions.

Using **MMD** with clustering, solves this issue, as it compares the structure of the sampled residual distributions to the ground truth residual distributions. However, while the **MMD** only quantifies their similarity, it does not give information on the structure of the residual distributions. Therefore, it might be interesting to visualize the distributions of residual force the **MMD** is calculated on. Given the high dimensionality of the conditioning vectors and residual force vectors for a given system, there needs to be some sort of dimensionality reduction performed beforehand.

This is achieved by employing **PCA** to project both the conditioning vectors and the residual force vector onto their respective first two principal components. For the residual force vectors, kernel PCA is used with an **RBF** kernel to match the kernel used in **MMD**. Tests with varying values for the width of the **RBF** kernel gave the best results for $\sigma^2 = 1$. The **PCA** transformation for the conditioning vectors is performed on the entire validation dataset by extracting all the conditioning vectors. For the single residual force for each UAV in the **MPS** to be comparable, the transform **PCA** should not map the concatenated residual force vector of the whole system in two-dimensional space, but take the individual three-dimensional residual force vectors $\mathbf{f}_{ai}$ as input. Therefore, the **PCA** transform is fitted on all three dimensional residual force vectors $\mathbf{f}_{ai}$ of the concatenated residual vectors present in the validation dataset. The function for fitting the **PCA** used is `sklearn.decomposition.PCA` and `sklearn.decomposition.KernelPCA` respectively from the *Python* module *Scikit-learn* [51].

Another way of trying to visualize the residual distributions is to use histograms, to compare the sampled residual forces to the ground truth residual forces. Here, this is done by computing the magnitudes of each residual force vector, as this makes comparison easier. However, the loss of direction of the three-dimensional vectors has a drastic effect on the residual distributions, particularly in relation to oscillations that may occur in specific directions.

8 Ablation studies

Since the other hyperparameters are difficult to tune manually, ablation studies were conducted with the goal of finding sets of hyperparameters for optimal performance across different test settings. The method used to find estimates for the optimal set is multiple iterations of grid searches. Starting from relatively small values to bigger values using the prior iteration as guesses for performance improvement. Grid search is an exhaustive way of searching the hyperparameter space for the generative model specific hyperparameters as discussed in Chapter 6 by training on each element of the Cartesian product of the sets of all selected values for each hyperparameter. The hyperparameters shared by both were generative models were tuned manually and set to the values according to Table 8.1.

This method was used because of its systematic nature in finding the best possible combination of all hyperparameters. Since many hyperparameters have interdependent influence on the model performance, most hyperparameters have to be searched in this way. To restrict the space of hyperparameters the VAE model that can in theory have a different number of hidden nodes and hidden layers for the encoder, decoder and conditioning encoder, all three of those modules share the same number.

Since the number of UAVs in the system affects the number of features for input, output, and conditioning, ablation studies were performed separately for both the team sizes 2 and 3.

The datasets used for training and validation are the combined datasets from the repeated flights of the flight problem *window* and *forest* using the *optimized* reference trajectories. To achieve a training and validation split of the datasets, the validation is done on reference trajectories from the *geometric* planner. Because there are no datasets for a reference trajectory from the *geometric* planner for the flight problem *forest* with for a team of 3 UAVs, the validation dataset consists only of the *geometric* reference trajectories for the flight problem *window*. While other methods like cross validation are often used to achieve a split of training and validation data, this approach of testing against a different reference trajectory should give a better intuition on how well the model can perform in real world scenarios where the MPS needs to follow slightly different trajectories.

All training was conducted with the maximum number of conditioning features using *full regression conditioning* to limit the search space further.

To avoid overfitting, all training was done using early stopping as explained in Section 6.2 after 20 validations without improvement. As the validation occurs every 20 epochs of training, this means, that the training is stopped early after no improvement of 400 epochs.

Hyperparameter	Value
Maximum number of training epochs	4000
Batch size	128
Learning rate	1×10^{-4}

Table 8.1: Values for the shared hyperparameters used in the ablation studies and final models.

All training was performed on a single NVIDIA GeForce RTX 3080 with 10 GiB of memory.

8.1 Choice of final hyperparameters

The ablation studies show that not all metrics agree on which model is best. Therefore, the choice of the optimal hyperparameters corresponding to the best performing model for the ablation studies must be based on the choice of one metric, or by accounting for multiple metrics. Since for the purpose of this thesis the generative models are compared regarding their fitness replicating the residual distributions accurately, the metric $\text{MMD}_{\text{small}}$ is chosen as the deciding values, as its small search radius shows the theoretically achievable performance using the best available validation data. Another reason for this choice is that the sensitivity of the values MMD and $\text{MMD}_{\text{small}}$ is much higher than for the values of MSE or MSE_n .

For the repeated grid search for different sets of hyperparameters, certain sets are trained and validated multiple times as demonstrated in Table 8.3. Despite identical hyperparameters, some metrics like $\text{MMD}_{\text{small}}$ can deviate significantly. Especially the average inference time deviated significantly between iterations for the models using 8 latent space dimensions. Due to the stochastic nature of the training process, such as the shuffling of training data, stochastic gradient descent, and random initialization of the model weights, repeated training iterations yield unique models with varying performance.

This shows how averaging each metrics across multiple runs for each combination of hyperparameters would improve the final verdict on which sets of hyperparameters yield the highest performance.

8.2 Results

8.2.1 Variational autoencoder

The best performance according to the metric $\text{MMD}_{\text{small}}$ for the team of 2 UAVs MPS was achieved using a latent space of 8 with a shared count of three hidden layers with a count of nodes 2048 per layer. Adding more hidden nodes or dimensions to the latent space had no significant effect on the performance of the 2 UAVs MPS. Increasing the number of hidden layers beyond the maximum of three in the initial run also did not further increase the model performance for both metrics using MMD. The general trend for a higher MSE correlating with an increase in the number of latent space dimensions also manifested itself in an increase of hidden layers. As expected, especially the average inference time increases with the number of hidden layers used, therefore, a low count of hidden layers is preferable if the increase of performance for more layers is not significant.

For the 3 UAVs team model, the ablation studies produce the lowest values in three out of five metrics with a set of hyperparameters of 8 latent space dimensions, 2 hidden layers and 4096 hidden nodes. However, since one of the metrics that does not reach

Run	Dim.	Layers	Nodes	MMD	MMD _{small}	MSE	MSE _n	Inf. time
1	8	2	1024	0.309	0.108	0.811	0.527	0.000526
2	8	2	1024	0.300	0.141	0.865	0.543	0.000902
1	8	2	2048	0.346	0.109	0.758	0.512	0.000435
2	8	2	2048	0.280	0.132	0.768	0.537	0.000934
1	16	2	1024	0.293	0.105	0.941	0.547	0.000519
2	16	2	1024	0.200	0.118	0.851	0.534	0.000526
1	16	2	2048	0.258	0.134	0.973	0.584	0.000604
2	16	2	2048	0.300	0.178	0.982	0.603	0.000604

Table 8.3: Excerpt of ablation study results for the **VAE** from different runs for a team of 2 UAVs. *Run* denotes the iteration using the corresponding set of conditioning features, *Dim.* is the number of latent space dimensions, *Layers* the number of hidden layers and *Nodes* the number of nodes per hidden layer. *Inf. time* is the average inference time calculated in seconds after sampling $n = 100$ batches of using a conditioning vector $\mathbf{c} = \mathbf{0}$.

the lowest value with this set is MMD_{small}, which was the deciding factor for the final hyperparameters for the team of 2 UAVs, it should also be used to determine the final hyperparameters for the team of 3 UAVs to maintain consistency. In this instance, this involves using 3 instead of 2 hidden layers, which theoretically should not restrict the model’s representational capacity, but will increase average inference time. Similar to the 3 UAVs system, adding more hidden layers does not improve performance, as configurations with fewer hidden layers perform better in each metric, as shown in Table 6.

Consequently, the final selection of hyperparameters for the **VAE** to be applied to teams of 2 and 3 UAVs is illustrated in Table 8.4.

Team Size	Hyperparameter	Value
2 UAVs	Latent space dimensions	8
	Hidden layers	3
	Hidden nodes	2048
3 UAVs	Latent space dimensions	8
	Hidden layers	3
	Hidden nodes	4096

Table 8.4: Final hyperparameters for the **VAE** for teams of 2 and 3 UAVs.

8.2.2 Diffusion model

Discussion

In contrast to the [VAE](#) the ablation studies for the diffusion model show a much higher variance of values in many metrics. The values for [MSE](#) are especially interesting, as there are some combinations of hyperparameters that suggest some form of training instability, rather than being a representative result. The graphs for the resampled test flight indicate predictions for residual forces that are significantly higher than the ground truth residual forces. This discrepancy probably accounts for the high error rate, given that the [MSE](#), due to its reliance on the arithmetic mean, is highly influenced by outliers. In this case, the outliers are very large, but occur infrequently, as demonstrated by Figure [8.1](#).

The comparable number of validations for each unstable configuration compared to more stable runs does suggest that too early stopping of the training is likely not the issue for this.

For the 2 UAVs team, the best performance was found according to three of the five metrics, including the deciding MMD_{small} uses a combination of 2 hidden layers, 2048 hidden nodes, and 200 denoising steps. Especially comparing this configuration with the other for the metric MMD show a particularly high performance with a value less than half of that of the next-best configuration using only a single hidden layer instead of two.

Using the MMD_{small} metric to find the best hyperparameter combination for the 3 UAVs team would suggest using 2 hidden layers, 512 hidden nodes, and 100 denoising steps. The corresponding value for the MSE of over 90 suggest a failed training due to instability, therefore, in this case the choice for the final combination of hyperparameters used for the 3 UAVs system was based on the second best value for the MMD_{small} metric with an approximately 5% worse value of 0.057 instead of 0.054. This configuration, using 3 hidden layers and 2048 hidden nodes and 100 denoising steps, also performs well in terms of the other metrics, giving the second best values for MMD and MSE as well as the best value for MSE_n . However, the added hidden layer increases the average inference time by approximately 10% per batch.

As this configuration used the maximum amount of hidden layers and hidden nodes tested in the respective run, an additional ablation study was conducted that explored the performance of even more complex models using more hidden layers or hidden nodes, with the results demonstrated in Table [12](#). Using 5 hidden layers gives even better performance for the unchanged 2048 hidden nodes per layer with 100 denoising steps across all metrics except for the average inference time, yielding the final set of hyperparameters. Increasing the number of hidden nodes shows the computational limitations of the available workstation, making it unable to train and validate the model in some configurations.

The final selection of hyperparameters for the diffusion model of both team sizes is summarized in Table [8.5](#).

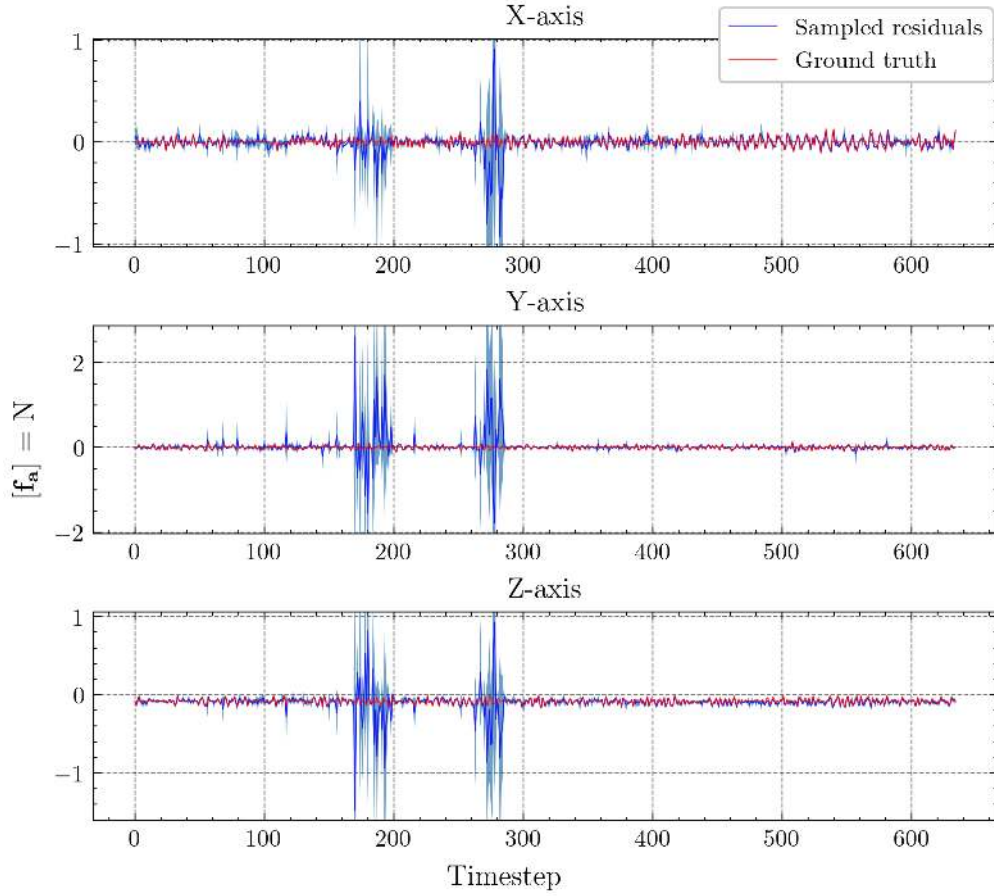


Figure 8.1: Prediction of the residual forces $\mathbf{f}_{a2}$ for one test flight, highlighting the share of intervals of low and high prediction error. Each timestep was sampled $n = 5$ times using the diffusion model for a team of 2 UAVs. The datasets used involved the conditioning set *reduced regression conditioning* and no smoothing. The plot shows the mean residual forces for each timestep as well as the one standard deviation (STD) confidence band. The sampling frequency is 100 Hz.

Team Size	Hyperparameter	Value
2 UAVs	Hidden layers	2
	Hidden nodes	2048
	Denoising steps	200
3 UAVs	Hidden layers	3
	Hidden nodes	2048
	Denoising steps	100

Table 8.5: Final hyperparameters for the diffusion model for teams of 2 and 3 UAVs.

9 Evaluation

As the work for this thesis involved generating datasets with and without smoothing, different sets of conditioning features as well as different generative models, this chapter tries to explore the impact of each choice in regards to the prediction of residual forces. For this, a series of experiments analogous to the ablation studies were conducted, involving all combinations of generative models, conditioning feature sets, MPS team sizes, and data sets using smoothed or non-smoothed data. Due to the number of possible combinations, the effects will be discussed using selected examples that align or contradict observed trends, using the validation metrics and visualizations discussed in Chapter 7. To be consistent with the ablation studies, all training datasets used the *optimized* trajectories for both flight scenarios *forest* and *window*, while the validation datasets consist of trajectories for the flight scenario *window* from the *geometric* planner. All models were trained for a maximum of 4000 epochs in combination with early stopping and choosing the model with the lowest validation loss for the evaluations. The sets of hyperparameters used are the ones discussed in Chapter 8. Furthermore, all models were trained from scratch for the purpose of the evaluation, which may explain the deviations in the metrics for the final models and ablation studies using the otherwise same settings.

9.1 General comparison of the generative models

9.1.1 Impact on the general performance

In contrast to the ablation studies, the generative models chosen for comparison as overall best performing are not based on the single metric, but rather based on achieving the lowest values for most metrics. In other words, the model with the highest number of best scores in all metrics is considered to be the best model overall, allowing for a more balanced comparison in several aspects. Furthermore, generative models are evaluated separately for both UAV team sizes. The metrics consistently show the best overall performance for the *full regression conditioning* conditioning set as datasets without smoothing across all combinations of generative models as well as UAVs teams sizes.

As demonstrated in Table 9.1, a direct comparison between diffusion model and VAE shows better scores for the diffusion model in all metrics except for the average inference time for both team sizes. Although the VAE achieved marginally better scores in the ablation studies than in the training of the final models discussed here, the diffusion model outperforms the VAE in all metrics, even if the scores of the ablation studies were matched.

9 Evaluation

UAVs	generative model	MMD	MMD _{small}	MSE	MSE _n	inference time
2	VAE	0.259	0.109	0.822	0.530	0.000754
2	diffusion model	0.020	0.025	0.185	0.051	0.087306
3	VAE	0.289	0.166	1.398	0.939	0.001258
3	diffusion model	0.070	0.075	0.230	0.119	0.141687

Table 9.1: Overall best performing generative models according to validation metrics across all different UAVs team sizes. All models were trained on datasets without smoothing and used the conditioning set *full regression conditioning*. The better scores for each model and UAV team size are highlighted.

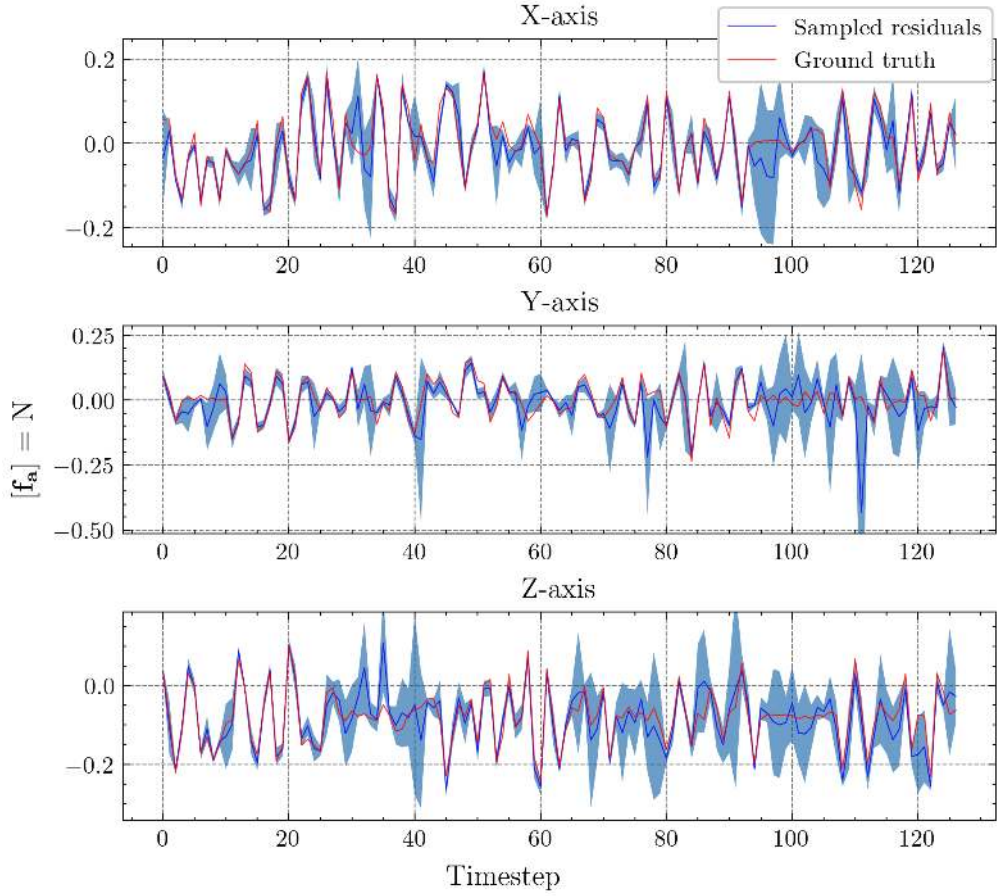


Figure 9.1: Prediction of the residual forces $\mathbf{f}_{a0}$ for a segment representing 20 % of the entire test flight. Each timestep was sampled $n = 5$ times using the diffusion model for a team of 2 UAVs. The datasets used involved the conditioning set *full regression conditioning* and the dataset without smoothing. The plot shows the mean residual forces for each timestep as well as the one **STD** confidence band. The sampling frequency is 100 Hz.

9.1 General comparison of the generative models

Comparing the individual residual forces for the duration of a single test flight, Figure 9.1 gives an additional visual proof of the very good performance of the diffusion model. Although the ground truth residual forces oscillates with a high frequency, the means of the residual force sampled by the diffusion model follow the original curve very closely, especially in high-frequency spikes. Only for small intervals involving no fast changing residual forces the predictions of the diffusion model show higher errors. Furthermore, for these lower residual forces the uncertainty regarding the prediction rises, as the confidence band visualizing one STD shows. Especially the consistent high accuracy of the mean residual force predictions give intuition for how low values with respect to the MSE_n metric in Table 9.1 reflect very good predictions. Similarly, the variance in residual force predictions depicted by the confidence bands reflect the improvement in performance from MSE to MSE_n by averaging over as little $n = 5$ samples per state compared to sampling only one time per state.

The combined information of the mean residual force predictions and confidence bands also suggest good performance of the diffusion model at approximating prediction uncertainties. For sections of the test flight in which the prediction of the diffusion model follows the curve of the ground truth residual forces closely, the confidence bands are often narrow, implying a low variance in residual distributions and therefore high prediction certainty. However, for sections of higher errors of the predicted mean residual forces, as seen around timestep 100 on the X-axis or timestep 480 on the Z-axis, the variance of predictions is also higher. This correlation of high variance and higher error of mean residual force predictions suggests the variance of samples by the diffusion model can be used to approximate the general prediction uncertainty for each state.

The Figure 9.2 mirrors the significantly worse performance of the VAE in predicting the residual forces compared to the diffusion model of the MSE validation metric scores. Nevertheless, unlike the metric scores which only compute an average error across all elements of the validation dataset, the figure illustrates that the inferior performance is consistent for every timestep of a single validation test flight, rather than manifesting exclusively within specific, smaller intervals of the test flight. Similarly, the confidence bands show no distinctive intervals in which the prediction uncertainty is significantly lower or higher.

Nevertheless, the overall comparison of the generative models therefore not only shows the superior performance of the diffusion model over the VAE, but also gives intuition about the limited expressiveness of the MSE metrics.

9.1.2 Residual distributions

Comparing the metrics MMD against MMD_{small} in Table 9.1, in addition to the diffusion model consistently achieving better values when compared to the VAE, the diffusion model performs similar for both MMD and MMD_{small} , while the values for the VAE deviate significantly between both search radii. For the team of two UAVs, the VAE achieves an approximately 57.7 % lower value for the MMD_{small} metric compared to the value for the MMD metric, and a 42.3 % value for the team of three UAVs respectively. The VAE therefore profits predicting residual distributions from clustering more similar

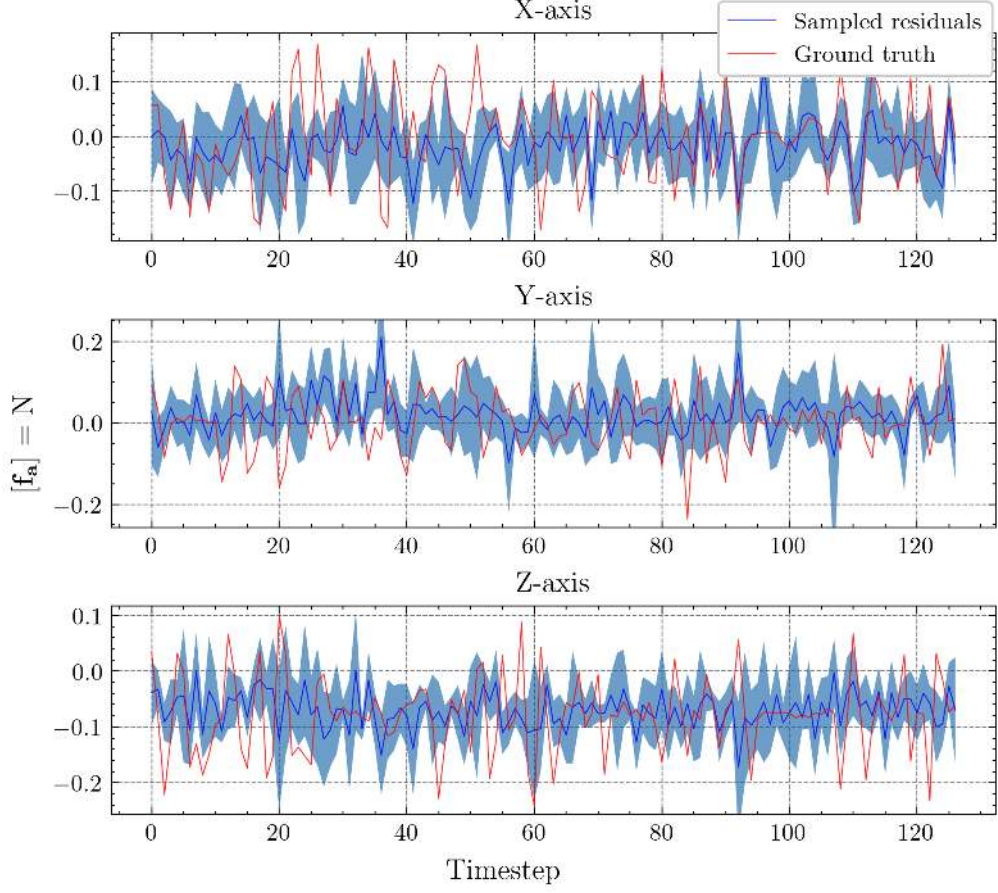


Figure 9.2: Prediction of the residual forces $\mathbf{f}_{a0}$ for a segment representing 20 % of the entire test flight. Each timestep was sampled $n = 5$ times using the **VAE** for a team of 2 UAVs. The datasets used involved the conditioning set *full regression conditioning* and the dataset without smoothing. The plot shows the mean residual forces for each timestep as well as the one **STD** confidence band. The sampling frequency is 100 Hz.

states.

In contrast, the scores of the diffusion model even increase by 28.9% for the team of two UAVs and by 6.8% for the team of three UAVs respectively. This counterintuitive decrease in performance for predicting residual distributions of more similar states demonstrates the difficulty of comparing high dimensional distributions by single values using MMD metrics, although the significantly lower overall MMD metric values that the diffusion model achieves align with the observations for the MSE metrics and the visualizations of a single test flight.

In contrast to this, visualizing the distributions of residual forces using Kernel-PCA does not show similarly better performance of the diffusion model replicating ground truth distributions, as Figure 9.3 shows. While the depicted cluster consists of similar conditioning vectors as seen in Figure 9.3, the distributions of ground truth residual forces are scattered across the whole embedded space. Similarly the predicted residual forces of both the diffusion model and VAE are in the proximity of the ground truth points for the selected cluster. The only exception for this is the residual forces vector component f_{a_2} , that shows a majority of the points sampled by both generative models close to the ground truth residual forces. However, the general structure of validation dataset points depicted suggest a much higher density of points near the border of the structure. This creates a bias for points appearing close to this border, with a resulting loss in expressiveness of samples appearing in the proximity of the ground truth residual forces. This is reflected by a seemingly similarly better prediction of residual distributions by the VAE.

The analysis of the residual force magnitudes in Figure 9.4 exemplifies that both generative models tend to have a bias towards higher residual forces and fail to capture the approximately Gaussian distribution of ground truth residual force magnitudes in the validation dataset. As a general trend, the VAE tends to capture the magnitude of residual forces better than the diffusion model.

As a result, the performance of the generative models in regards to their capability of capturing residual distributions is much more difficult to compare, as the seemingly better performance of the diffusion model in regards to lower values for MMD metrics is not reflected in the visualizations. Similarly, the opposing trends of increasing or decreasing performance for different search radii in clustering required more insights in the residual distributions themselves in regards to multi-modality to be explainable.

9.2 Comparison of the effects of team sizes

Comparing the results of the team with 2 and 3 UAVs for the best performing generative models, the models for a team of 2 UAVs perform better on average across almost all metrics than the models for a team of 3 UAVs, as demonstrated in Table 9.1.

Comparing the distributions of the magnitudes of residual force of a team of 3 UAVs in Figure 9.6 with those of a team of 2 UAVs in Figure 9.4, an even higher bias towards high residual forces becomes apparent, giving additional proof of lower performance.

One reason for this lower performance are the more complex aerodynamic and ca-

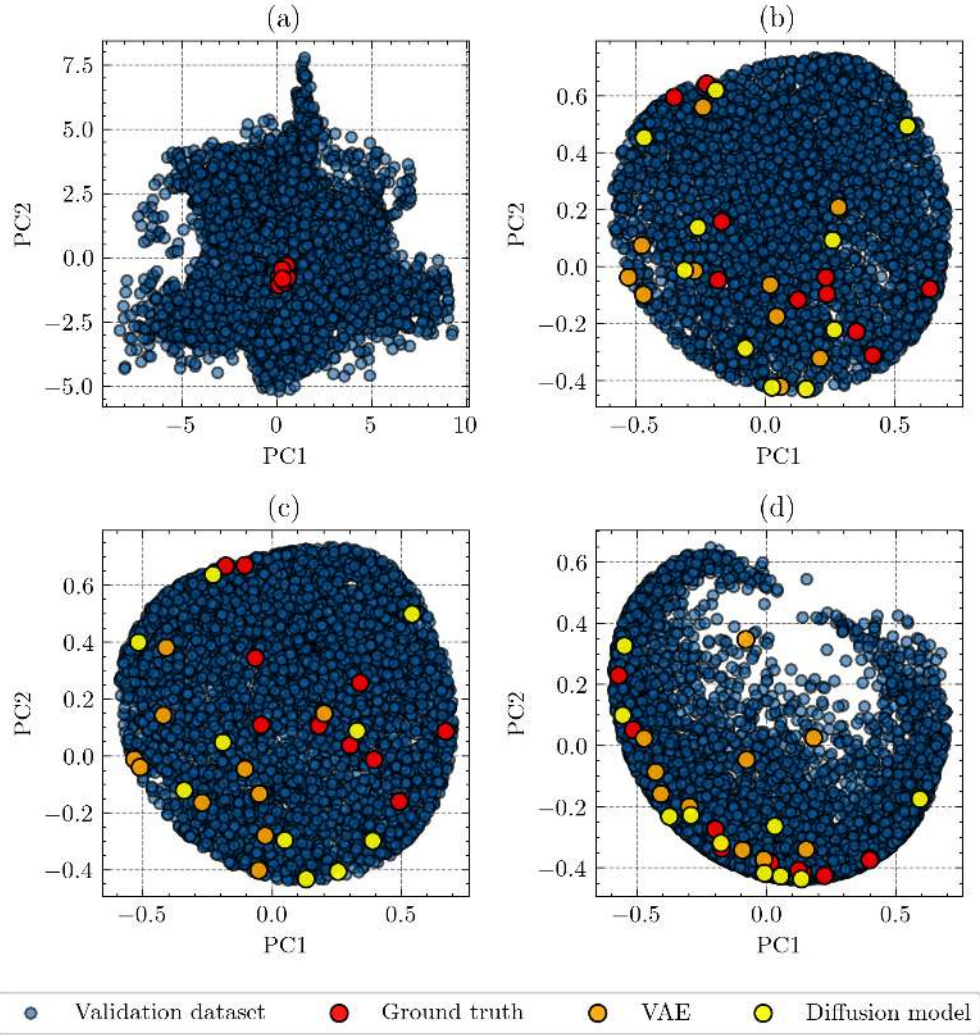


Figure 9.3: Visualization of the average residual forces after sampling for $n = 5$ for each state using two dimensional Kernel-PCA. A visualization of the distribution of states using linear PCA is shown in (a). The corresponding residual forces $\mathbf{f}_{a_0}$ [(b)], $\mathbf{f}_{a_1}$ [(c)], $\mathbf{f}_{a_2}$ [(d)]

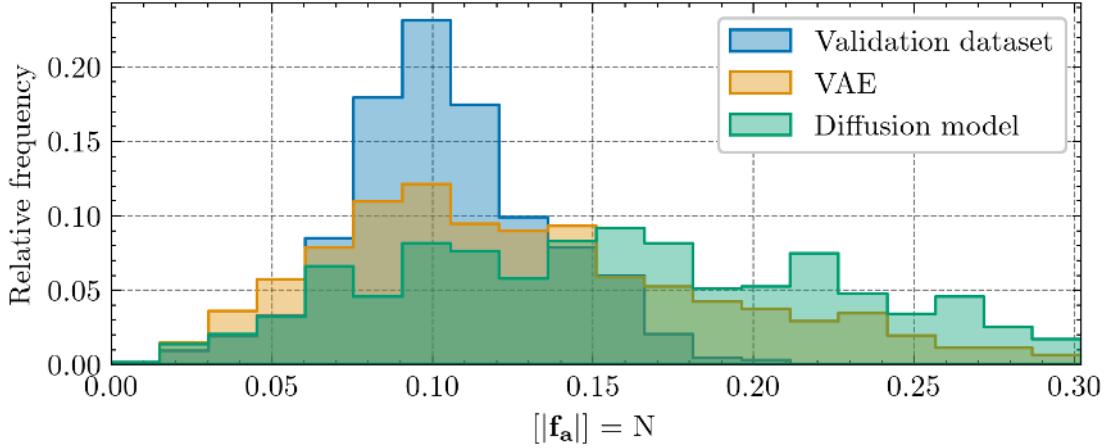


Figure 9.4: Distribution of the magnitudes of residual forces $|\mathbf{f}_{a1}|$ in the validation dataset and the corresponding sampled residual forces by the generative models after sampling $n = 5$ times for each state without applying average. The distributions show the relative frequency of residual forces magnitudes with the sum of all bins of each distribution equals 1, but high magnitudes are not shown for better visualization. The data corresponds to non-smoothed data and a team of 2 UAVs. The generative models were trained and validated using *full regression conditioning*.

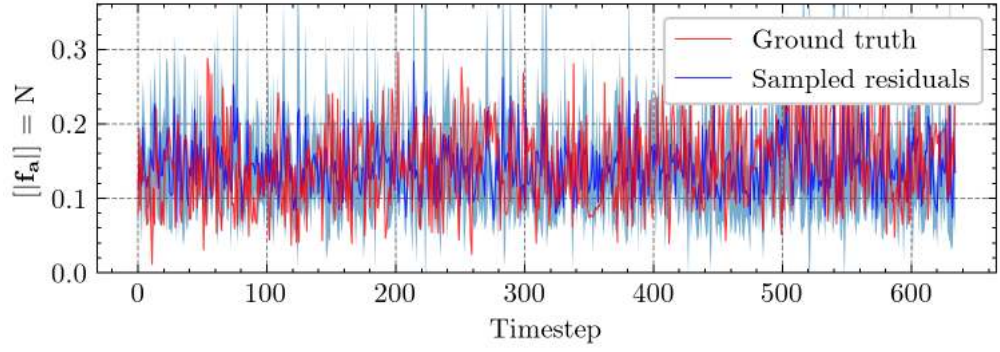
ble interactions of larger teams, and a resulting increase in difficulty for the regression problem. This increase is evident in the higher residual forces magnitudes for a team of 3 UAVs, which can reach 1.4 N, compared to the 2 UAVs team where magnitudes are below 0.32 N, as illustrated in Figure 9.5.

Another factor could be the limited computational resources, as the ablation studies showed a general trend of increasing performance with a greater model complexity. As the number of features of both residual forces and conditioning increases with larger team UAV team sizes, a corresponding rise in model complexity is expected to maintain similar performance. As a result, the performance of the 3 UAVs team is limited due to the complexity of the generative models being limited by the computational resources available during this study.

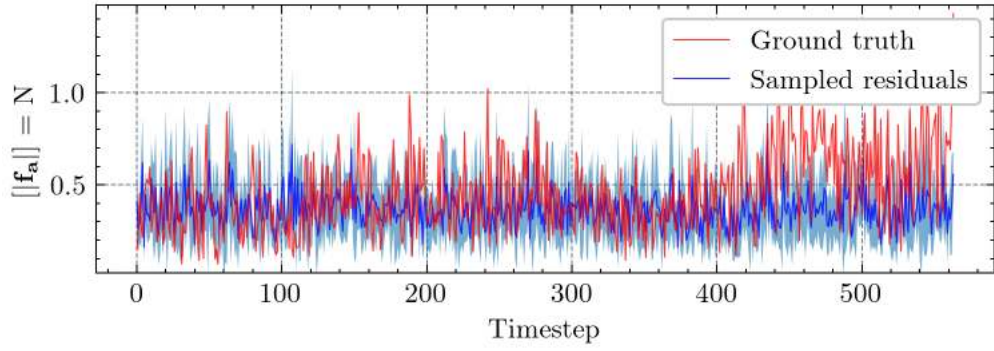
9.3 Effect of smoothing

9.3.1 Impacts of smoothing on overall performance

An initial approximation of the effect of using smoothed data is shown by comparing the validation metrics for models trained and validated on both smoothed and non-smoothed data shown in Table 9.2. Except for one instance, all models in both the 2 and 3 UAVs team configurations perform much worse on smoothed data across all metrics. For some



(a) 2 UAVs



(b) 3 UAVs

Figure 9.5: Comparison of the magnitudes of the residual forces $\|f_{a0}\|$ predicted by the diffusion model for one test flight in a 2 UAVs and 3 UAVs configuration.

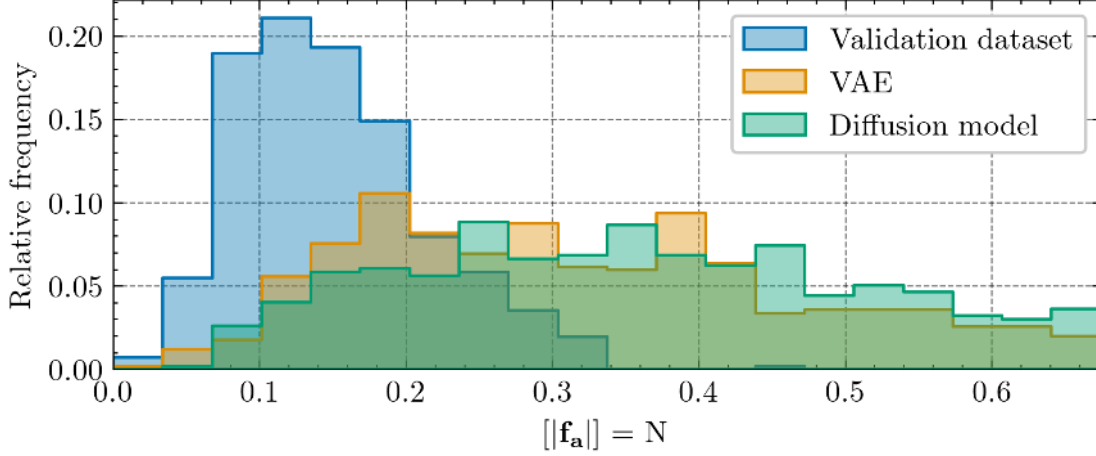


Figure 9.6: Distribution of the magnitudes of residual forces $|\mathbf{f}_{a1}|$ in the validation dataset and the corresponding sampled residual forces by the generative models after sampling $n = 5$ times for each state without applying average. The distributions show the relative frequency of residual forces magnitudes with the sum of all bins of each distribution equals 1, but high magnitudes are not shown for better visualization. The data corresponds to non-smoothed data and a team of 3 UAVs. The generative models were trained and validated using *full regression conditioning*.

models like the diffusion model for the 3 UAV team, the metrics achieve values that are approximately an order of magnitude higher in comparison to using non-smoothed data.

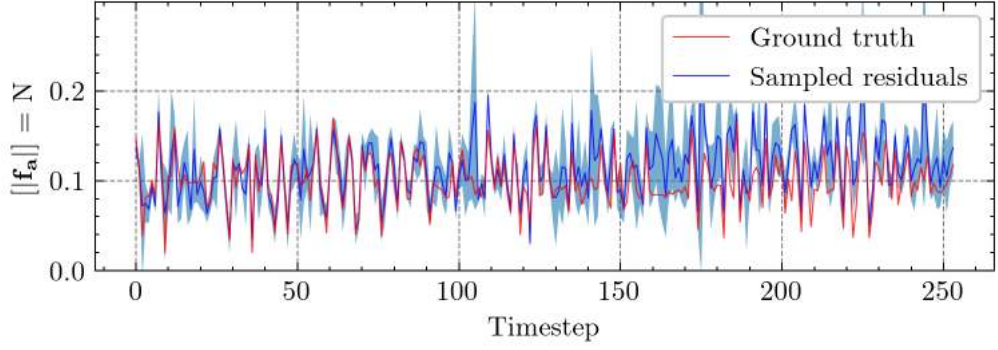
This effect can also be observed in the visualizations of residual forces predictions for a single test flight. In most other cases, where the generative models achieve poor performance for at least one axis of a given residual force vector $\mathbf{f}_{ai}$, computing the magnitudes of the residual force vector results in high errors compared to the ground truth residual force vectors. However, in the case of the diffusion model in the 2 UAVs team configuration that performs best according to the MSE_n metric, even the magnitudes align well with the ground truth values as seen in Figure 9.7. In direct comparison, the model that was trained and validated on smoothed data performs significantly worse in predicting residual forces.

A more detailed analysis of the diffusion model trained on smoothed data in Figure 9.8 shows a consistently high prediction error, although for timesteps in $[80, 110]$ the model captures general trends like sign changes when comparing the X-axis and Y-axis only. Since the ground truth residual forces vectors depicted in Figure 9.8 dominate in direction of the Z-axis, the magnitude of the residual forces vector is therefore affected the most by the component along this direction. Therefore, the poor prediction performance for this component introduces a bias when comparing the performance with respect to the magnitude of the vectors. For intervals of low differences between sampled and ground truth residual forces such as $[80, 110]$, the magnitudes shown in Figure 9.7 are aligned,

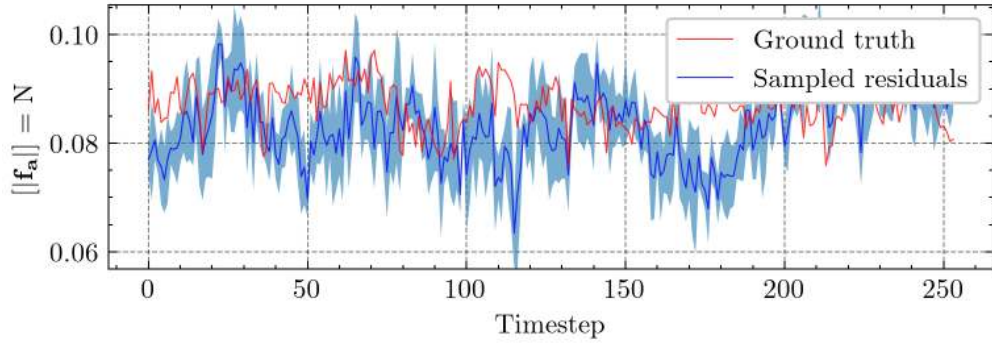
9 Evaluation

UAVs	Generative model	Smoothing	MMD	MMD _{small}	MSE	MSE _n
2	VAE	No	0.259	0.109	0.822	0.530
2	VAE	Yes	1.098	0.926	1.431	1.267
2	Diffusion model	No	0.020	0.025	0.185	0.051
2	Diffusion model	Yes	0.703	0.436	1.048	0.784
3	VAE	No	0.289	0.166	1.398	0.939
3	VAE	Yes	0.354	0.398	0.713	0.592
3	Diffusion model	No	0.070	0.075	0.230	0.119
3	Diffusion model	Yes	0.324	0.248	0.681	0.456

Table 9.2: Validation metrics for generative models in 2 and 3 UAVs teams using *full regression conditioning* for conditioning. Highlighted are the better scores for each number of UAVs and generative model.



(a) Dataset without smoothing



(b) Smoothed dataset

Figure 9.7: Comparison of the magnitudes of the residual forces $|\mathbf{f}_{a2}|$ predicted by the diffusion model for a segment representing 20 % of the entire test flight for a team of 2 UAVs using smoothed data and not smoothed data. The sampling frequency is 100 Hz.

although only the Z-axis component of the residual prediction is accurate. Although this explains some parts of the seemingly worse performance in terms of visualizations, it does not explain the significantly higher values of MSE and MSE_n as they are calculated on the standardized residual forces as opposed to visualizations that are based on their original scale.

In summary, the visualizations support the effect that manifests in the values of the validation metrics using MSE . Furthermore, the analysis of single test flights shows a consistently inferior performance, rather than for certain states.

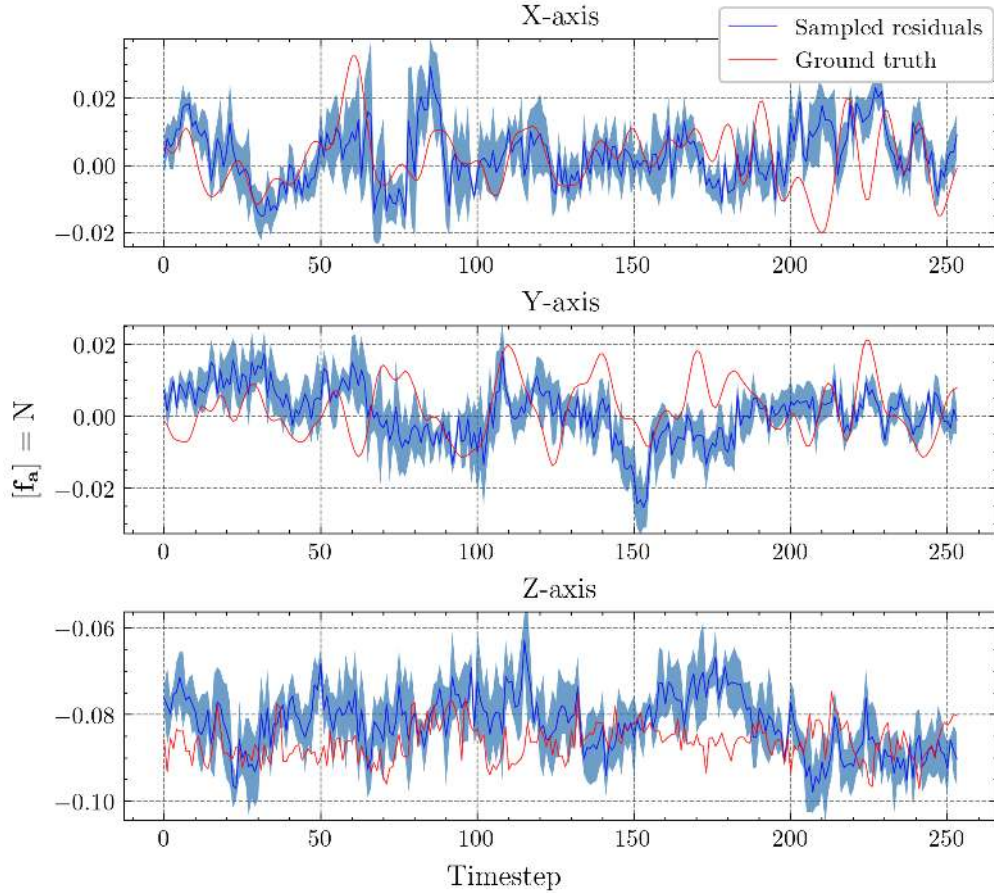


Figure 9.8: Prediction of the residual forces $\mathbf{f}_{a2}$ for a segment representing 40 % of the entire test flight. Each timestep was sampled $n = 5$ times using the diffusion model for a team of 2 UAVs. The datasets used involved the conditioning set *full regression conditioning* and the dataset with smoothing. The plot shows the mean residual forces for each timestep as well as the one STD confidence band. The sampling frequency is 100 Hz.

9.3.2 Effect on residual distributions

The validation metrics using **MMD** indicate that smoothing also has a strong negative effect in capturing residual distributions demonstrated in Table 9.2. However, this effect is not consistently reflected in the visualizations involving **kernel PCA** as depicted in Figure 9.9. For the selected cluster, both generative models do not capture the visual distribution of ground truth points well for the non-smoothed datasets. In contrast to this, diffusion model shows a distribution similar to the one of ground truth points. The distribution of points sampled by the **VAE** is shaped and located in a different section of the whole two-dimensional visualization space, suggesting an entirely different distribution.

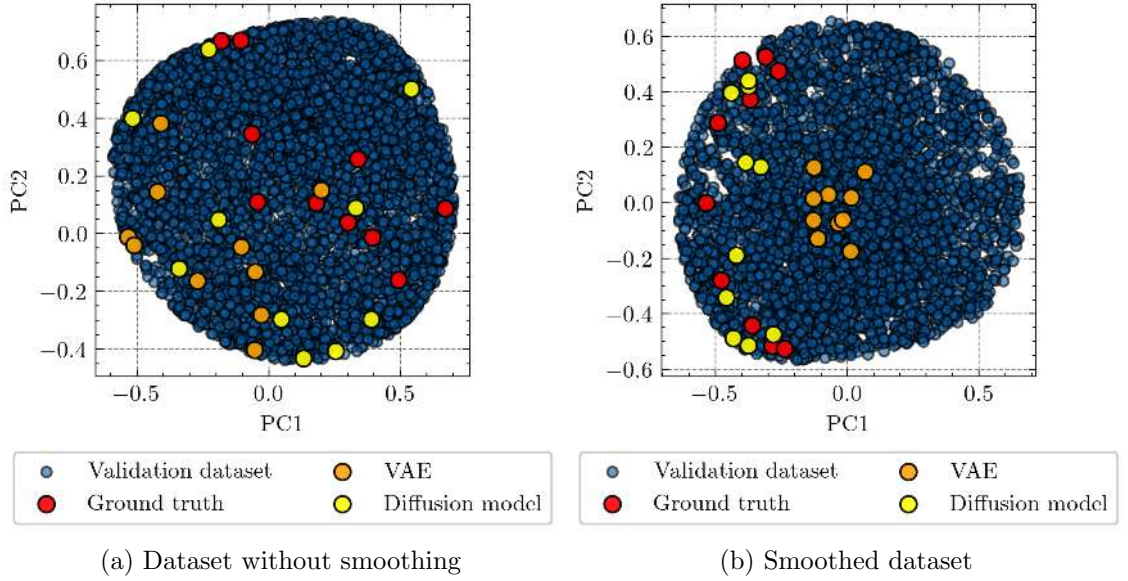


Figure 9.9: Visualization of the average residual forces using two dimensional **kernel PCA** after sampling for $n = 5$ for each state using the diffusion models trained on datasets with and without smoothing. The plots show the residual forces $\mathbf{f}_{a1}$ of a team of 2 UAVs using *full regression conditioning*

Comparing the distributions of the magnitudes of residual forces in Figure 9.10, the distributions sampled by the generative models trained on smoothed data capture the approximate Gaussian distribution of magnitudes in the validation dataset much better than the generative models trained on non-smoothed data that is shown in Figure 9.4.

However, just as in the comparison of diffusion model and **VAE** for predicting residual distributions in Section 9.1.2, visualizing distributions within single clusters offers more intricate observations, even though it might not align perfectly with the metric trends and thus its ability to offer enhanced insight is debatable.

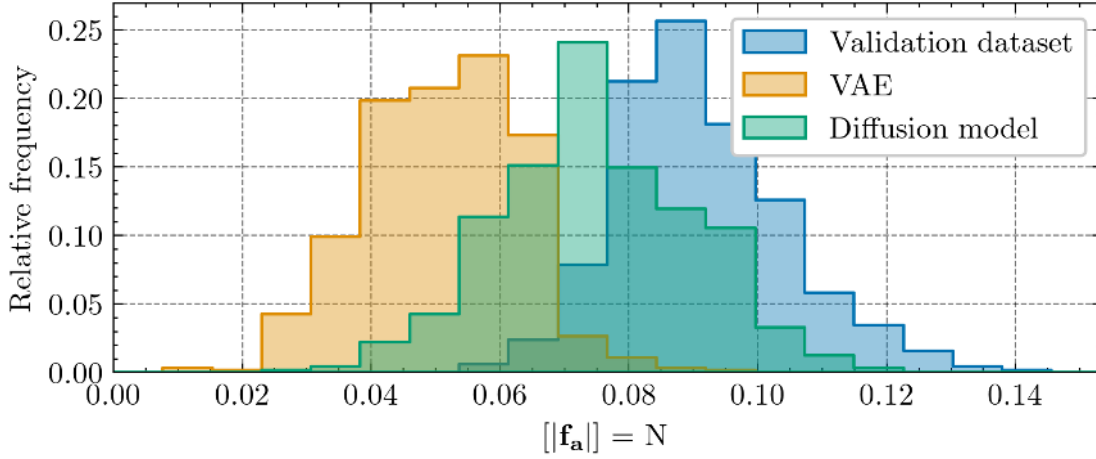


Figure 9.10: Distribution of the magnitudes of residual forces $|\mathbf{f}_{a1}|$ in the validation dataset and the corresponding sampled residual forces by the generative models after sampling $n = 5$ times for each state without applying average. The distributions show the relative frequency of residual forces magnitudes with the sum of all bins of each distribution equals 1, but high magnitudes are not shown for better visualization. The data corresponds to smoothed data and a team of 2 UAVs. The generative models were trained and validated using *full regression conditioning*.

9.4 Selection of conditioning features

Although the general trend of superior generative models performance using non-smoothed data also holds for all sets of conditioning features discussed here, a separate analysis of the effects of conditioning features choice for both non-smoothed data and smoothed data provides better insight.

9.4.1 Analysis using non-smoothed data

To get a first estimate of how the choice of conditioning features affects the general performance of the models, the plots can be compared qualitatively. Although many plots show poor overall performance even using the full conditioning information set *full regression conditioning*, there are exceptions that show good performance. Some models that use this set of conditioning manage to predict the residual forces during a test flight well, as Figure 9.11 demonstrates. In these cases, of models performing well using the full conditioning information, a general trend of increasingly worse performance with the removal of conditioning features can be observed.

Even when transitioning to the set *reduced regression conditioning*, which - in theory - contains all the necessary information for the regression of residual forces, the Figure 9.12 demonstrates how residual force predictions of high error and variance in certain small intervals can lead to high values for the metrics MSE and MSE_n in Table 9.4.

9 Evaluation

This is the first instance, in which the performance degrades in a non-consistent matter. Figure 9.12 suggests that the states that correspond to intervals of high prediction error (such as $[100, 110]$ and $[160, 180]$) are outliers with respect to the training dataset, as the magnitude of the predictions residual force is up to one magnitude higher than typical.

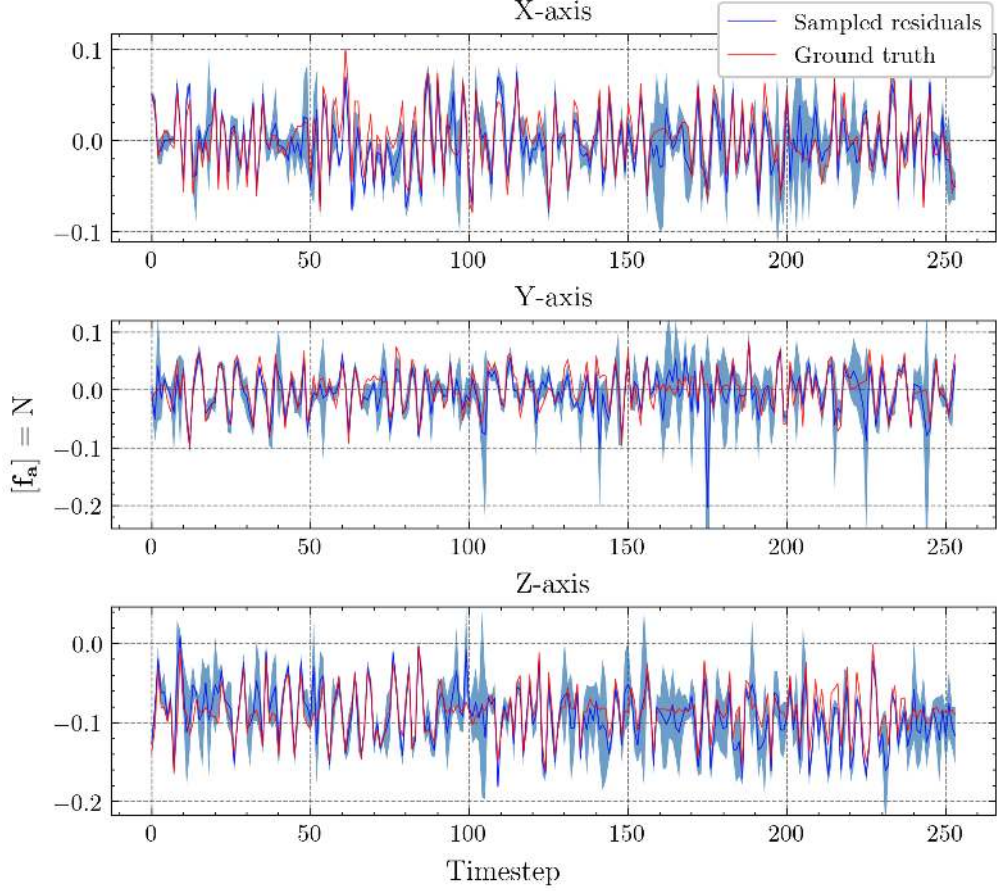


Figure 9.11: Prediction of the residual forces $\mathbf{f}_{a2}$ for a segment representing 40 % of the entire test flight. Each timestep was sampled $n = 5$ times using the diffusion model for a team of 2 UAVs. The datasets used involved the conditioning set *full regression conditioning* and no smoothing. The plot shows the mean residual forces for each timestep as well as the one **STD** confidence band. The sampling frequency is 100 Hz.

However, this phenomenon cannot be observed for a team of 3 UAVs with otherwise unchanged parameters, as Figure 9.13 shows. Judging purely by visualization, performance is not significantly affected by the transition from *full regression conditioning* to *reduced regression conditioning*, even though the metrics for **MSE** in Table 9.4 increase by over 100 %.

These conflicting observations are reflected in their respective values of the **MSE** met-

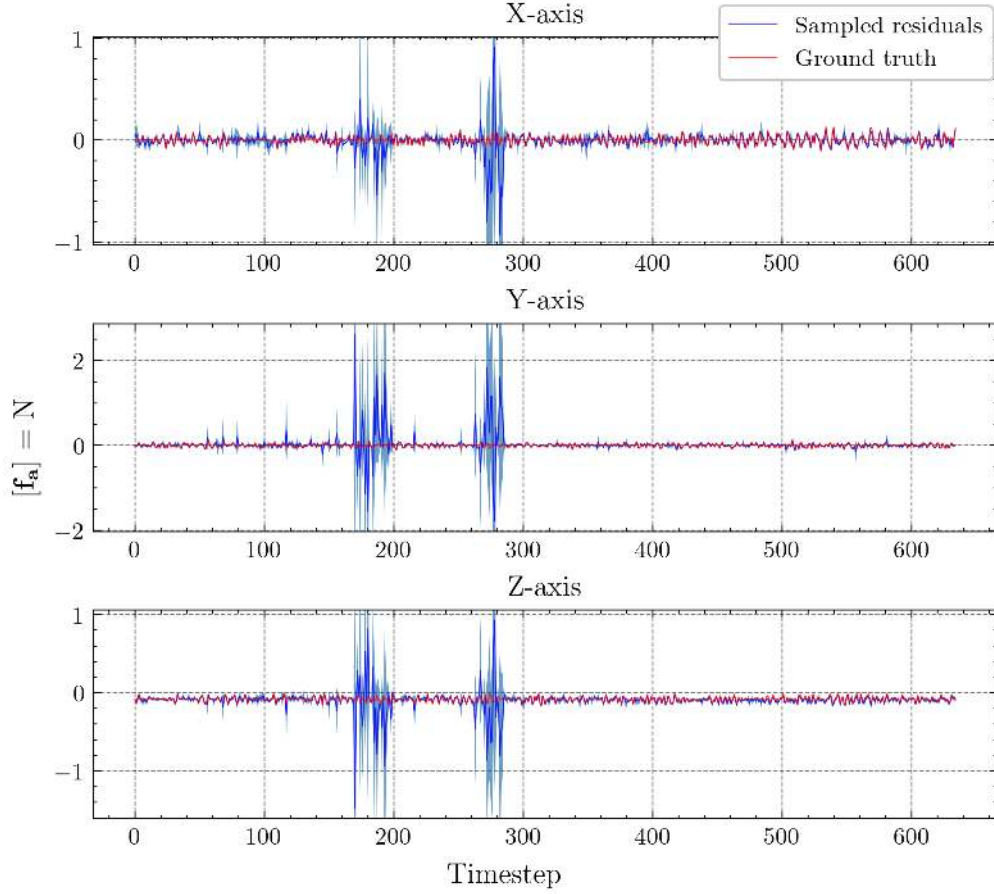


Figure 9.12: Prediction of the residual forces $\mathbf{f}_{a2}$ for one test flight, highlighting the share of intervals of low and high prediction error. Each timestep was sampled $n = 5$ times using the diffusion model for a team of 2 UAVs. The datasets used involved the conditioning set *reduced regression conditioning* and no smoothing. The plot shows the mean residual forces for each timestep as well as the one STD confidence band. The sampling frequency is 100 Hz.

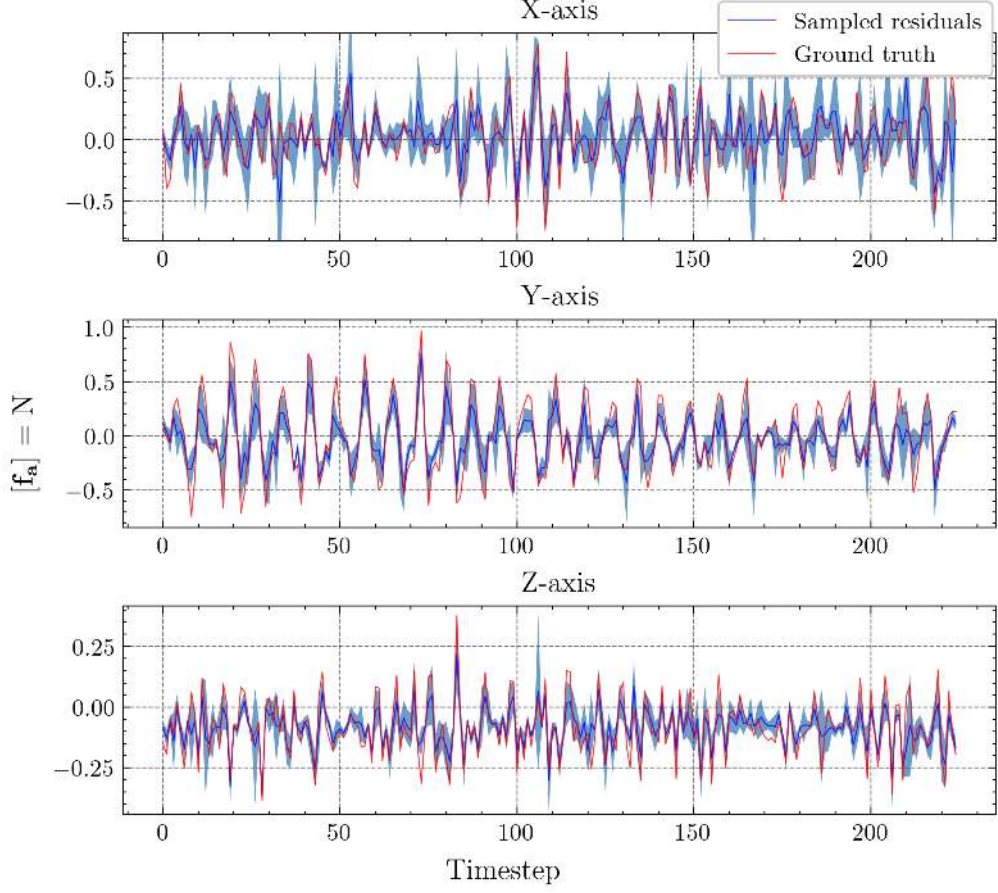


Figure 9.13: Prediction of the residual forces $\mathbf{f}_{a0}$ for a segment representing 40 % of the entire test flight. Each timestep was sampled $n = 5$ times using the diffusion model for a configuration of 3 UAVs. The datasets used involved the conditioning set *reduced regression conditioning* and no smoothing. The plot shows the mean residual forces for each timestep as well as the one STD confidence band. The sampling frequency is 100 Hz.

rics for the transition of *full regression conditioning* to *reduced regression conditioning*. Especially the value of the **MSE** metric without sampling multiple times increases by approximately 710 % for the 2 UAVs team, whereas the increase for the 3 UAVs team is approximately 146 %.

Team size	Conditioning set	MMD	MMD _{small}	MSE	MSE _n
2	<i>full regression conditioning</i>	0.259	0.109	0.822	0.530
2	<i>reduced regression conditioning</i>	0.302	0.130	0.802	0.531
2	<i>modified state conditioning</i>	0.174	0.121	0.868	0.575
3	<i>full regression conditioning</i>	0.289	0.166	1.398	0.939
3	<i>reduced regression conditioning</i>	0.597	0.199	1.431	0.964
3	<i>modified state conditioning</i>	0.412	0.179	1.514	1.000

Table 9.3: Validation metrics for the **VAE** using different sets of conditioning and non smoothed data. Highlighted are the best scores for each size of UAV teams.

Team size	Conditioning set	MMD	MMD _{small}	MSE	MSE _n
2	<i>full regression conditioning</i>	0.020	0.025	0.185	0.051
2	<i>reduced regression conditioning</i>	0.030	0.023	1.501	0.327
2	<i>modified state conditioning</i>	0.212	0.132	2.575	1.064
3	<i>full regression conditioning</i>	0.070	0.075	0.230	0.119
3	<i>reduced regression conditioning</i>	0.158	0.069	0.566	0.302
3	<i>modified state conditioning</i>	0.487	0.299	1.721	1.114

Table 9.4: Validation metrics for the diffusion model using different sets of conditioning and non-smoothed data. Highlighted are the best scores for each size of UAV teams

9.4.2 Effect on residual distributions

In contrast to the effect on the MSE metrics and the visualized accuracy in test flight plots, there are two conflicting trends for MMD metric values with a reduced number of conditioning features for the diffusion model, as demonstrated in Table 9.4. For both team sizes, the diffusion model consistently achieves better values of MMD_{small} for the *reduced regression conditioning* set, whereas the values increase in case of the MMD metric for the larger search radius. However, in the case of the VAE, the effect of increasing values for both MMD metrics with reduced number of conditioning features is consistent.

The general trend of degrading performance for replicating residual distribution for both generative models is reflected by the visualization in Figure 9.14a. The selected clusters are single instances of residual distributions of clusters used for the calculated

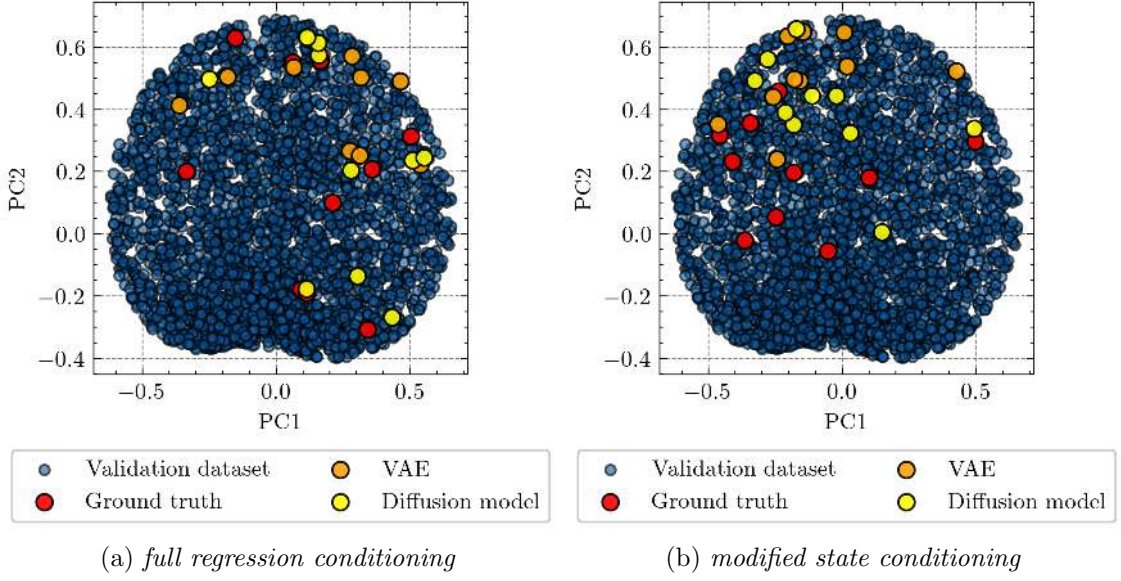


Figure 9.14: Visualization of the average residual forces using different sets of conditioning after sampling for $n = 5$ for each state using two dimensional **kernel PCA**. The plots show the residual forces $\mathbf{f}_{a1}$ for a team 3 UAVs and data without smoothing

$\text{MMD}_{\text{small}}$ for *full regression conditioning* and *reduced regression conditioning* respectively. Although the points representing the residual forces for *full regression conditioning* are spread afar over the overall domain of residual forces of the validation dataset, the sampled residual forces of both the diffusion model as well as the VAE are close to the ground truth points. The decrease in performance using *reduced regression conditioning* suggested by the metrics is then reflected in Figure 9.14b, where only a few points of residual forces are matched by samples from the ground truth residual forces. However, this conflicts with the discussed anomaly of better values for $\text{MMD}_{\text{small}}$ metric using *reduced regression conditioning* in the case of the diffusion model. Intuitively, the ground truth residual distribution should be matched better by the samples of the diffusion model in Figure 9.14b for the *reduced regression conditioning* than in Figure 9.14a for the *full regression conditioning*.

Possible reasons for discrepancies like this between the metrics and the visualizations are numerous. One possible reason is that the metrics are calculated using the average over a number of clusters or test flight states, while the plots show a single cluster, and therefore possible outliers. Another possible reason is the limited validity and expressiveness of both validation tools to capture residual distributions.

Furthermore, using a different set of conditioning features also influences the distributions of similar states selected using the KD-Tree. As a result, the clustering produces different distributions of states and corresponding residual distribution. The **MMD** only computes differences between distributions, but does not provide information about the

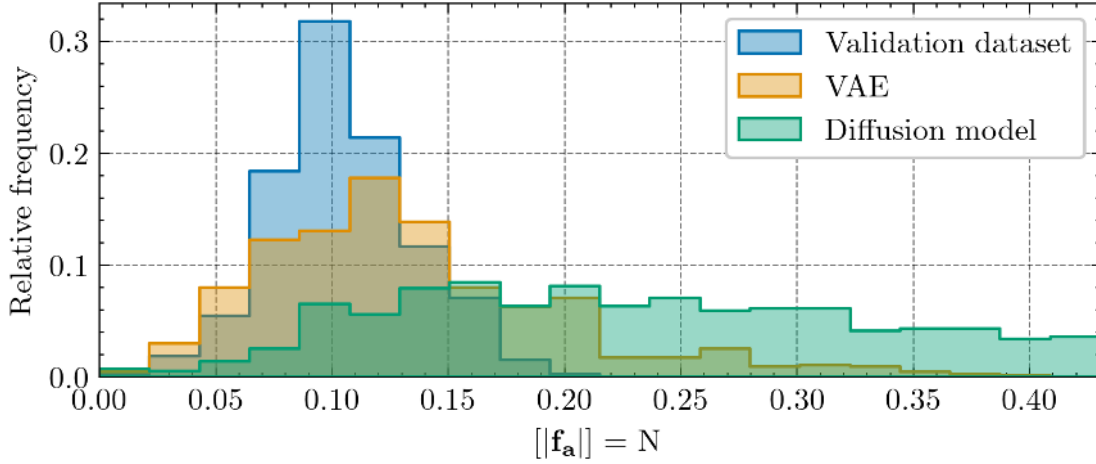


Figure 9.15: Distribution of the magnitudes of residual forces $|\mathbf{f}_{a1}|$ in the validation dataset and the corresponding sampled residual forces by the generative models after sampling $n = 5$ times for each state without applying average. The distributions show the relative frequency of residual force magnitudes with the sum of all bins of each distribution equals 1, but high magnitudes are not shown for better visualization. The data corresponds to non-smoothed data and a team of 2 UAVs. The generative models were trained and validated using *modified state conditioning*.

distributions themselves. Because of this, it is difficult to reason about trends in metric values if the underlying distributions experience drastic changes, which, in turn, could lead to predicting the distributions more or less difficult.

The analysis of the distributions of residual force magnitudes over the whole validation dataset and their respective samples from the generative models in Figure 9.15 provides additional support for the inferior performance in capturing residual distributions using *modified state conditioning* compared to *full regression conditioning* as indicated in Table 9.5. In particular, the superior performance of the VAE compared to the diffusion model with *modified state conditioning* can be observed.

9.4.3 Analysis using smoothed data

While the overall impact of poorer performance with the use of smoothed data mentioned in Section 9.3 or larger UAV team sizes in Section 9.2 typically holds for the various sets of conditioning features discussed here, there are interesting effects observable when analyzing the *modified state conditioning* in conjunction with smoothed data.

For example, in the case of a team of 2 UAV, both MSE metric values decrease significantly with the transition from *reduced regression conditioning* to *modified state conditioning*. However, Figure 9.16 shows that the *reduced regression conditioning* suffers from the same effect of certain intervals of high prediction error already present using

9 Evaluation

Generative model	Conditioning set	MMD	MMD _{small}	MSE	MSE _n
VAE	<i>full regression conditioning</i>	0.289	0.166	1.398	0.939
VAE	<i>reduced regression conditioning</i>	0.597	0.199	1.431	0.964
VAE	<i>modified state conditioning</i>	0.412	0.179	1.514	1.000
diffusion model	<i>full regression conditioning</i>	0.070	0.075	0.230	0.119
diffusion model	<i>reduced regression conditioning</i>	0.158	0.069	0.566	0.302
diffusion model	<i>modified state conditioning</i>	0.487	0.299	1.721	1.114

Table 9.5: Validation metrics for different sets of conditioning. All models use the same non-smoothed data for a team of 3 UAVs. Highlighted are the best scores for each generative model.

Team size	Conditioning set	MMD	MMD _{small}	MSE	MSE _n
2	<i>full regression conditioning</i>	0.703	0.436	1.048	0.784
2	<i>reduced regression conditioning</i>	0.646	0.612	5.508	2.16363
2	<i>modified state conditioning</i>	0.939	0.774	3.904	1.816
3	<i>full regression conditioning</i>	0.324	0.248	0.681	0.456085
3	<i>reduced regression conditioning</i>	0.312	0.199	0.674	0.498
3	<i>modified state conditioning</i>	0.365	0.404	0.734	0.569

Table 9.6: Validation metrics for the diffusion model using different conditioning sets and smoothed data. Highlighted are the best scores for each UAV team size.

non-smoothed data depicted in Figure 9.12.

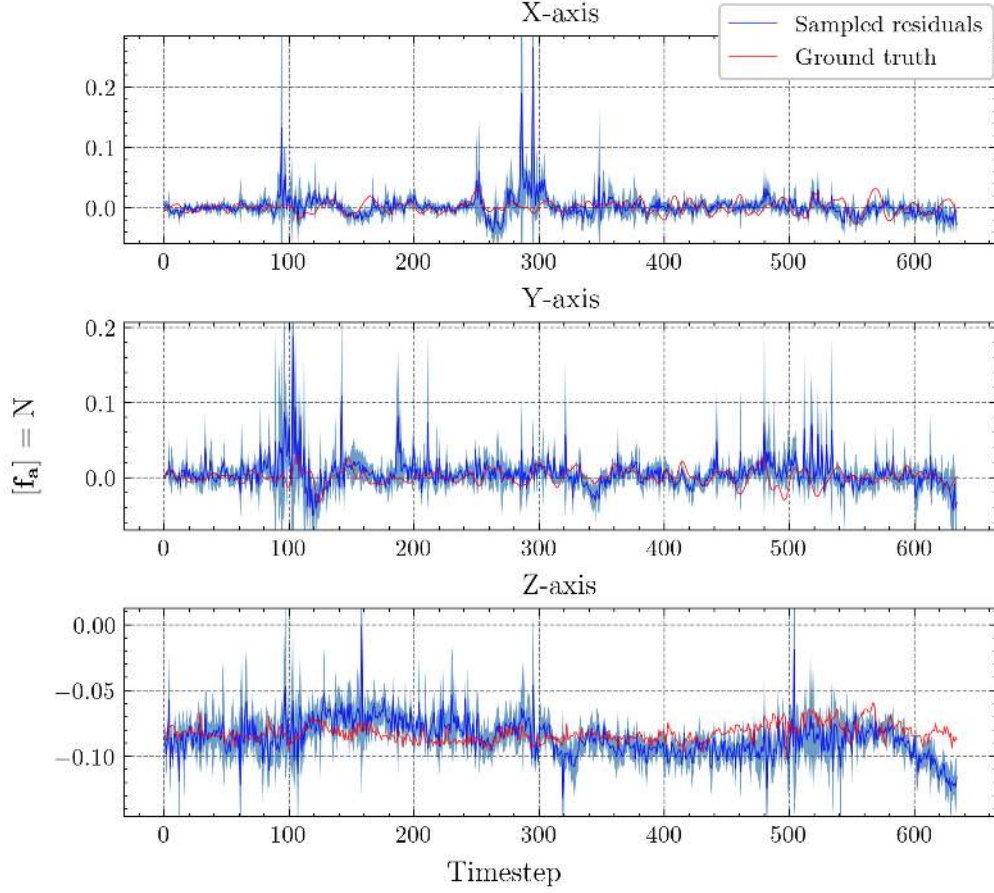


Figure 9.16: Prediction of the residual forces $\mathbf{f}_{a2}$ for one test flight. Each timestep was sampled $n = 5$ times using the diffusion model for a team of 2 UAVs. The datasets used involved the conditioning set *reduced regression conditioning* and the dataset with smoothing. The plot shows the mean residual forces for each timestep as well as the one **STD** confidence band. The sampling frequency is 100 Hz.

Another exception from other trends is the significantly lower MSE values achieved with non-smoothed data, approximately 1.72 (or 1.11 for MSE_n), in contrast to about 0.73 (or 0.57 for MSE_n) when smoothed data is used for the 3 UAV team paired with *modified state conditioning*. Although not representative for all vectors residual force, Figure 9.17 demonstrates this effect of low prediction errors for the X-axis and Y-axis.

When comparing the distributions of a single cluster on smoothed data, the observations generally do not deviate significantly from clusters of non-smoothed data. However, in some cases, the visualized ground truth residual distributions are constricted to small regions as depicted in Figure 9.19 and discussed in Section 9.3.2. In this case, the better

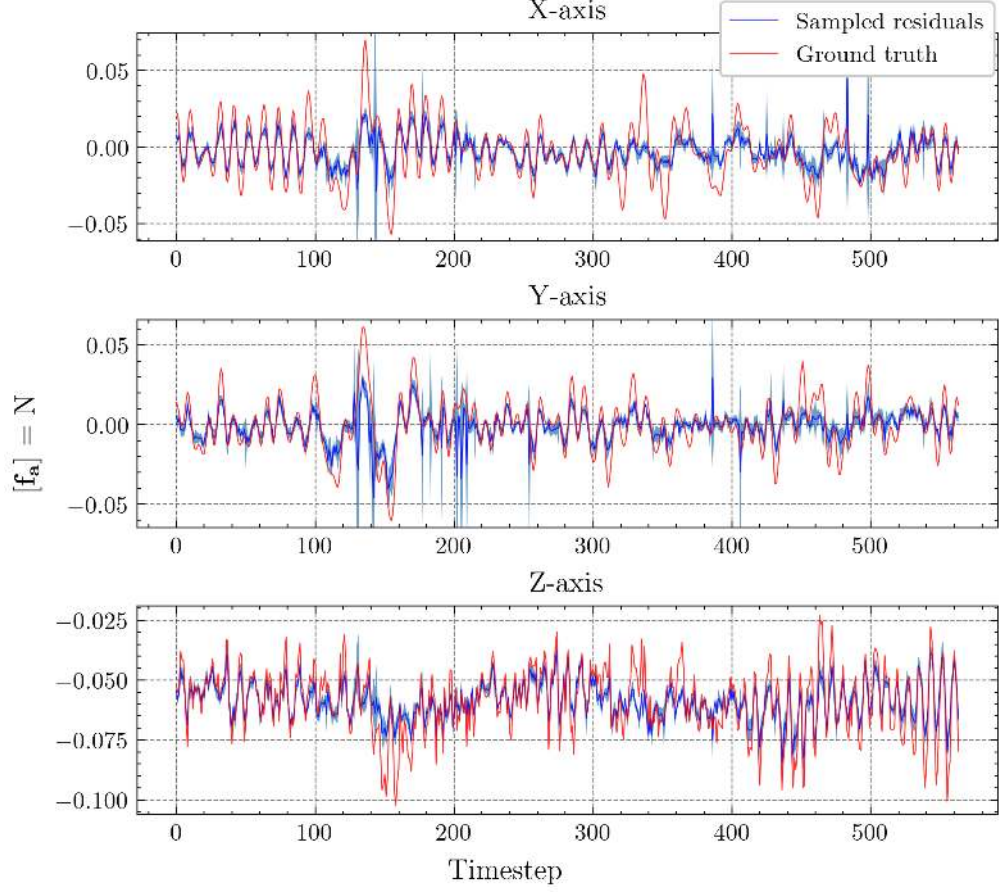


Figure 9.17: Prediction of the residual forces $\mathbf{f}_{a1}$ for one test flight. Each timestep was sampled $n = 5$ times using the diffusion model for a team of 3 UAVs. The datasets used involved the conditioning set *reduced regression conditioning* and the dataset with smoothing. The plot shows the mean residual forces for each timestep as well as the one **STD** confidence band. The sampling frequency is 100 Hz.

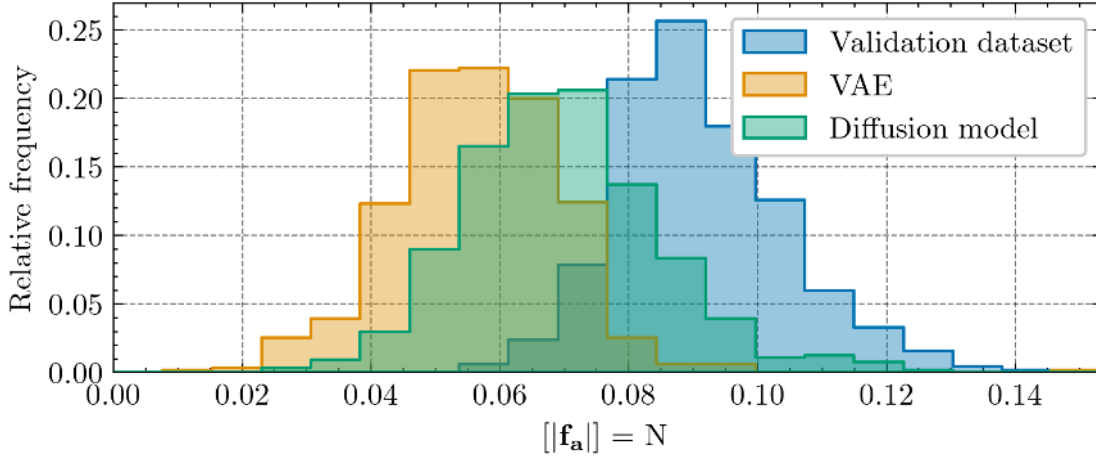


Figure 9.18: Distribution of the magnitudes of residual forces $|f_{a1}|$ in the validation dataset and the corresponding sampled residual forces by the generative models after sampling $n = 5$ times for each state without applying average. The distributions show the relative frequency of residual force magnitudes with the sum of all bins of each distribution equals 1, but high magnitudes are not shown for better visualization. The data corresponds to smoothed data and a team of 2 UAVs. The generative models were trained and validated using *modified state conditioning*.

performance of the diffusion model over the **VAE** can also be observed in replicating residual distributions.

Further evidence for this is illustrated in Figure 9.18. Although both generative models capture the shape of the distribution of residual force magnitudes with biases towards smaller values, the bias of the diffusion model is smaller than that of the **VAE**.

Most importantly, the visualizations using smoothed data supports the trend of inferior performance using less conditioning, which is the case for *modified state conditioning*.

In summary, the conflicting results do not allow for a final answer to the question of how the choice of different sets of conditioning affects the residual distributions for both smoothed and non-smoothed data, apart from the fact that comparing *full regression conditioning* consistently performs better than *modified state conditioning*. Because of this, there is little evidence for a possible theory on how the change of conditioning features change the underlying residual distributions or the general structure of the residual distributions in itself. However, there are hints that the residual distributions found in clustering become less uniform with less conditioning information, that might relate to an increase in difficulty of capturing such residual distributions. But in regards to the structure and possible multimodality of residual distributions in the provided data, the limited expressiveness of the visualizations using scatter plots and single clusters makes final verdicts questionable. Only the **MMD** metrics seem to give a good approximation of the fitness of generative models to accurately capture the residual distributions.

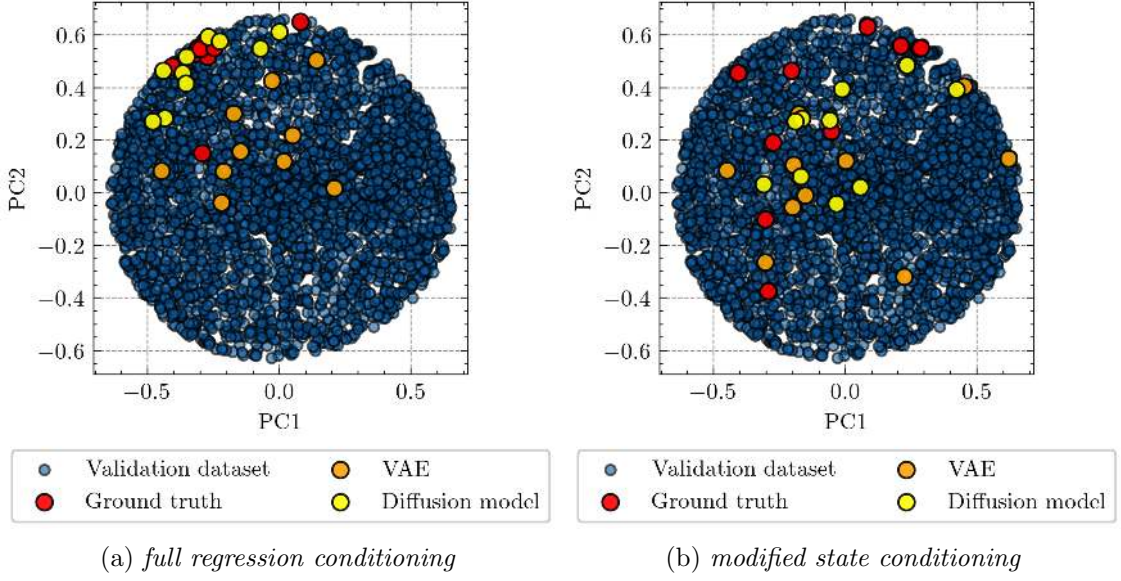


Figure 9.19: Visualization of the average residual forces using different sets of conditioning after sampling for $n = 5$ for each state using two dimensional [kernel PCA](#). The plots show the residual forces $\mathbf{f}_{a1}$ for a team of 3 UAVs and data with smoothing

9.5 Comparison of different residual force components

Although there are generally few differences for the individual components $\mathbf{f}_{ai}$ of the complete residual forces vector of the [MPS](#), some models show different behavior for the components, often showing accurate predictions for single components, while other components show high errors. The example shown in Figure [9.20](#) illustrates prediction errors that are several magnitudes higher than the ground truth values. Despite this example using *reduced regression conditioning*, the magnitude prediction error cannot be attributed to changes in the residual distributions and the physical principles, but rather to issues within the training process.

9.6 Discussion

The evaluation studying different effects with respect to the choice of generative model, number of UAVs used in the [MPS](#), smoothed data and the choice of conditioning features managed to show various trends, with implications for the underlying higher-level questions about the quality of the datasets, nominal models and the approach of capturing conditional residual distributions in general.

However, the high dimensionality that applies for all sets of conditioning features and the space of residual distributions allow for too many combinations to evaluate all possibilities, making it hard to account for interdependencies, leading to lower evidence

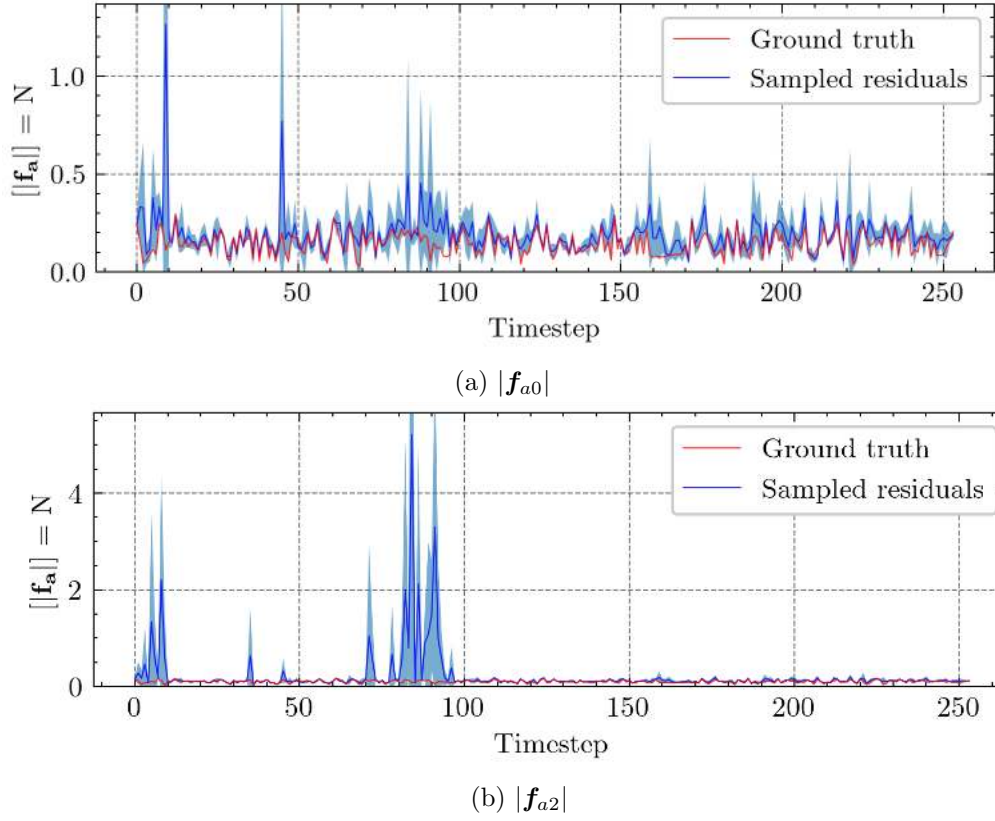


Figure 9.20: Comparison of the magnitudes of the residual forces $|f_{a0}|$ and $|f_{a2}|$ predicted by the diffusion model for a segment representing 40% of the entire test flight for a team of 2 UAVs using non-smoothed data and *reduced regression conditioning* for conditioning.

9 Evaluation

of trends and therefore limited expressiveness regarding their implications. Similarly, the stochastic nature of many components involved, such as the training processes of generative models, certain steps in clustering itself, or the random choice of clusters to calculate metrics or compare visually, add to the problem of finding causes to these trends. As a consequence, the metrics that calculate the averages in multiple clusters or residual forces give a good estimate of the general performance of the generative models when compared against each other. Nevertheless, the abstract values of the metrics make it difficult to build an actual intuition about the performance of a single model. To some extent, this is achieved using plots for single test flights, comparing sampled means to ground truth residual forces values along with confidence bands showing the sampling variance. These confidence bands even showed indications for the ability to calculate prediction uncertainty in at least one instance, such as for the best performing diffusion model visualized in Figure 9.1. Since the visualizations of residual distributions of clusters sampled from the KD-Tree and using kernel PCA for dimensionality reduction can often not replicate or even contradict the trends shown in the metrics using the **MMD**, the evidence for trends in the replication of residual distributions is generally low. Nevertheless, they are the only tools in the validation pipeline that provide insight to the residual distributions as conditional probability distributions sampled by generative models as well as ground truth distributions in the datasets themselves, which is crucial to explain the possibly contradictory trends that show in the metrics that use **MMD** and **MSE**.

Being aware of these limitations of metrics and visualizations the general trends can be summarized in regards to the different variables. Although the signals of oscillating residual forces with high frequencies suggested that smoothing is beneficial to find more uniform distributions of residual forces for similar states within a small time frame, both generative models struggle learning the underlying distributions of the smoothed datasets. One possible reason for this is that by smoothing the conditioning features over time the distributions of states change too much, so that the distributions in the validation datasets become too distinct from the training dataset. As a consequence, the inferior performance of the generative models shows a limited generalization capability. However, since the performance of the diffusion model, in particular, is already good learning the distribution of the residual forces function using not smoothed data, there is no need to smooth the data from a pure learning perspective. Nevertheless, given it is uncertain if the distribution of states leading to high-frequency oscillations of residual forces present in all datasets are accurate measurements, smoothing could still be beneficial from this point of view. This is similar to the question of whether, after smoothing, the conditioning features resemble states that the **MPS** actually reaches. Since this cannot be verified using the provided data, exploring this issue is beyond the scope of this thesis.

The effect of the choice of different sets of conditioning features is even more difficult to analyze given the contradicting metrics. The overall best performance was achieved using conditioning features not required to evaluate the residual forces function. This might indicate the generative models suffering from the so-called *clever hans effect*,

which is a problem in machine learning that occurs when a machine learning model exploits unintended patterns in the training data and therefore produces correct results even though it does not work as intended [53]. However, since the full information in *full regression conditioning* contains velocities of the payload and UAVs, and therefore implicit information from previous states. As a consequence, the good performance using *full regression conditioning* may not be solely explained by the clever hans effect alone, but rather by the possibility of the MPS not being markovian.

Exploring this is difficult using only the provided datasets, as the same exploitable patterns in the training datasets might be present in the validation datasets, explaining the good validation scores. While it would be possible to further test the markov property in regards to the residual distributions, by using more explicit prior state information using the entirety of the prior state introduces even more conditioning information, which would require even more test data to give meaningful results for clustering similar states. Since in the case of the metrics relying on clustering, the effect of shifting residual distributions is attributed to the KD-Tree returning different sets of states and possibly completely different ground truth residual distributions because of the changing values of distance between states using the state space metric. The distributions of close states and therefore the residual distributions are also greatly impacted by the choice of this metric and the available data. More conditioning information adds information that leads to more distinction between states. In extreme cases, this leads to an equal distance of all points in very high-dimensional spaces as found in [16], leading to potentially less meaningful clustering results, as actual patterns become harder to detect.

However, comparing the distributions of residual force magnitudes over the complete validation dataset instead of conditioned on single clusters give insight about the capabilities of capturing residual distributions that do not face this issue of shift in ground truth distributions in clustering. Instead, the comparison consistently indicated, that potentially poor performing generative models also struggle in replicating the joint distribution of residual forces for all states in the validation dataset. Furthermore, analysis of residual force magnitudes revealed that while the residual distributions for a given state condition could be multimodal, the combined residual forces magnitudes follow an approximately Gaussian distribution.

The significantly lower predictive performance across all metrics using the *minimal regression conditioning* that resembles the combined state and motor forces of a MPS under the markovian assumption. This inferior performance for predicting conditional residual distribution indicates that the markov property does not hold for the definition of the system state using *modified state conditioning*. Instead, the better performance of using conditioning information containing features of the states derivative suggest, that concurrent states of a MPS could be better described by a Markov chain with memory, and therefore indicate, that prior states are of importance for predicting the next state of the MPS.

Although the implications of the choice of conditioning or smoothing are difficult to grasp and possibly require further research, some effects like the inferior performance linked to a higher number of UAVs in the MPS suggest, that the interactions become

9 Evaluation

more complex for bigger UAV teams.

Similarly, the most notable impact on performance in all evaluated variables is the choice of generative models. Diffusion models seem to be a much better tool for predicting both residual distributions as well as single residual forces. However, the considerably shorter average inference time of the [VAE](#) might be a reason to favor it over the diffusion model. The *Lee controller* used in the training data collection process uses a cascaded attitude and position controller running at 250 Hz and 100 Hz respectively [\[54\]](#), [\[55\]](#) which corresponds to 0.004 s and 0.01 s, which could be interpreted as the time available for predicting a single batch of residual forces for the given state. Although absolute values depend on the computational resources available and implementation details as discussed in Section [7.3](#), it is questionable if real-time applications are possible using diffusion models, given that the absolute values are not matched, even with the substantial computational resources provided by the workstation used for training. However, the average inference time of the [VAE](#) are therefore more likely to be suitable in real-time applications.

10 Conclusion

This chapter summarizes the key findings and insights gained from this research, the challenges encountered, and the implications for future work in the field of enhancing multirotor systems with cable-suspended payloads. In addition, it outlines potential directions for further research.

10.1 Summary

This research investigated the viability of using generative models to enhance first-principle physics models of a multirotor system with a cable-suspended payload with a hybrid approach by accounting for learned higher-order dynamics of the generative models. For this, the approach of learning residual dynamics was used to compute residual force utilizing data from repeated test flights of four different reference trajectories and teams of two and three UAVs, yielding approximations of the actual residual distribution the **MPS** encounters in typical problems. To learn these underlying distributions **VAEs** and diffusion models were implemented and compared with respect to their general applicability in real-world scenarios. Further evaluations on the various effects of using smoothed data versus raw, but noisy data or the effects of choosing different sets of input features used for conditional sampling were conducted.

To address the high complexity of the **MPS** in regards to the number of dimensions of the space of state space features and the corresponding residual force being a major challenge in evaluating the effects, different metrics and tools for quantitative and qualitative analysis of the performance of the developed generative models were employed. Nearest-neighbor search has been used to find clusters of similar states and corresponding residual distribution, to benchmark the generative model not only for their fitness as a regression model, but also for approximating underlying residual distributions for a single given state.

This required the definition a metric that is suitable for being used as a state space metric as well as being used when only part of the full information is used as conditioning information for the residual distributions. Here, an initial approach of using the standardized Euclidean distance on a modified state space using Euler angles instead of rotation matrices made it possible to find clusters of similar states using KD-Trees and visualize them as well as their corresponding residual distributions using **PCA**. While this approach potentially gives good insight on the distribution of residual force, the so queried clusters of similar states suggest a highly multimodal ground truth residual distributions for non-smoothed data, aligning with the high frequency oscillation of residual forces encountered during test flights.

10.2 Key findings

The general comparison of **VAEs** and diffusion models found the diffusion models to be superior in all metrics regarding their performance in capturing residual distributions and giving best estimates for residual forces. Especially the diffusion model demonstrated sampling variance to be suitable as a measure of uncertainty for residual prediction. Exploring the compensation of residual dynamics depending on the prediction uncertainty in both certain and uncertain environments could therefore improve the stability of the in certain environments without endangering the system in uncertain ones.

Assuming that the non-smoothed data captures all higher-order dynamics, the analysis of the relative frequency of magnitudes of residual forces over the complete validation dataset showed an approximate unimodal distribution for residual forces defined for both payload and UAVs. The general finding that the lower **MSE** loss scores of applying the arithmetic mean to multiple times sampled residual forces suggest that the residual forces could indeed be unimodal for most states, seemingly independent of the choice of conditioning information. However, the oscillations of residual forces as a function of time, along with some visualizations using plots for individual clusters indicate, that the overall unimodality of residual force magnitudes might not be an accurate approximation. Instead, the non-smoothed data might be regarded as multimodal.

While smoothing the features of state space features of the test flight data with respect to time indicated more uniform residual distributions and removed the oscillations of residual forces of high frequency, it remains unclear if the obtained datasets provide better representations of the states which the **MPS** is likely to encounter in real-world scenarios.

Assuming the found residual distributions indeed capture distributions that could be encountered in real flights, the lack of patterns found in these residual distributions for single states do not provide valuable insight in itself, other than the common finding that the residual forces are distributed almost randomly and for example do not correlate with more or less configurations of the **MPS**, as the high frequency parts in the residual force signals over time and visualization of single clusters indicate.

Related to this is the inherent problem of the high dimensional data in regards to the conditioning information and number of residual forces dependent on the UAV team size. The proposed state space metric allows for a flexible amount of conditioning features. However, equal weighting of each feature may impose unwanted effects on clustering and as a consequence, of the structure of the associated residual distributions of a given cluster. The metrics proposed using **MSE** and **MMD** for the evaluation of the performance of generative models capturing residual distributions allowed for a comparison

Similarly, while the visualization developed using scatter plots in conjunction with kernel PCA could provide insight into the patterns and the structure of residual distributions, these patterns showed strong inconsistencies. The reliance on comparing single clusters imposes sensitivity to outliers, making it difficult to give a final verdict about their expressiveness. However, some general trends indicated by the metrics are supported by the visualizations, providing additional evidence. The general purpose of

providing qualitative measures for the models' performance could therefore be achieved.

A common problem in machine learning lies in the substantial amount of data required to train robust models that perform well in areas outside of the distribution of training data. Considering the extensive range of **MPS** states in regard to relative positions, speeds, and flight trajectories and the expense of collecting such data, it is impossible to gather training data for all possible situations in real experiments. Consequently, the partitioning of the provided data in training and validation datasets could be insufficient to fully assess the performance in all possible situations, given the general similarity of the training flights conducted.

However, the general inferior performance of generative models when trained and validated against smoothed data with sufficient conditioning information for pure regression suggests that the generative models do not generalize well for states too dissimilar from the training data. Instead, the already very good prediction accuracy of the trained diffusion models against the validation data set using different reference trajectories than those of the training data set suggests that the oscillations of residual forces during test flights are likely not related to problems in the data collection, but in fact due to actual higher-order dynamics not sufficiently addressed by the nominal model, aligning with the observations of cable vibrations in previous work.

This result is a strong indication that a proposed hybrid model compensating for these residual force can improve flight stability, provided that enough conditioning information is available at all times. However, despite the high prediction accuracy of the diffusion models, they might not be adequate for real-time applications, as the tests suggest a lower supported sampling frequency than necessary for tasks such as residual compensation. **VAEs** on the other hand, allow for fast computation of many samples of lower prediction accuracy.

The evaluation of other factors such as the smoothing of conditioning features, choice of conditioning features and the underlying residual distributions themselves proved difficult, as there is often conflicting evidence between different trends. This difficulty stems in part from the novelty of the approach, limiting judgments about the general expressiveness of some of the metrics and visualization techniques used in this context.

Therefore, the approach, as well as the metrics and visualization techniques for residual distributions developed and results found, establish a solid foundation with significant potential for further research to expand and refine individual components. The initial results with respect to the test flight data provided suggest diffusion models to be promising tools for learning accurate representations of residual distributions, with the potential to improve accuracy and robustness of a cable-driven payload with multiple UAVs system beyond the current state of the art.

10.3 Outlook

The current findings regarding the markov property for the **MPS** can be investigated further by experiments that explicitly incorporate prior states as conditioning features.

Another promising way to further improve the generative models discussed, could

10 Conclusion

involve leveraging additional invariants, such as permutations of residual forces within the concatenated residual vector and UAVs-specific conditioning features. This could be achieved by employing permutation-invariant neural networks. Similar approaches showed remarkable results for learning complex aerodynamic interaction forces in swarms consisting of multiple UAVs, with particularly good generalization capabilities for larger swarms [3, 4].

Although the VAE demonstrated lower performance than that of the diffusion model across all other metrics, its significantly higher supported sampling frequency may be advantageous provided more training data and more computational power is available to support more complex models. Additionally, exploring the latent space representation learned by such models could be promising to find more descriptive lower dimensional encodings of the MPS system state, possibly providing more insight and beneficial for finding better measures of similarity of states. Similarly, the reduction in dimensionality for both residual forces as well as system states achieved by PCA in this thesis could be replaced by the encoder of an appropriately trained VAE. Latent space representation of VAEs for system states have already been demonstrated to be useful for motion planning [56, 57]. Therefore, employing VAEs for the MPS may prove useful for several applications in addition to learning residual distribution.

Bibliography

- [1] Wenshuai Zhao, Jorge Peña Queralta, and Tomi Westerlund. “Sim-to-Real Transfer in Deep Reinforcement Learning for Robotics: a Survey”. In: *2020 IEEE Symposium Series on Computational Intelligence (SSCI)*. 2020 IEEE Symposium Series on Computational Intelligence (SSCI). Dec. 2020, pp. 737–744. DOI: [10.1109/SSCI47803.2020.9308468](https://doi.org/10.1109/SSCI47803.2020.9308468).
- [2] Daichi Wada, Sergio Araujo-Estrada, and Shane Windsor. “Sim-to-Real Transfer for Fixed-Wing Uncrewed Aerial Vehicle: Pitch Control by High-Fidelity Modelling and Domain Randomization”. In: *IEEE Robotics and Automation Letters* PP (Sept. 9, 2022), pp. 1–8. DOI: [10.1109/LRA.2022.3205442](https://doi.org/10.1109/LRA.2022.3205442).
- [3] Guanya Shi et al. *Neural-Swarm: Decentralized Close-Proximity Multirotor Control Using Learned Interactions*. Mar. 5, 2020. DOI: [10.48550/arXiv.2003.02992](https://doi.org/10.48550/arXiv.2003.02992).
- [4] Guanya Shi et al. “Neural-Swarm2: Planning and Control of Heterogeneous Multirotor Swarms Using Learned Interactions”. In: *IEEE Transactions on Robotics* 38.2 (Apr. 2022). Conference Name: IEEE Transactions on Robotics, pp. 1063–1079. DOI: [10.1109/TR0.2021.3098436](https://doi.org/10.1109/TR0.2021.3098436).
- [5] Leonard Bauersfeld* et al. “NeuroBEM: Hybrid Aerodynamic Quadrotor Model”. In: *Robotics: Science and Systems XVII*. Robotics: Science and Systems 2021. Robotics: Science and Systems Foundation, July 12, 2021. DOI: [10.15607/RSS.2021.XVII.042](https://doi.org/10.15607/RSS.2021.XVII.042).
- [6] Daniel K. D. Villa, Alexandre S. Brandão, and Mário Sarcinelli-Filho. “A Survey on Load Transportation Using Multirotor UAVs”. In: *Journal of Intelligent & Robotic Systems* 98.2 (May 2020), pp. 267–296. DOI: [10.1007/s10846-019-01088-w](https://doi.org/10.1007/s10846-019-01088-w).
- [7] Sangheon Lee and Hungsun Son. “Antisway Control of a Multirotor With Cable-Suspended Payload”. In: *IEEE Transactions on Control Systems Technology* 29.6 (Nov. 2021), pp. 2630–2638. DOI: [10.1109/TCST.2020.3035004](https://doi.org/10.1109/TCST.2020.3035004).
- [8] Koushil Sreenath and Vijay Kumar. “Dynamics, Control and Planning for Cooperative Manipulation of Payloads Suspended by Cables from Multiple Quadrotor Robots”. In: *Robotics: Science and Systems IX*. Robotics: Science and Systems 2013. Robotics: Science and Systems Foundation, June 23, 2013. DOI: [10.15607/RSS.2013.IX.011](https://doi.org/10.15607/RSS.2013.IX.011).
- [9] Taeyoung Lee. “Geometric Control of Quadrotor UAVs Transporting a Cable-Suspended Rigid Body”. In: *IEEE Transactions on Control Systems Technology* 26.1 (Jan. 2018), pp. 255–264. DOI: [10.1109/TCST.2017.2656060](https://doi.org/10.1109/TCST.2017.2656060).

Bibliography

- [10] Taeyoung Lee. “Collision avoidance for quadrotor UAVs transporting a payload via Voronoi tessellation”. In: *2015 American Control Conference (ACC)*. 2015 American Control Conference (ACC). Chicago, IL, USA: IEEE, July 2015, pp. 1842–1848. DOI: [10.1109/ACC.2015.7171001](https://doi.org/10.1109/ACC.2015.7171001).
- [11] Khaled Wahba and Wolfgang Hönig. *Efficient Optimization-based Cable Force Allocation for Geometric Control of a Multirotor Team Transporting a Payload*. Jan. 3, 2024. arXiv: [2304.02359](https://arxiv.org/abs/2304.02359).
- [12] Jana Schicke. “Learning-based Multirotor System Model Enhancements”. In: (2023). URL: <https://imrclab.github.io/assets/pdf/schicke2023bsthesis.pdf>.
- [13] Rafael Kelly, Victor Santibáñez, and Antonio Loría. *Control of robot manipulators in joint space*. Advanced textbooks in control and signal processing. London [Heidelberg]: Springer, 2005. 426 pp. DOI: [10.1007/b135572](https://doi.org/10.1007/b135572).
- [14] Khaled Wahba et al. *Kinodynamic Motion Planning for a Team of Multirotors Transporting a Cable-Suspended Payload in Cluttered Environments*. Oct. 5, 2023. arXiv: [2310.03394](https://arxiv.org/abs/2310.03394).
- [15] David J. Weller-Fahy, Brett J. Borghetti, and Angela A. Sodemann. “A Survey of Distance and Similarity Measures Used Within Network Intrusion Anomaly Detection”. In: *IEEE Communications Surveys & Tutorials* 17.1 (2015). Conference Name: IEEE Communications Surveys & Tutorials, pp. 70–91. DOI: [10.1109/COMST.2014.2336610](https://doi.org/10.1109/COMST.2014.2336610).
- [16] Charu C. Aggarwal, Alexander Hinneburg, and Daniel A. Keim. “On the Surprising Behavior of Distance Metrics in High Dimensional Space”. In: *Database Theory — ICDT 2001*. Ed. by Jan Van Den Bussche and Victor Vianu. Red. by Gerhard Goos, Juris Hartmanis, and Jan Van Leeuwen. Vol. 1973. Series Title: Lecture Notes in Computer Science. Berlin, Heidelberg: Springer Berlin Heidelberg, 2001, pp. 420–434. DOI: [10.1007/3-540-44503-X_27](https://doi.org/10.1007/3-540-44503-X_27).
- [17] Elena Deza and Michel Marie Deza. *Encyclopedia of Distances*. Berlin, Heidelberg: Springer Berlin Heidelberg, 2009. DOI: [10.1007/978-3-642-00234-2](https://doi.org/10.1007/978-3-642-00234-2).
- [18] Zhi-Hua Zhou. *Machine Learning*. Singapore: Springer Singapore, 2021. DOI: [10.1007/978-981-15-1967-3](https://doi.org/10.1007/978-981-15-1967-3).
- [19] Zakaria Jaadi and Brennan Whitfield. *When and Why to Standardize Your Data*. Built In. Aug. 4, 2023. URL: <https://builtin.com/data-science/when-and-why-standardize-your-data>.
- [20] Gintare Karolina Dziugaite, Daniel M. Roy, and Zoubin Ghahramani. *Training generative neural networks via Maximum Mean Discrepancy optimization*. May 14, 2015. arXiv: [1505.03906](https://arxiv.org/abs/1505.03906).
- [21] Arthur Gretton et al. “A Kernel Two-Sample Test”. In: *Journal of Machine Learning Research* 13.25 (2012), pp. 723–773. ISSN: 1533-7928.

- [22] Arthur Gretton. “The Maximum Mean Discrepancy for Training Generative Adversarial Networks”. In: (). URL: <http://www.gatsby.ucl.ac.uk/~gretton/papers/cardiff.pdf>.
- [23] Alfred Müller. “Integral Probability Metrics and Their Generating Classes of Functions”. In: *Advances in Applied Probability* 29.2 (June 1997), pp. 429–443. DOI: [10.2307/1428011](https://doi.org/10.2307/1428011). (Visited on 07/20/2024).
- [24] Sashank J. Reddi et al. *On the Decreasing Power of Kernel and Distance based Nonparametric Hypothesis Tests in High Dimensions*. Nov. 23, 2014. arXiv: [1406.2083](https://arxiv.org/abs/1406.2083).
- [25] Christopher M. Bishop and Hugh Bishop. *Deep Learning: Foundations and Concepts*. Cham: Springer International Publishing, 2024. DOI: [10.1007/978-3-031-45468-4](https://doi.org/10.1007/978-3-031-45468-4).
- [26] Michael W. Berry, Azlinah Mohamed, and Bee Wah Yap, eds. *Supervised and Unsupervised Learning for Data Science*. Unsupervised and Semi-Supervised Learning. Cham: Springer International Publishing, 2020. DOI: [10.1007/978-3-030-22475-2](https://doi.org/10.1007/978-3-030-22475-2).
- [27] Samreen Naeem et al. “An Unsupervised Machine Learning Algorithms: Comprehensive Review”. In: *International Journal of Computing and Digital Systems* 13.1 (Apr. 16, 2023), pp. 911–921. DOI: [10.12785/ijcds/130172](https://doi.org/10.12785/ijcds/130172).
- [28] Sunil Arya et al. “An optimal algorithm for approximate nearest neighbor searching fixed dimensions”. In: *J. ACM* 45.6 (Nov. 1, 1998), pp. 891–923. DOI: [10.1145/293347.293348](https://doi.org/10.1145/293347.293348).
- [29] Jim Jeffers, James Reinders, and Avinash Sodani. “Machine learning”. In: *Intel Xeon Phi Processor High Performance Programming*. Elsevier, 2016, pp. 527–548. DOI: [10.1016/B978-0-12-809194-4.00024-7](https://doi.org/10.1016/B978-0-12-809194-4.00024-7).
- [30] Sanparith Marukatat. “Tutorial on PCA and approximate PCA and approximate kernel PCA”. In: *Artificial Intelligence Review* 56.6 (June 2023), pp. 5445–5477. DOI: [10.1007/s10462-022-10297-z](https://doi.org/10.1007/s10462-022-10297-z).
- [31] Quan Wang. *Kernel Principal Component Analysis and its Applications in Face Recognition and Active Shape Models*. Aug. 30, 2014. DOI: [10.48550/arXiv.1207.3538](https://doi.org/10.48550/arXiv.1207.3538).
- [32] Sushanth Sreenivasa. *Radial Basis Function (RBF) Kernel: The Go-To Kernel — by Sushanth Sreenivasa — Towards Data Science*. Dec. 10, 2020. URL: <https://towardsdatascience.com/radial-basis-function-rbf-kernel-the-go-to-kernel-acf0d22c798a>.
- [33] Nimisha Singh. “Unlocking the Power of RBF Kernel: Advantages, Local Boundaries, Gamma’s Influence”. Medium. Nov. 2, 2023. URL: <https://pub.aimind.so/unlocking-the-power-of-rbf-kernel-advantages-local-boundaries-gammas-influence-2aebafce336a>.

Bibliography

- [34] Jakub M. Tomczak. *Deep Generative Modeling*. Cham: Springer International Publishing, 2022. DOI: [10.1007/978-3-030-93158-2](https://doi.org/10.1007/978-3-030-93158-2).
- [35] Sheng-Jhih Wu and Moody T. Chu. “Markov chains with memory, tensor formulation, and the dynamics of power iteration”. In: *Applied Mathematics and Computation* 303 (June 2017), pp. 226–239. DOI: [10.1016/j.amc.2017.01.030](https://doi.org/10.1016/j.amc.2017.01.030).
- [36] Wojciech Giernacki et al. “Crazyflie 2.0 quadrotor as a platform for research and education in robotics and control engineering”. In: *2017 22nd International Conference on Methods and Models in Automation and Robotics (MMAR)*. 2017 22nd International Conference on Methods and Models in Automation and Robotics (MMAR). Aug. 2017, pp. 37–42. DOI: [10.1109/MMAR.2017.8046794](https://doi.org/10.1109/MMAR.2017.8046794).
- [37] Aris Perperoglou. “A review of spline function procedures in R.” In: (2019).
- [38] Pauli Virtanen et al. “SciPy 1.0: fundamental algorithms for scientific computing in Python”. In: *Nature Methods* 17.3 (Mar. 2, 2020), pp. 261–272. DOI: [10.1038/s41592-019-0686-2](https://doi.org/10.1038/s41592-019-0686-2).
- [39] Vyas Ramasubramani. *rowan 1.3.0.post1 : Perform quaternion operations using NumPy arrays*. 2020. URL: <https://github.com/glotzerlab/rowan>.
- [40] Guanya Shi et al. *Neural Lander: Stable Drone Landing Control using Learned Dynamics*. Mar. 4, 2019. DOI: [10.1109/ICRA.2019.8794351](https://doi.org/10.1109/ICRA.2019.8794351).
- [41] Ian Goodfellow et al. “Generative adversarial networks”. In: *Communications of the ACM* 63.11 (2020), pp. 139–144. DOI: [10.1145/3422622](https://doi.org/10.1145/3422622).
- [42] Carl Doersch. *Tutorial on Variational Autoencoders*. Jan. 3, 2021. DOI: [10.48550/arXiv.1606.05908](https://doi.org/10.48550/arXiv.1606.05908).
- [43] Kihyuk Sohn, Honglak Lee, and Xinchen Yan. “Learning Structured Output Representation using Deep Conditional Generative Models”. In: *Advances in Neural Information Processing Systems*. Vol. 28. Curran Associates, Inc., 2015.
- [44] Konstantin Soifeikov. *Implementing conditional variational auto-encoders(CVAE) from scratch — by Konstantin Soifeikov — Medium*. Apr. 26, 2023. URL: <https://medium.com/@soifeikov/implementing-conditional-variational-auto-encoders-cvae-from-scratch-29fcbb8cb08f>.
- [45] Irina Higgins et al. “beta-VAE: Learning Basic Visual Concepts with a Constrained Variational Framework”. In: *International Conference on Learning Representations*. 2017. URL: <https://openreview.net/forum?id=Sy2fzU9gl>.
- [46] Rushikesh Shende. *Autoencoders, Variational Autoencoders (VAE) and beta-VAE*. Medium. Apr. 20, 2023. URL: <https://medium.com/@rushikesh.shende/autoencoders-variational-autoencoders-vae-and-%CE%B2-vae-ceba9998773d>.
- [47] Adam Paszke et al. *PyTorch: An Imperative Style, High-Performance Deep Learning Library*. Dec. 3, 2019. arXiv: [1912.01703](https://arxiv.org/abs/1912.01703).
- [48] Jonathan Ho, Ajay Jain, and Pieter Abbeel. *Denoising Diffusion Probabilistic Models*. Dec. 16, 2020. arXiv: [2006.11239](https://arxiv.org/abs/2006.11239).

- [49] Stanley H. Chan. *Tutorial on Diffusion Models for Imaging and Vision*. Mar. 26, 2024. arXiv: [2403.18103](#).
- [50] Jonathan Ho and Tim Salimans. *Classifier-Free Diffusion Guidance*. July 25, 2022. arXiv: [2207.12598\[cs\]](#). URL: <http://arxiv.org/abs/2207.12598>.
- [51] Fabian Pedregosa et al. “Scikit-learn: Machine Learning in Python”. In: *Journal of Machine Learning Research* 12.85 (2011), pp. 2825–2830. ISSN: 1533-7928.
- [52] Onur Tunali. *Maximum Mean Discrepancy (MMD) in Machine Learning*. Aug. 3, 2019. URL: <https://www.onurtunali.com/ml/2019/03/08/maximum-mean-discrepancy-in-machine-learning.html#implementation-of-mmd>.
- [53] Jacob Kauffmann et al. *The Clever Hans Effect in Unsupervised Learning*. Aug. 15, 2024. arXiv: [2408.08041](#).
- [54] Bitcraze. *Controllers in the Crazyflie — Bitcraze*. URL: <https://www.bitcraze.io/documentation/repository/crazyflie-firmware/master/functional-areas/sensor-to-control/controllers/> (visited on 08/28/2024).
- [55] Taeyoung Lee, Melvin Leok, and N. Harris McClamroch. “Geometric tracking control of a quadrotor UAV on SE(3)”. In: *49th IEEE Conference on Decision and Control (CDC)*. 49th IEEE Conference on Decision and Control (CDC). ISSN: 0191-2216. Dec. 2010, pp. 5420–5425. DOI: [10.1109/CDC.2010.5717652](#).
- [56] Brian Ichter, James Harrison, and Marco Pavone. “Learning Sampling Distributions for Robot Motion Planning”. In: *2018 IEEE International Conference on Robotics and Automation (ICRA)*. 2018 IEEE International Conference on Robotics and Automation (ICRA). May 2018, pp. 7087–7094. DOI: [10.1109/ICRA.2018.8460730](#).
- [57] Jung-Su Ha et al. “Adaptive Path-Integral Autoencoders: Representation Learning and Planning for Dynamical Systems”. In: *Advances in Neural Information Processing Systems*. Vol. 31. Curran Associates, Inc., 2018. URL: https://proceedings.neurips.cc/paper_files/paper/2018/hash/aa0d2a804a3510442f2fd40f2100b054-Abstract.html.

Appendix

1 Ablation studies tables

Dim.	Layers	Nodes	MMD	MMD _{small}	MSE	MSE _n	Inf. time	Valid.
8	2	512	0.265	0.121	0.841	0.553	0.000339	14
8	2	1024	0.309	0.108	0.811	0.527	0.000526	31
8	2	2048	0.346	0.109	0.758	0.512	0.000435	16
16	2	512	0.192	0.096	0.970	0.567	0.000339	21
16	2	1024	0.293	0.105	0.941	0.547	0.000519	36
16	2	2048	0.258	0.134	0.973	0.584	0.000604	26
32	2	512	0.154	0.128	0.987	0.560	0.000344	28
32	2	1024	0.335	0.153	1.050	0.599	0.000345	59
32	2	2048	0.280	0.149	1.009	0.551	0.000413	16

Table 1: Results for the **VAE** for a team of 2 UAVs. *Dim.* is the number of latent space dimensions, *Layers* the number of hidden layers and *Nodes* the number of nodes per hidden layer. *Inf. time* is the average inference time calculated in seconds after sampling $n = 100$ batches of using a conditioning vector $\mathbf{c} = \mathbf{0}$. *Valid.* denotes the number of validations until the training was stopped.

Appendix

Dim.	Layers	Nodes	MMD	MMD _{small}	MSE	MSE _n	Inf. time	Valid.
8	1	1024	0.249	0.124	0.743	0.506	0.000689	49
8	2	1024	0.300	0.141	0.865	0.543	0.000902	29
8	3	1024	0.190	0.149	0.887	0.574	0.001032	17
8	1	2048	0.327	0.162	0.787	0.544	0.000871	20
8	2	2048	0.280	0.132	0.768	0.537	0.000934	17
8	3	2048	0.200	0.077	0.777	0.522	0.001217	27
8	1	4096	0.340	0.147	0.764	0.517	0.001250	37
8	2	4096	0.179	0.101	0.805	0.530	0.000947	28
8	3	4096	0.265	0.166	0.876	0.569	0.001377	33
16	1	1024	0.273	0.102	0.896	0.548	0.000281	37
16	2	1024	0.200	0.118	0.851	0.534	0.000526	34
16	3	1024	0.372	0.152	1.024	0.577	0.000601	20
16	1	2048	0.294	0.121	1.082	0.602	0.000512	40
16	2	2048	0.301	0.178	0.982	0.603	0.000604	11
16	3	2048	0.183	0.096	0.958	0.568	0.000523	13
16	1	4096	0.287	0.248	0.963	0.574	0.000806	23
16	2	4096	0.264	0.100	0.859	0.540	0.000947	14
16	3	4096	0.320	0.110	1.073	0.604	0.001232	20

Table 2: Results for the **VAE** with 2 UAVs (second iteration). *Dim.* is the number of latent space dimensions, *Layers* the number of hidden layers and *Nodes* the number of nodes per hidden layer. *Inf. time* is the average inference time calculated in seconds after sampling $n = 100$ batches of using a conditioning vector $\mathbf{c} = \mathbf{0}$. *Valid.* denotes the number of validations until the training was stopped.

Dim.	Layers	Nodes	MMD	MMD _{small}	MSE	MSE _n	Inf. time	Valid.
8	4	2048	0.304	0.114	0.835	0.532	0.000668	41
8	5	2048	0.224	0.106	0.825	0.540	0.000782	26
8	6	2048	0.151	0.082	0.833	0.533	0.001053	27
16	4	2048	0.349	0.145	0.968	0.575	0.001727	21
16	5	2048	0.226	0.140	1.010	0.581	0.001176	26
16	6	2048	0.256	0.119	0.930	0.550	0.001238	12

Table 3: Results for the **VAE** with 2 UAVs (High layer count). *Dim.* is the number of latent space dimensions, *Layers* the number of hidden layers and *Nodes* the number of nodes per hidden layer. *Inf. time* is the average inference time calculated in seconds after sampling $n = 100$ batches of using a conditioning vector $\mathbf{c} = \mathbf{0}$. *Valid.* denotes the number of validations until the training was stopped.

Dim.	Layers	Nodes	MMD	MMD _{small}	MSE	MSE _n	Inf. time	Valid.
8	2	512	0.329	0.282	1.414	0.959	0.000339	31
8	2	1024	0.466	0.302	1.444	0.986	0.000342	23
8	2	2048	0.416	0.168	1.374	0.943	0.000625	19
16	2	512	0.351	0.291	1.685	1.044	0.000342	55
16	2	1024	0.273	0.264	1.481	0.928	0.000727	39
16	2	2048	0.335	0.168	1.583	0.980	0.000899	49
32	2	512	0.452	0.273	1.706	1.049	0.000658	34
32	2	1024	0.295	0.233	1.621	0.997	0.000342	46
32	2	2048	0.304	0.188	1.624	0.962	0.000418	58

Table 4: Results for the **VAE** with 3 UAVs. *Dim.* is the number of latent space dimensions, *Layers* the number of hidden layers and *Nodes* the number of nodes per hidden layer. *Inf. time* is the average inference time calculated in seconds after sampling $n = 100$ batches of using a conditioning vector $\mathbf{c} = \mathbf{0}$. *Valid.* denotes the number of validations until the training was stopped.

Dim.	Layers	Nodes	MMD	MMD _{small}	MSE	MSE _n	Inf. time	Valid.
8	1	1024	0.429	0.248	1.457	0.946	0.0008	24
8	2	1024	0.326	0.177	1.391	0.933	0.0015	19
8	3	1024	0.380	0.277	1.377	0.934	0.0009	22
8	1	2048	0.376	0.236	1.391	0.946	0.0009	16
8	2	2048	0.401	0.193	1.345	0.928	0.0011	19
8	3	2048	0.397	0.223	1.425	0.952	0.0012	35
8	1	4096	0.323	0.167	1.321	0.898	0.0010	16
8	2	4096	0.280	0.156	1.302	0.877	0.0013	23
8	3	4096	0.325	0.128	1.426	0.965	0.0023	15
16	1	1024	0.383	0.277	1.572	0.993	0.0012	32
16	2	1024	0.364	0.240	1.587	0.995	0.0008	40
16	3	1024	0.336	0.192	1.548	0.973	0.0009	24
16	1	2048	0.470	0.322	1.616	1.017	0.0009	22
16	2	2048	0.323	0.205	1.559	0.995	0.0010	18
16	3	2048	0.387	0.158	1.576	0.994	0.0005	13
16	1	4096	0.421	0.230	1.599	1.001	0.0006	18
16	2	4096	0.374	0.176	1.529	0.967	0.0010	11
16	3	4096	0.393	0.237	1.573	0.984	0.0016	27

Table 5: Results for the **VAE** with 3 UAVs (Third iteration). *Dim.* is the number of latent space dimensions, *Layers* the number of hidden layers and *Nodes* the number of nodes per hidden layer. *Inf. time* is the average inference time calculated in seconds after sampling $n = 100$ batches of using a conditioning vector $\mathbf{c} = \mathbf{0}$. *Valid.* denotes the number of validations until the training was stopped..

Dim.	Layers	Nodes	MMD	MMD _{small}	MSE	MSE _n	Inf. time	Valid.
8	4	4096	0.299	0.139	1.347	0.938	0.0016	31
8	5	4096	0.333	0.165	1.411	0.958	0.0017	16
8	6	4096	0.405	0.130	1.390	0.924	0.0021	14
16	4	2048	0.306	0.181	1.643	1.014	0.0022	18
16	5	2048	0.302	0.183	1.586	0.969	0.0024	28
16	6	2048	0.335	0.154	1.567	0.992	0.0024	17

Table 6: Results for the **VAE** with 3 uav (Fourth iteration with more hidden layers). *Dim.* is the number of latent space dimensions, *Layers* the number of hidden layers and *Nodes* the number of nodes per hidden layer. *Inf. time* is the average inference time calculated in seconds after sampling $n = 100$ batches of using a conditioning vector $\mathbf{c} = \mathbf{0}$. *Valid.* denotes the number of validations until the training was stopped.

Layers	Nodes	Steps	MMD	MMD _{small}	MSE	MSE _n	Inf. time	Valid.
1	256	100	0.104	0.060	0.397	0.245	0.068	18
1	256	200	0.109	0.054	0.754	0.306	0.067	14
1	256	400	1.481	1.466	61.633	15.410	0.068	13
1	512	100	0.154	0.060	0.474	0.282	0.068	13
1	512	200	0.065	0.035	0.302	0.138	0.068	17
1	512	400	0.159	0.153	2.568	0.629	0.068	14
1	1024	100	0.104	0.030	0.253	0.163	0.068	16
1	1024	200	0.048	0.018	0.183	0.097	0.068	21
1	1024	400	0.038	0.028	0.391	0.151	0.069	30
1	2048	100	0.048	0.037	0.287	0.120	0.069	35
1	2048	200	0.204	0.206	1445.863	269.773	0.069	54
1	2048	400	0.032	0.018	0.221	0.077	0.069	31
2	256	100	0.073	0.032	0.232	0.154	0.085	25
2	256	200	0.059	0.065	1.170	0.219	0.085	34
2	256	400	0.044	0.020	0.455	0.151	0.084	30
2	512	100	0.054	0.033	0.219	0.132	0.085	27
2	512	200	0.039	0.020	2.134	0.498	0.086	26
2	512	400	1.221	0.757	3817.186	925.043	0.086	34
2	1024	100	0.076	0.039	0.246	0.161	0.085	15
2	1024	200	0.290	0.228	3214.409	273.427	0.086	30
2	1024	400	3.470	3.367	172021.672	42350.285	0.131	32
2	2048	100	0.048	0.028	0.196	0.111	0.087	33
2	2048	200	0.025	0.025	0.179	0.055	0.087	79
2	2048	400	2.737	2.514	4045.114	848.975	0.088	32

Table 7: Results for the diffusion model with 2 UAVs. *Layers* denotes the number of hidden layers, *Nodes* the number of nodes per hidden layer and *Steps* the number of denoising steps. *Inf. time* is the average inference time calculated in seconds after sampling $n = 100$ batches of using a conditioning vector $\mathbf{c} = \mathbf{0}$. *Valid.* denotes the number of validations until the training was stopped.

Appendix

Layers	Nodes	Steps	MMD	MMD _{small}	MSE	MSE _n	Inf. time	Valid.
1	512	100	0.086	0.033	0.208	0.142	0.069	39
1	512	200	0.044	0.022	0.167	0.088	0.082	42
1	1024	100	0.064	0.049	0.237	0.131	0.102	43
1	1024	200	0.034	0.039	0.204	0.081	0.103	34
2	512	100	0.066	0.037	0.248	0.133	0.122	45
2	512	200	0.254	0.207	4238.737	367.426	0.086	47
2	1024	100	0.052	0.039	0.218	0.115	0.086	58
2	1024	200	0.442	0.315	846.392	241.652	0.086	41
3	512	100	0.058	0.040	0.246	0.129	0.142	31
3	512	200	0.300	0.176	17.549	21.191	0.103	39
3	1024	100	0.055	0.022	0.247	0.123	0.102	36
3	1024	200	0.093	0.107	7.835	1.555	0.102	40

Table 8: Results for the diffusion model with 2 UAVs (second iteration). *Layers* denotes the number of hidden layers, *Nodes* the number of nodes per hidden layer and *Steps* the number of denoising steps. *Inf. time* is the average inference time calculated in seconds after sampling $n = 100$ batches of using a conditioning vector $\mathbf{c} = \mathbf{0}$. *Valid.* denotes the number of validations until the training was stopped.

Layers	Nodes	Steps	MMD	MMD _{small}	MSE	MSE _n	Inf. time	Valid.
1	2048	100	0.043	0.030	0.174	0.099	0.070	35
1	2048	200	0.153	0.153	891.148	1061.224	0.069	26
1	2048	400	0.045	0.024	0.188	0.071	0.069	20
2	2048	100	0.073	0.027	0.194	0.110	0.089	29
2	2048	200	0.018	0.017	0.193	0.058	0.089	43
2	2048	400	2.393	2.241	500.342	134.950	0.088	29
3	2048	100	0.080	0.036	0.232	0.131	0.107	18
3	2048	200	0.069	0.065	0.889	0.227	0.106	39
3	2048	400	1.213	1.059	61.158	13.791	0.106	40

Table 9: Results for the diffusion model with 2 UAVs (third iteration). *Layers* denotes the number of hidden layers, *Nodes* the number of nodes per hidden layer and *Steps* the number of denoising steps. *Inf. time* is the average inference time calculated in seconds after sampling $n = 100$ batches of using a conditioning vector $\mathbf{c} = \mathbf{0}$. *Valid.* denotes the number of validations until the training was stopped.

Layers	Nodes	Steps	MMD	MMD _{small}	MSE	MSE _n	Inf. time	Valid.
1	256	100	0.164	0.075	0.451	0.315	0.069	44
1	256	200	0.091	0.071	0.740	0.391	0.079	48
1	256	400	0.445	0.212	4.214	2.621	0.068	31
1	512	100	0.157	0.069	0.460	0.330	0.110	27
1	512	200	0.145	0.136	5.614	6.122	0.103	43
1	512	400	0.504	0.258	6.065	5.647	0.098	35
1	1024	100	0.159	0.083	0.408	0.297	0.068	32
1	1024	200	0.081	0.068	0.545	0.281	0.114	21
1	1024	400	0.561	0.430	11.060	8.030	0.115	28
1	2048	100	0.128	0.082	0.428	0.299	0.069	33
1	2048	200	1.055	0.681	767.680	626.445	0.146	47
1	2048	400	0.602	0.510	168.204	59.342	0.117	40
2	256	100	0.145	0.085	0.480	0.328	0.085	30
2	256	200	0.128	0.102	1.120	0.752	0.085	22
2	256	400	2.153	1.715	2542.174	617.447	0.154	46
2	512	100	0.106	0.054	92.921	114.769	0.145	54
2	512	200	0.147	0.130	2.020	2.583	0.139	29
2	512	400	0.611	0.471	18.023	10.396	0.09	23
2	1024	100	0.168	0.111	0.462	0.323	0.086	16
2	1024	200	0.101	0.113	5.041	3.932	0.157	21
2	1024	400	0.739	0.545	12.696	6.698	0.159	18
2	2048	100	0.108	0.072	0.507	0.311	0.171	24
2	2048	200	0.123	0.138	3.599	4.203	0.172	15
2	2048	400	0.683	0.511	251.740	47.378	0.172	22

Table 10: Results for the diffusion model with 3 UAVs. *Layers* denotes the number of hidden layers, *Nodes* the number of nodes per hidden layer and *Steps* the number of denoising steps. *Inf. time* is the average inference time calculated in seconds after sampling $n = 100$ batches of using a conditioning vector $\mathbf{c} = \mathbf{0}$. *Valid.* denotes the number of validations until the training was stopped.

Layers	Nodes	Steps	MMD	MMD _{small}	MSE	MSE _n	Inf. time	Valid.
1	2048	100	0.101	0.084	8.685	72.842	0.070	38
1	2048	200	0.843	0.674	1503.919	449.064	0.070	50
1	2048	400	2.486	2.527	2471.356	936.493	0.069	33
2	2048	100	0.104	0.059	1.044	0.827	0.088	19
2	2048	200	0.414	0.228	39.919	19.412	0.089	18
2	2048	400	3.153	3.411	1115.383	335.967	0.089	40
3	2048	100	0.098	0.057	0.412	0.215	0.159	27
3	2048	200	0.465	0.430	84.624	26.709	0.126	26
3	2048	400	1.155	0.897	17.300	10.703	0.106	13

Table 11: Results for the diffusion model with 3 UAVs (second iteration). *Layers* denotes the number of hidden layers, *Nodes* the number of nodes per hidden layer and *Steps* the number of denoising steps. *Inf. time* is the average inference time calculated in seconds after sampling $n = 100$ batches of using a conditioning vector $\mathbf{c} = \mathbf{0}$. *Valid.* denotes the number of validations until the training was stopped.

Layers	Nodes	Steps	MMD	MMD _{small}	MSE	MSE _n	Inf. time	Valid.
4	2048	100	0.147	0.085	0.470	0.294	0.124	12
4	4096	100	NaN	NaN	NaN	NaN	NaN	NaN
5	2048	100	0.059	0.051	0.309	0.144	0.140	35
5	4096	100	NaN	NaN	NaN	NaN	NaN	NaN
6	2048	100	0.169	0.123	0.565	0.383	0.158	19
6	4096	100	NaN	NaN	NaN	NaN	NaN	NaN

Table 12: Results for the diffusion model with 3 UAVs (third iteration with high layer count). *Layers* denotes the number of hidden layers, *Nodes* the number of nodes per hidden layer and *Steps* the number of denoising steps. *Inf. time* is the average inference time calculated in seconds after sampling $n = 100$ batches of using a conditioning vector $\mathbf{c} = \mathbf{0}$. *Valid.* denotes the number of validations until the training was stopped. *NaN* indicates that no model could be validated because of insufficient memory.

2 Crazyflie and payload parameters

- Mass of the payload m_0 : 0.01 kg
- Mass of the Crazyflie m_i : 0.0356 kg
- Cable length l_i : 0.5 m
- Moment of inertia of the Crazyflie J_i :

$$J_i = \begin{pmatrix} 16.571710 \times 10^{-6} & 0 & 0 \\ 0 & 16.655602 \times 10^{-6} & 0 \\ 0 & 0 & 29.261652 \times 10^{-6} \end{pmatrix}$$

- Actuation matrix B_0 :

$$B_0 = \begin{pmatrix} 1 & 1 & 1 & 1 \\ 0 & 0.046 & 0 & -0.046 \\ -0.046 & 0 & 0.046 & 0 \\ -0.006 & -0.006 & -0.006 & -0.006 \end{pmatrix}$$

- Gravitational acceleration (g): 9.81 m/s²